MorphoLibJ user manual

David Legland, Ignacio Arganda-Carreras

February 19, 2026

Contents

1. Overview **6**

2. Installation instructions **7**
2.1. Installation in ImageJ 7
2.2. Installation in Fiji 7

3. Definitions and data types **10**
3.1. Digital images 11
3.2. Coordinate system 12
3.3. Digital connectivity 13

4. Morphological filtering **15**
4.1. Principles 16
4.2. Grayscale morphological filters 16
4.3. Directional filters 20
4.4. Structuring elements 21
4.5. Plugins 22

5. Connected components operators **25**
5.1. Morphological reconstruction 26
5.2. Regional and extended extrema 29
5.3. Attribute filtering 32

6. Watershed segmentation **34**
6.1. Classic Watershed plugin 35
6.2. Marker-controlled Watershed plugin 37
6.3. Interactive Marker-controlled Watershed plugin 38
6.4. Morphological Segmentation plugin 42
6.5. Distance Transform Watershed 47

7. Measurements **51**
7.1. Region Analysis 52
7.2. Region Analysis 3D 65
7.3. Intensity measurements 70
7.4. Label Overlap measures 71
7.5. Microstructure analysis 72

8. Binary images 74
8.1. Distance transforms . . . 75
8.2. Geodesic distance transform . . . 81
8.3. Connected component labeling . . . 84
8.4. Binary images processing . . . 85

9. Label images 87
9.1. Visualization of label images . . . 88
9.2. Conversion to ROI . . . 90
9.3. Visualization of region features . . . 91
9.4. Label image processing . . . 92
9.5. Region adjacencies . . . 96
9.6. Label Edition plugin . . . 99

10. Library interoperability 100
10.1. Library organization . . . 100
10.2. Scripting MorphoLibJ . . . 101

Bibliography 103

A. List of operators 108

B. Computation of equivalent ellipsoid coefficients 110
B.1. Notations . . . 110
B.2. Computation of angles . . . 111
B.3. Computation of ellipsoid radius . . . 113

Abstract *The «MorphoLibJ» library for ImageJ/Fiji contains a variety of plugins for processing and analyzing 2D or 3D images. It was originally designed for the study of plant cell images, but the algorithms are generic and can be applied to any type of image.*

1. Overview

The library implements several functionalities that were missing in the ImageJ software (Schneider et al. (2012); Schindelin et al. (2012)), and that were not or only partially covered by other plugins.

- **morphological filtering** for 2D/3D and binary or grayscale images: erosion & dilation, closing & opening, morphological gradient & Laplacian, top-hat...
- **morphological reconstruction**, for 2D/3D and binary or grayscale images, allowing fast detection of regional or extended extrema, removing of borders, or hole filling
- **watershed segmentation** + GUI, making it possible to segment 2D/3D images of cell tissues
- **2D/3D measurements**: volume, surface area, bounding boxes, equivalent ellipse/ellipsoid...
- **several utilities** for the manipulation of binary and label images

The home page of the project is located at http://ijpb.github.io/MorphoLibJ/, and the source code can be found on GitHub: http://github.com/ijpb/MorphoLibJ. The exhaustive code documentation includes many use-case examples, and can be found online[1].

The reference publication for MorphoLibJ is Legland et al. (2016).

[1] http://ijpb.github.io/MorphoLibJ/javadoc/

2. Installation instructions

2.1. Installation in ImageJ

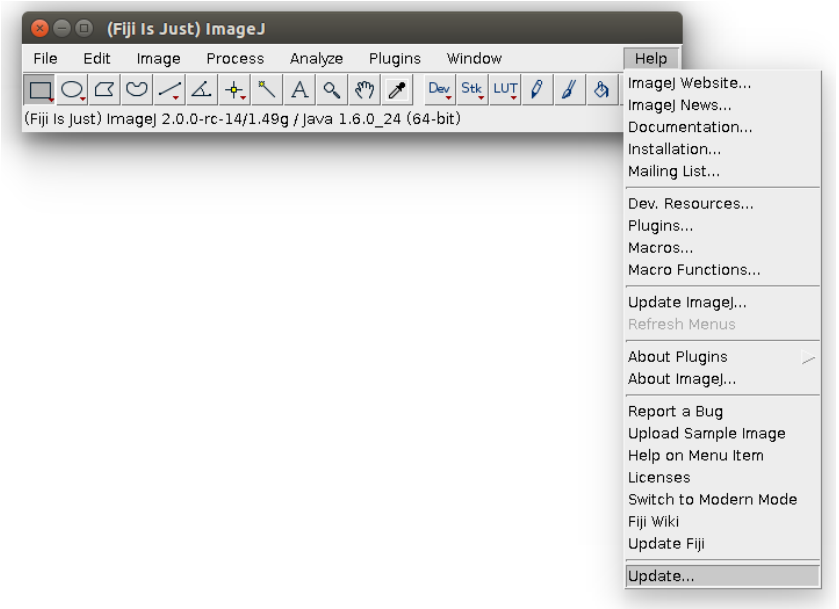
To install the MorphoLibJ toolkit in ImageJ, you only need to download the latest release (in the form of a JAR file) into ImageJ's plugins folder and restart ImageJ.

All released versions can be found at `https://github.com/ijpb/MorphoLibJ/releases`.

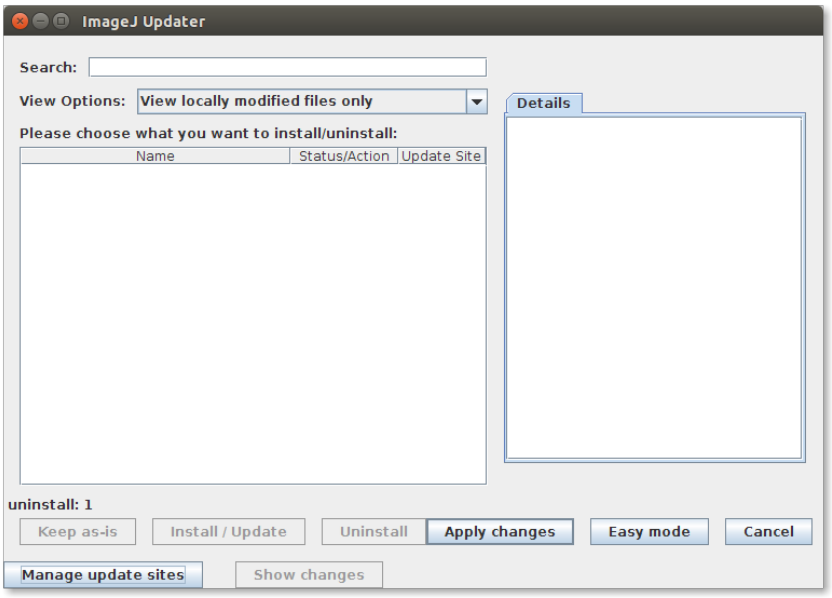
2.2. Installation in Fiji

If you use Fiji (Schindelin et al. (2012); Schneider et al. (2012)), the MorphoLibJ toolkit can be easily installed by adding its update site to Fiji's list of update sites:

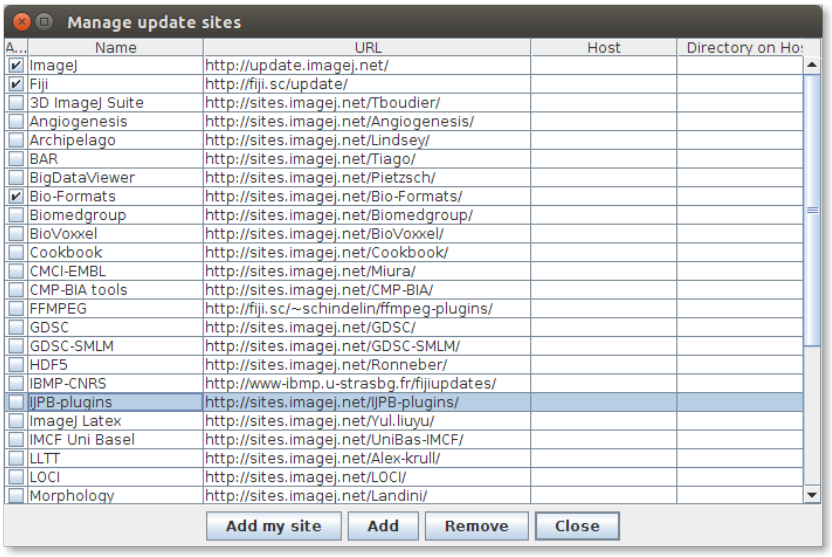
1. Open Fiji and select *Help* > *Update...* from the Fiji menu to start the updater.



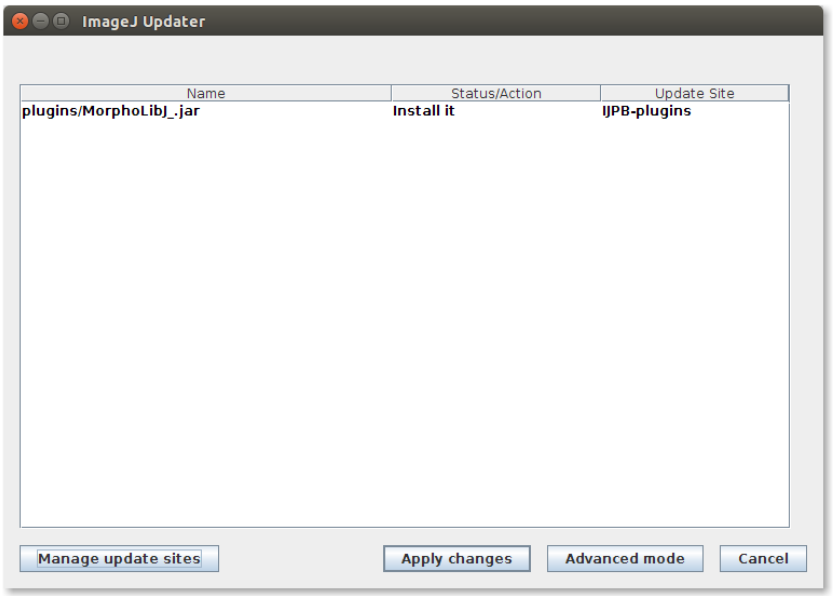
2. When the updater is open, click on *Manage update sites*.



3. This brings up a dialog where you can activate additional update sites:



4. Activate the MorphoLibJ update site and close the dialog. Now you should see an additional jar file for download:



5. Finally, click on *Apply changes* and restart Fiji. All the plugins should be available after restarting.

3. Definitions and data types

This chapter recalls some principles from the ImageJ software, summarizes the vocabulary and definitions used within the library, and presents some generic concepts about image processing and associated data structures.

Contents

3.1. Digital images **11**
3.1.1. Data types 11
3.1.2. Image types 11
3.1.3. 3D images 11
3.2. Coordinate system **12**
3.3. Digital connectivity **13**
3.3.1. 2D connectivities 13
3.3.2. 3D connectivities 14

3.1. Digital images

Digital images may be defined by a mapping from a pair (i, j) of integer coordinates to a value. The value may be an integer, a floating point number, a color... Image elements are usually refered to as "**pixels**", for "picture element".

3.1.1. Data types

Images are usually represented as 2D arrays of elementary data types. Within ImageJ, the pixels may be of different types, according to the bit depth:

- 8-bits images contains pixels with (positive) integer values between 0 and 255.
- 16-bits images contains pixels with (positive) integer values between 0 and $2^{16} = 65536$.
- 24-bits images corresponds to **color images**. Pixels are represented as triplet of red, green and blue components, each of them ranging between 0 and 255.
- 32-bits images contains pixels with floating-points values, possibly negative.

Note that other software may use additionnal data type for representation of images, such as 32-bits integers, complex values, booleans...

3.1.2. Image types

Apart from the data type / bit depth, image data may be interpreted in different ways. The MorphoLibJ library focusses on the following image types:

grayscale images contains pixels with positive integer values. They can correspond to either 8-bits or 16-bits images.

binary images are expected to contain only two possible values. Within ImageJ, they are represented as 8-bits images containing only two values: 0 for the background, and 255 for the foreground (See chapter 8).

intensity images may correspond to any types that can be interpreted as a single scalar value. They can be represented with 8-bits, 16-bits, or 32-bits images.

label images (also called **label maps**) contains values that can be interpreted as the label or index of a region (See chapter 9). The label 0 usually refer to the background (i.e., no region). Label images can be represented using 8-bits, 16-bits, or 32-bits images. Binary images can be seen as a special case of label images that contain only one label.

3.1.3. 3D images

Most algorithms from mathematical morphology (and more generally, many image processing algorithms) extends naturally from two dimensions to three dimensions. Within ImageJ, 3D images are represented as "**image stacks**", seen as stacks of 2D arrays. Elements of a 3D image are usually refered to as "**voxels**", for volumetric picture element.

3.2. Coordinate system

Images usually refer to data obtained on some physical sample. It is therefore necessary to be able to relate the (i, j) coordinates of a pixel to its physical location. Image pixel values may be seen as sampled on a discrete grid, defined by an origin (the coordinates of the $(0, 0)$ pixel) and a spacing (the width and the height of the pixel).

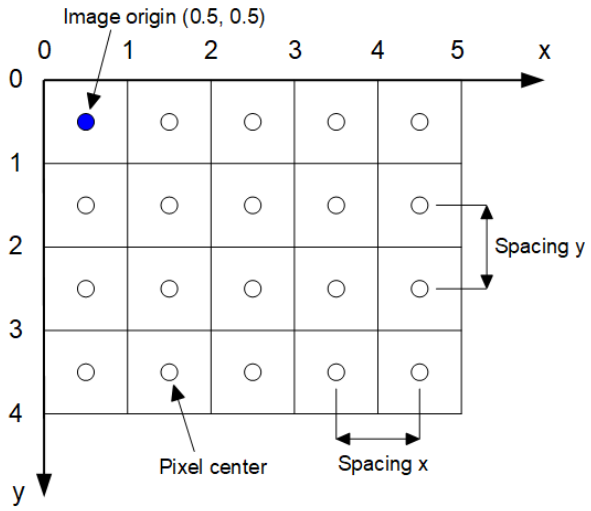


Figure 3.1.: *ImageJ coordinate system.*

Several software consider the pixel centers to be located on the grid vertices, i.e. pixels have integer coordinates. Within ImageJ, the origin pixel is located at the point $(0.5, 0.5)$, i.e. it occupies the space of the square with corners $(0, 0)$ and $(1, 1)$ (see Figure 3.1).

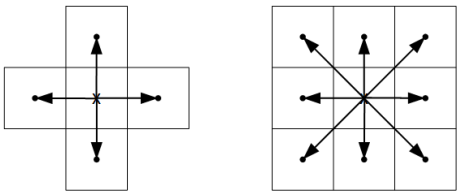
This notion of coordinate system is necessary to interpret the results obtained with region analysis algorithms (see section 7.1).

3.3. Digital connectivity

When reconstructing structures or regions from digital images, it is often necessary to define an **adjacency relationship**, or **connectivity**, between pixels of voxels (Rosenfeld, 1970; Serra, 1982; Kong & Rosenfeld, 1989; Soille, 2003). The notion of connectivity determines the result of many algorithms such as connected component labeling (see Section 8.3), or the result of image quantification based on Euler number (see Section 7.1.1.3).

3.3.1. 2D connectivities

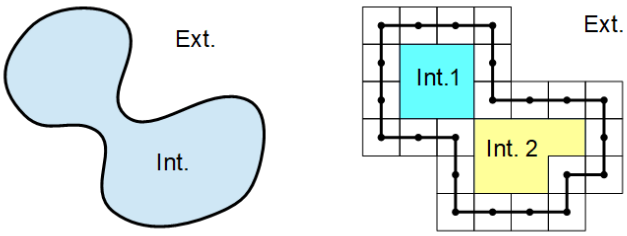
For planar images, typical choices are the 4- and the 8-connectivities. The 4-connectivity considers only the orthogonal neighbors of a given pixel (Figure 3.2-a). The 8-connectivity also considers the diagonal pixels (Figure 3.2-b).



(a) 4-connectivity **(b)** 8-connectivity

Figure 3.2.: *Digital connectivities for 2D images*

The discrete nature of images results in potential problems when considering geometric properties on reconstructed structures. A typical example is the Jordan curve theorem, which states than any curve of the plane that do not self-intersects divides the plane into exactly two regions: one interior and one exterior (Figure 3.3-a).



(a) Continuous curve **(b)** Digital curve using the 4-adjacency

Figure 3.3.: *Jordan curve theorem in continuous plane and in digital plane. The digitization of a 4-connected curve results in the creation of two inner regions that are not 4-connected.*

When choosing a digital connectivity and considering reconstructed curves, the theorem is not always true, as a 4-connected curve may create several disconnected interior regions (Figure 3.3-b). The common solution is to consider pairs of adjacencies, one for the foreground (here the curve), and one for the background. One can check that the two interior regions are 8-connected, making the Jordan curve theorem valid for the (4,8)-adjacency pair.

3.3.2. 3D connectivities

In 3D, two connectivities are commonly used. The 6-connectivity considers only orthogonal neighbors of the center voxel along the three principal directions (Figure 3.4-a). The 26-connectivity considers all the possible neighbors within a $3 \times 3 \times 3$ cube centered around the reference voxel (Figure 3.4-b). As for the 2D case, one often considers pairs of complementary adjacencies for the foreground and for the background (Kong & Rosenfeld, 1989; Ohser & Schladitz, 2009).

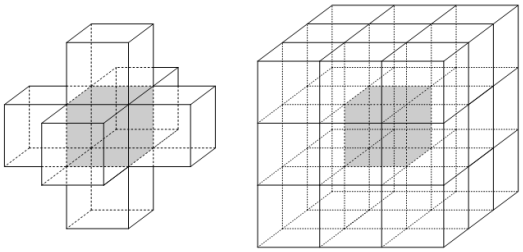


Figure 3.4.: *Digital connectivities for 3D images*

4. Morphological filtering

Morphological filters are very common filters that can be combined together to provide large variety of solutions. They are local filters, in the sense they consider the neighborhood of each pixel/voxel.

Contents

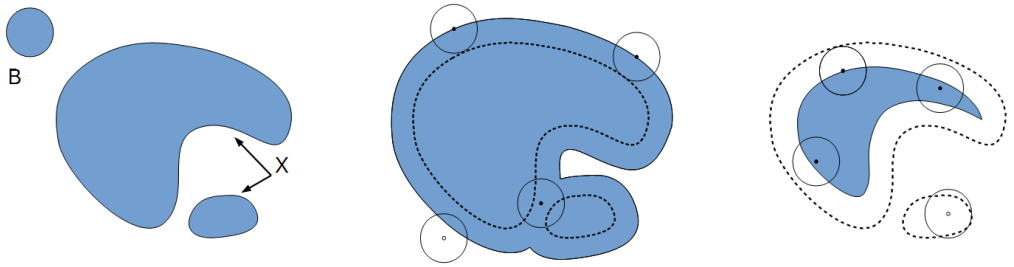
4.1. Principles **16**
4.2. Grayscale morphological filters **16**
4.2.1. Grayscale erosion and dilation 16
4.2.2. Opening and closing 17
4.2.3. Morphological gradients 18
4.2.4. Top-hats 19
4.3. Directional filters **20**
4.4. Structuring elements **21**
4.5. Plugins **22**
4.5.1. Planar images 22
4.5.2. 3D images 23
4.5.3. Directional Filters 23
4.5.4. Utility plugins 24

4.1. Principles

The original idea was to define a methodology to describe shapes by using another shape as test probe (Serra, 1982; Serra & Vincent, 1992). The test probe is usually referred to as **structuring element**. Common structuring element include squares or disks. Linear structuring element of various orientations may also be used to assess local orientation of the structures. The discretization of structuring element is discussed in section 4.4

The most basic morphological operators are the **morphological dilation** and the **morphological erosion**. The principle of morphological dilation is to test for each point of the plane, if the structuring element centred on this point **intersects** the structure of interest (Fig. 4.1-b). It results in a set larger than the original set.

The principle of morphological erosion is to test for each point of the plane if the structuring element centred on this point **is contained** within the original set. It results in a set smaller than original set (Fig. 4.1-c).



(a) Binary set X and structuring element B

(b) Dilation of X by B

(c) Erosion of X by B

Figure 4.1.: *Principle of morphological dilation and erosion on a binary set, using a disk-shaped structuring element.*

Morphological dilation and erosion change the size and the resulting set. It may also change its topology: after a dilation, components may merge and holes be filled. After an erosion, components may disappear, or components be separated into several parts.

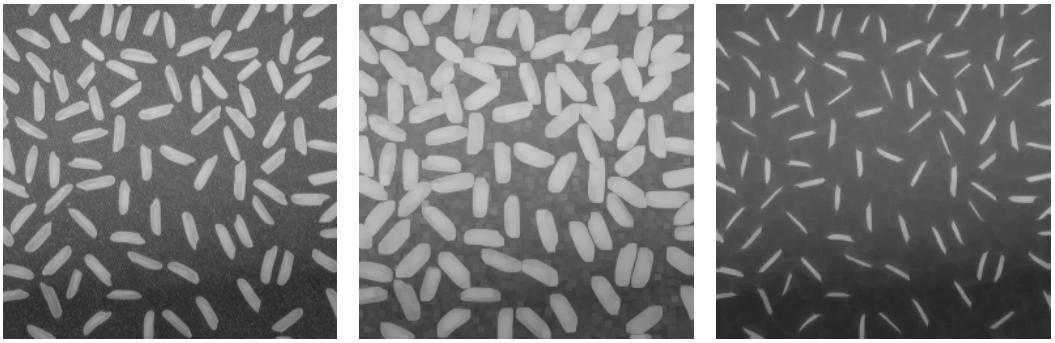
4.2. Grayscale morphological filters

While originally defined for binary sets, morphological operators are of great interest when applied to grayscale images. They can be easily be applied for noise reduction (section 4.2.2), detection of boundary structures (section 4.2.3), or background removal (section 4.2.4).

4.2.1. Grayscale erosion and dilation

Morphological erosion and dilation may also be applied on grayscale images. In that case, the morphological dilation computes for each pixel the maximum within its neighborhood

(defined by the structuring element), whereas the morphological erosion considers the minimum value within the neighborhood (Fig 4.2).



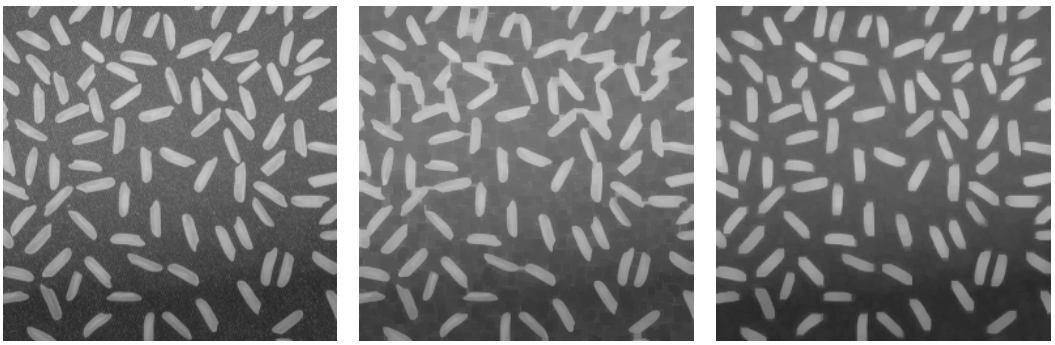
(a) Original image **(b)** Morphological dilation **(c)** Morphological erosion

Figure 4.2.: *Some examples of morphological filters on a grayscale image. Original image, result of dilation with a square structuring element, and result of erosion with the same structuring element.*

Applying a dilation or an erosion changes the size of the structures in the image: the grains in the result of the dilated image are larger. As for binary sets, these operations may also merge, separate, or make disappear some components of the image.

4.2.2. Opening and closing

Morphological dilation and erosion are often used in combination for **removing noise within images**. For example, the result of a dilation followed by an erosion is called a **morphological closing**, and removes dark structures smaller than the structuring element (Figure 4.3-b). It can also connect bright structures that were separated by a thin dark space.



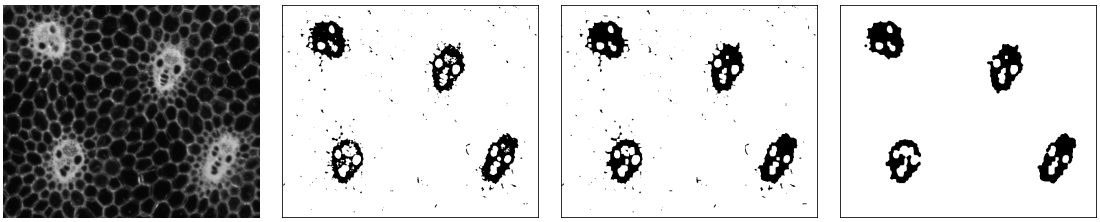
(a) Original image **(b)** Morphological closing **(c)** Morphological opening

Figure 4.3.: *Grayscale morphological closing and opening. (a) Original image. (b) Result of morphological closing. (c) Result of morphological opening.*

In a symmetric way, the result of an erosion followed by a dilation is called a **morphological opening**, and removes bright structures smaller than the structuring element (Figure 4.3-c).

Note that even if opening and closing preserve the size of the structures in the original image, their shape is slightly altered. For example, the morphological closing on Figure 4.3-b creates artificial connections between grains, whereas the morphological opening (Figure 4.3-c) tends to remove sharp corners of the grains. Choosing the best size for the structuring element is often a compromise between noise removal and preservation of shape structure. In some cases, attribute filtering operators (section 5.3) may provide better results than morphological filters.

Morphological opening and closing are also often used as **post-processing** operators to enhance segmentation results (Figure 4.4). Segmentation operators such as thresholding often results in many small components not corresponding to the structures of interest (Figure 4.4-b). Many holes may also be present within the structures.



(a) Original image **(b)** Raw segmentation **(c)** Morphological closing **(d)** Morphological opening

Figure 4.4.: *Enhancing segmentation with morphological filters. (a) Original image representing a section of maize tissue. (b) Binarisation by thresholding (inverted LUT). (c) Morphological closing removes most holes within the bundles (inverted LUT). (d) Morphological opening removes small dirts outside of vascular bundles (inverted LUT).*

The application of a morphological closing on the binary images allows to consolidate the structures of interest by removing small holes within them (Figure 4.4-c). Then, a morphological opening removes the small particles that do not correspond to the vascular bundles (Figure 4.4-d). The result is a binary image showing only the vascular bundles.

4.2.3. Morphological gradients

More complicated combinations of elementary operations can be used. The **morphological gradient** (also known as the **Beucher gradient**) is obtained as the difference of the result of a morphological dilation with the result of a morphological erosion, both computed with the same structuring element. It reveals the **boundaries** of the structures within the image (Figure 4.5-a). When applied to a binary image, the morphological gradient is a convenient way to obtain the boundaries of the structures of interest within the image (Figure 4.5-b).

The **morphological Laplacian** is defined as half the sum of a morphological dilation and a morphological erosion with the same structuring element, minus the original image. It results in enhancing the edges of the image (Figure 4.5-c).



(a) Morphological gradient

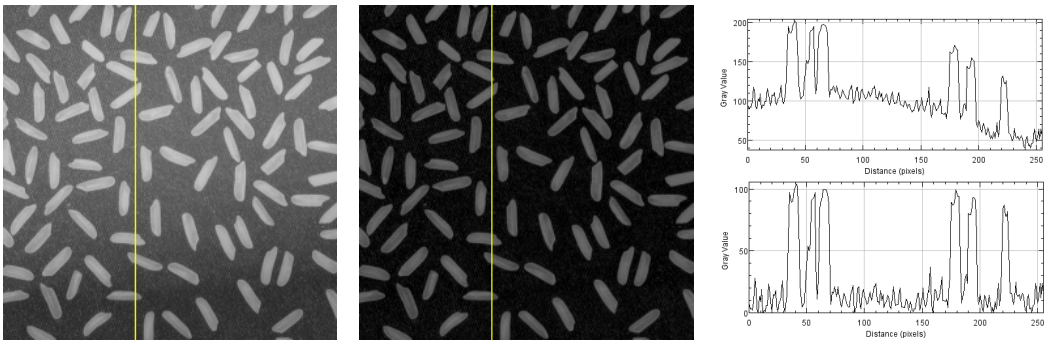
(b) Morphological gradient on a binary image

(c) Morphological Laplacian

Figure 4.5.: *Some examples of morphological filters resulting from the combination of elementary morphological filters. (a) Morphological gradient on a grayscale image. (b) Morphological gradient on the binary image represented in Figure 4.4-d (inverted LUT). (c) Morphological Laplacian on a grayscale image.*

4.2.4. Top-hats

The **white top-hat** operator first computes a morphological opening (resulting in removing bright structures smaller than structuring elements), and removes the result from the original image. When applied with a large structuring element, the result is an homogenization of the background, making bright structures easier to segment (Figure 4.6-c).



(a) Original image

(b) White Top-Hat

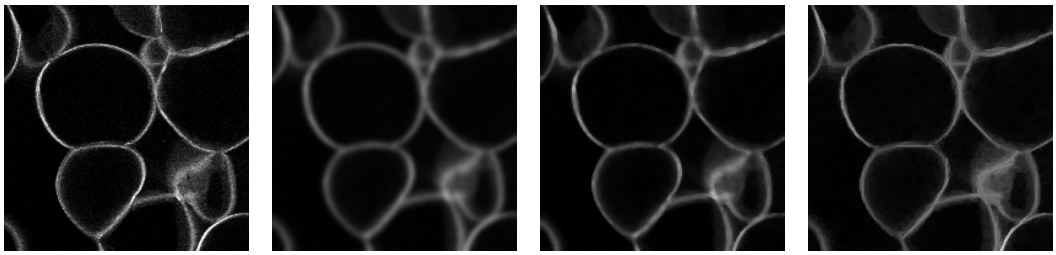
(c) Profile plots along vertical line

Figure 4.6.: *Top-hat filtering. (a) Original grayscale image, showing inhomogeneous background. (b) Result of the White Top-Hat operator using square with radius 20 as structuring element. (c) Profile plots along line region of interest for each image.*

Similarly, the **dark top-hat** can be used to enhance dark structures observed on an non-homogeneous background.

4.3. Directional filters

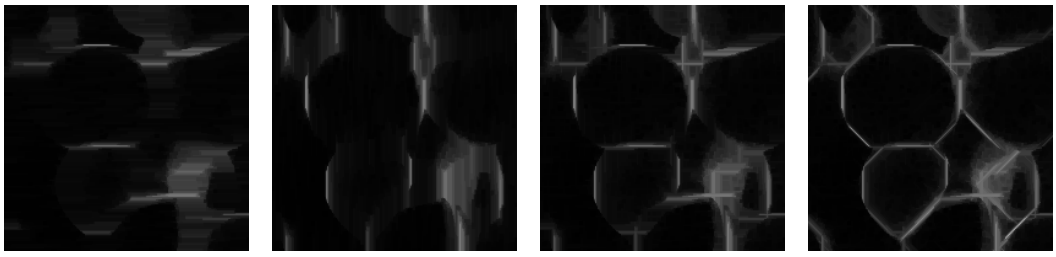
For images containing very thin curvilinear structures (for example blood vessels, cell wall sections...), the application of common filters may be difficult due to the small size of the structures. Even for small structuring elements, the application of a morphological opening or closing let the structure disappear (Fig. 4.7-b, 4.7-c). Moreover, it may be difficult to preserve the whole thickness of the structure.



(a) Original image **(b)** Gaussian filtering **(c)** Median filtering **(d)** Directional filtering

Figure 4.7.: *Filtering of a thin structure. (a) Original image representing apple cells observed with confocal microscopy (Legland & Devaux, 2009). The application of a Gaussian filter (b) or median filter (c) results in noise reduction, but also in a loss of the signal along the cell walls. The directional filtering (d) better preserves the thickness of the structure.*

An alternative is to apply directional filtering. The principle is to consider an oriented structuring element such as a line segment of a given length, and to perform morphological operations for various orientations of the structuring element (Soille & Talbot, 2001; Heneghan et al., 2002; Luengo Hendriks & van Vliet, 2003). For example, applying a median filter or a morphological opening with horizontal direction results in the enhancement of horizontal parts of bright structures (Fig. 4.8-a). Similarly, using a vertical structuring element results in the enhancement of the vertical portions of the structures (Fig. 4.8-b).



(a) Horizontal direction **(b)** Vertical direction **(c)** Two directions **(d)** Four directions

Figure 4.8.: *Principle of directional filtering of a thin structure. (a) and (b): result of median filter using an horizontal and a vertical linear structuring element. (c) and (d): combination of the results obtained from two directions (horizontal and vertical) and four directions (by adding diagonal directions).*

The results of oriented filters for each direction can be combined by computing the maximum value over all orientations (Fig. 4.7-d). Figures 4.8-c and 4.8-d show the results obtained when combining two or four directions. Here, 32 orientations of line with length 25 were used. This results in the enhancement of the image while preserving the thickness of the bright structures.

Similar results may be obtained for enhancing dark curvilinear structures, by using morphological closing or median filters, and combining the results by computing the minimum over all directions.

4.4. Structuring elements

Common structuring element include squares, discrete disks and octagons. Linear structuring element of various orientations may also be used to assess local orientation of the structures.

The size of the structuring element is often expressed with its **radius**. For symmetry reasons, structuring elements are centered on a center pixel or voxel. For square or disk structuring elements, the size therefore corresponds to the diameter, given as twice the radius plus the central pixel:

$$d = 2 \cdot r + 1$$

Structuring elements with “Disk” shape (and Ball in 3D) are generated based on the inner “RankFilters” computation class from ImageJ (see Figure).

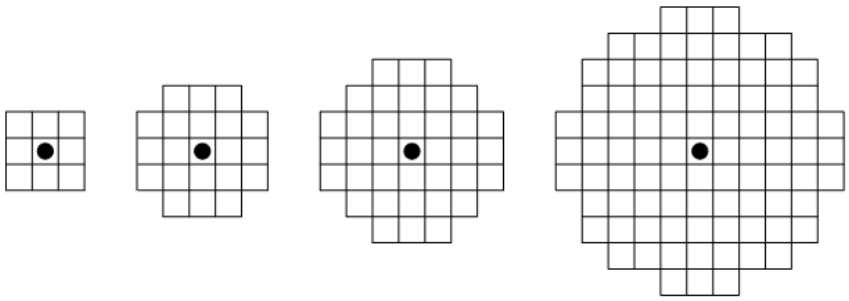


Figure 4.9.: *Disk structuring elements for radius values 1, 2, 3 and 5. The center pixel is indicated with a black dot.*

4.5. Plugins

The collection of morphological filters is available in the "Plugins ▸ MorphoLibJ" menu. Filters are implemented for both 2D and 3D images, and work for binary, grayscale or color (RGB) images.

4.5.1. Planar images

Morphological filters for planar images are available "Plugins ▸ MorphoLibJ ▸ Morphological filters". The dialog let the user choose the structuring element shape, radius, and eventually preview the result. The following operations can be chosen:

erosion keeps the minimum value within the neighborhood defined by the structuring element.

dilation keeps the maximum value within the neighborhood defined by the structuring element.

closing consists in the succession of a dilation with an erosion. Morphological closing makes dark structures smaller than the structuring element disappear.

opening consists in the succession of an erosion with a dilation. Morphological opening makes bright structures smaller than the structuring element disappear.

morphological gradient is defined as the difference of a morphological dilation and a morphological erosion with the same structuring element, and enhances edges of the original images.

morphological Laplacian is defined as half the sum of a morphological dilation and a morphological erosion with the same structuring element, minus the original image, and enhances edges of the image.

black top-hat consists in subtracting the original image from the result of a morphological closing, and results in the enhancement of dark structures smaller than structuring element.

white top-hat consists in subtracting the result of a morphological opening from the original image, and results in the enhancement of bright structures smaller than structuring element.

The following structuring elements can be used for 2D images:

- disk
- square
- octagon
- diamond
- line with angle of 0, 90, 45 or 135 degrees

4.5.2. 3D images

Morphological filters for 3D images are available under “Plugins ▸ MorphoLibJ ▸ Morphological filters (3D)”. The dialog let the user choose the structuring element shape and radius. The same list of operations as for planar images is provided. Planar structuring elements can be used (the operation is simply repeated on each slice), as well as a cubic or spherical structuring element. For most structuring elements, the size can be chosen for each direction.

4.5.3. Directional Filters

Directional filtering is available from “Plugins ▸ MorphoLibJ ▸ Directional Filtering”. It requires a planar image.

The parameters are:

Type to specify how to combine the results for each oriented filter

Operation the operation to apply using each oriented structuring element

Line Length the approximated length of the structuring element.

Direction Number the number of oriented structuring elements to consider. To be increased if the length of the line is large.

4.5.4. Utility plugins

Extend Image Borders

This plugin generates a new image larger than the original one, by padding border pixels with a specified strategy. Works for grayscale and color images, in 2D or 3D. The padding value can be specified for each direction. The different strategies for filling pixels can be:

Replicate — pads the values with the closest pixel from the original image (Fig. 4.10-b). This is often the default value for filtering operators.

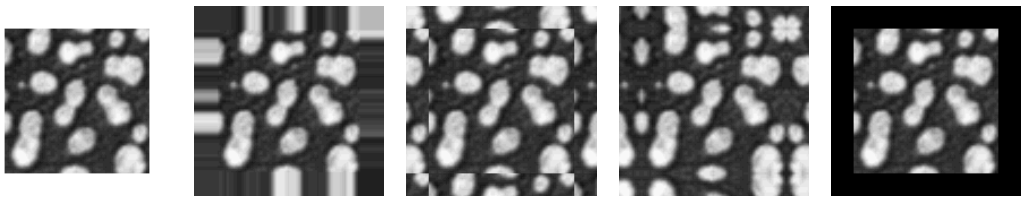
Periodid — pads the values by considering periodic boundaries: the pixels after the right border are retrieved from the left side of the image (Fig. 4.10-c)

Mirrored — pads the value by mirroring the image along each dimension (Fig. 4.10-d).

Black — pads the values with the value (or the color) corresponding to the black color (Fig. 4.10-e).

White — pads the values with the value (or the color) corresponding to the white color.

Grey — pads the values with the value (or the color) corresponding to the grey color.



(a) Original **(b)** Replicate **(c)** Periodic **(d)** Mirrored **(e)** Value (Black)

Figure 4.10.: *Illustration of the different strategies for generating a larger image by padding borders.*

5. Connected components operators

The "classical" morphological filters presented in chapter 4 transform an input image by using the values of pixels or voxels located in a close neighborhood, defined by the structuring element. Such filters can be seen as "local", as the result in a given position does not depend on image values located at a sufficient distance.

Connected components operators are more general as they propagate information within the image based on connectivity between pixels or voxels. More details can be found in the review of Breen & Jones (1996). Connected components operators encompass powerful operators, such as **morphological reconstruction** that allows to reconstruct a marker image by constraining it to a mask (section 5.1). An extension of morphological reconstruction is the detection of **extended minima and maxima**, that can be useful as marker detection for segmentation (section 5.2). Finally, **attribute opening and filtering** algorithms can filter images based on size or range properties, with better preservation of edges than classical filtering (section 5.3).

Contents

5.1. Morphological reconstruction 26
5.1.1. Principle 26
5.1.2. Applications to binary images 26
5.1.3. Applications to grayscale images 27
5.1.4. Application to the segmentation of chromocenters in a 3D image . 27
5.1.5. Usage 28
5.2. Regional and extended extrema 29
5.2.1. Regional minima and maxima 29
5.2.2. Extended minima and maxima 30
5.2.3. Minima or maxima imposition 30
5.2.4. Plugins usage 31
5.3. Attribute filtering 32
5.3.1. Application to binary images 32
5.3.2. Application to grayscale images 32
5.3.3. Usage 33

5.1. Morphological reconstruction

The morphological reconstruction is at the basis of many useful algorithms, such as border removing, hole filling, or detection of regional minima or maxima in grayscale images. It may be defined for binary as well as for grayscale images.

5.1.1. Principle

The principle of morphological reconstruction canbe illustrated by applying **conditional dilations or erosions** until idempotence. Conditional dilation is the result of a dilation, combined with a mask image using a logical operation. Conditional dilations are repeated until no more modification occur (idempotence condition).

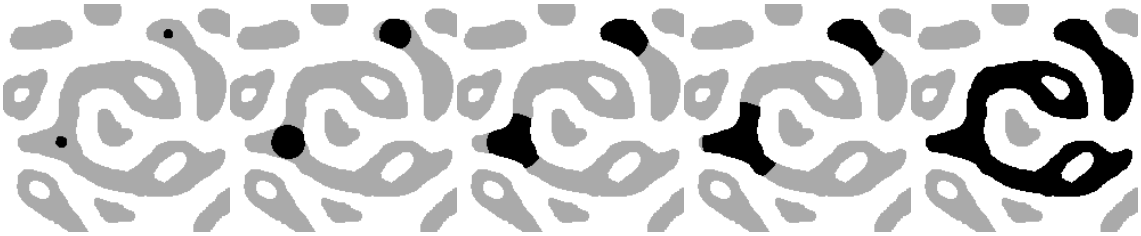


Figure 5.1.: *Principle of the morphological reconstruction algorithm. Original image is represented in gray, with marker and result of conditional dilations with increasing sizes superimposed in black.*

The figure 5.1 shows several steps of a morphological reconstruction by dilation on a pair of binary images. The mask image is shown in gray, and the marker image is shown in black on the first image. The reconstructed images at each step are shown in black. The markers propagates within the mask until an additional conditional dilation will not modify the image any more. The result is the set of regions that contains the markers, with the same shape as in the original image.

In practice, morphological reconstruction is often implemented using more efficient algorithms based on queues, by a adding pixels or voxels to the queue based on the connectivity with processed elements (see Section 3.3).

5.1.2. Applications to binary images

By choosing the marker image, several operations may be automatized. For example, computing morphological reconstruction with image of borders, and combining with original image will **remove particles or regions touching the borders** (Figure 5.2).

In a similar way, computing morphological reconstruction by using the border of the complement of the image makes it possible to **fill holes** that may appear in particles.

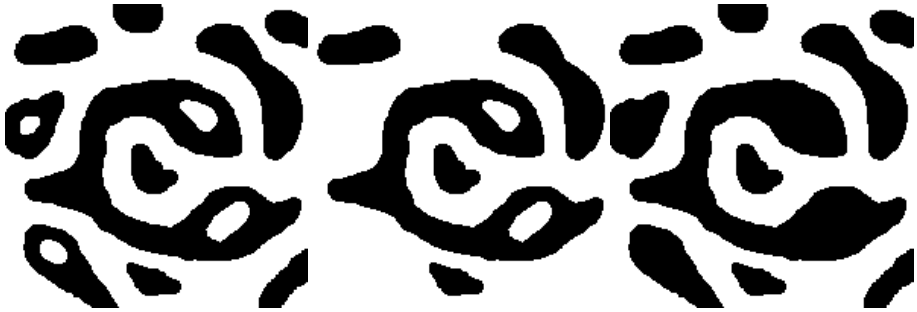


Figure 5.2.: *Some applications of morphological reconstruction. From left to right: original image, result of kill borders, result of fill holes.*

5.1.3. Applications to grayscale images

Morphological reconstructions can be applied to grayscale images. By manually choosing binary markers such that they overlay specific structures, and after applying a Morphological reconstruction by dilation, it is possible to obtain a grayscale image containing only the chosen structures (Figure 5.3). The algorithms for morphological reconstruction on grayscale images are discussed in Vincent (1993); Robinson & Whelan (2004).

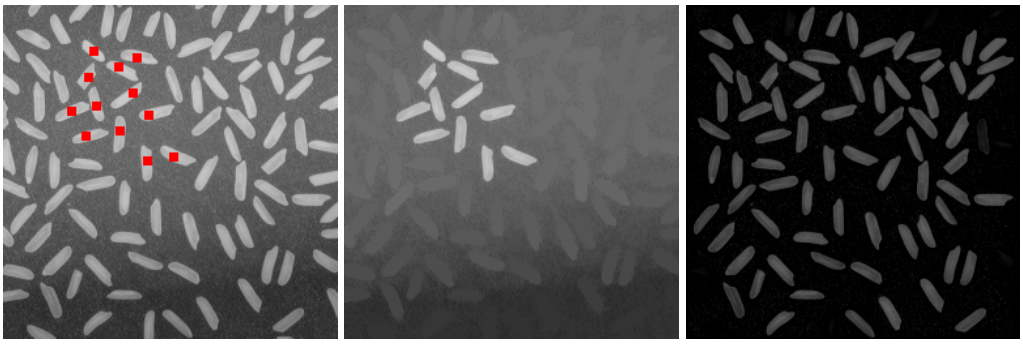


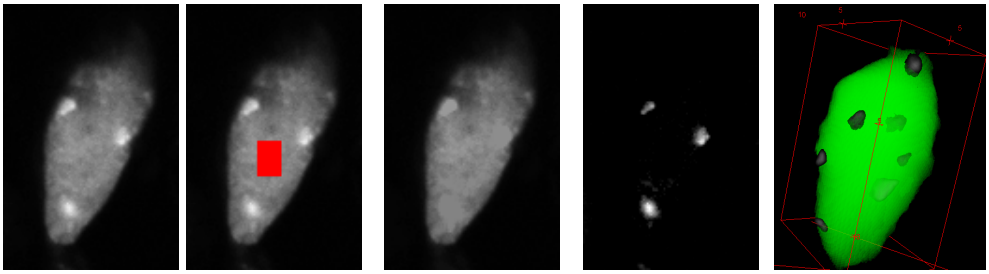
Figure 5.3.: *Some applications of morphological reconstruction on grayscale images. From left to right: original image with markers superimposed in red, result of morphological reconstruction by dilation, result of border kill operation.*

The **border kill** operation can also be applied on grayscale images, making possible to rapidly remove structures touching the image borders, while keeping the intensity information within the structures.

5.1.4. Application to the segmentation of chromocenters in a 3D image

An example of application is presented in figure 5.4. The original 3D image represents the nucleus of a plant cell, with chromocenters appearing as bright structures (Fig. 5.4-a). In order to discriminate the chromocenters, the nucleus is first identified by manually selecting a region of interest (Fig. 5.4-b) used as a marker for a morphological reconstruction with

the original image used as mask. The result of the morphological reconstruction retains the shape of the nucleus, while “clearing” the bright chromocenters (Fig. 5.4-c).



(a) (b) (c) (d) (e)

Figure 5.4.: *Application of 3D morphological reconstruction for isolating chromocenters within a nucleus. (a) Sample slice of a 3 image of nucleus with chromocenters represented as bright blobs. (b) Manual selection of a marker region within the nucleus. (c) Morphological reconstruction of 3D marker image using original 3D image as mask. (d) Difference of original image with the reconstruction. (e) 3D representation of the chromocenters within the nucleus. Image by courtesy of Kaori Sakai and Javier Arpon, INRA-Versailles.*

By computing the difference of the original image with the result of the morphological reconstruction, the chromocenters can be easily isolated and segmented (Fig. 5.4-d).

5.1.5. Usage

The reconstruction algorithm is often used within other operators. It is however provided as a plugin to allow its inclusion in user-designed macros or plugins:

Morphological Reconstruction

Computes the reconstruction by erosion or dilation using a marker image and a mask image, and a specified connectivity.

Interactive Morphological Reconstruction

Computes the reconstruction by erosion or dilation taking the current 2D image as mask image, creating the marker image out of user-defined ROIs (for example with the point selection tool) and using a specified connectivity. The plugin allows previewing the result.

Morphological Reconstruction 3D

Computes the reconstruction by erosion or dilation on a 3D image (marker and mask images are defined by the user among the open ones).

Interactive Morphological Reconstruction 3D

Computes the reconstruction by erosion or dilation on using the current 3D image as mask and creating the marker image from the user-defined point selections.

The kill borders and fill holes operations are also provided as plugins. Both work for 2D and 3D images of 8, 16 or 32 bit-depth images.

Kill Borders

Removes the particles touching the border of a binary or grayscale image. See also the Remove Border Labels Plugin (Section 9.4.1), that performs the same operation on label maps but without requiring reconstruction.

Fill Holes

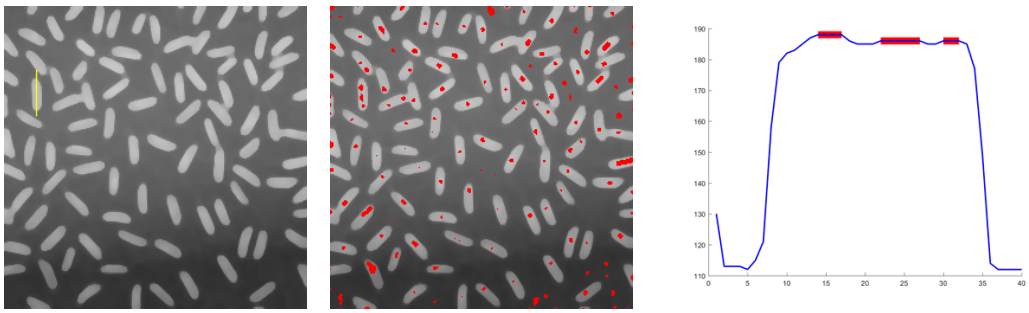
Removes holes inside particles in binary images, or remove dark regions surrounded by bright crests in grayscale images.

5.2. Regional and extended extrema

Minima and maxima within images are important features because they often correspond to relevant structures within the image. In mathematical morphology, the terms minima and maxima correspond to regions, i.e. they are not restricted to single pixels or voxels.

5.2.1. Regional minima and maxima

Regional minima are defined as connected regions of elements (pixels or voxels) with the same value, and whose neighboring elements all have values greater than that of the region. Similarly, **regional maxima** are regions of connected pixels or voxels with the same value, and whose neighbors all have smaller value (Figure 5.5). In both cases, the result depends on the choice of the connectivity of the regions (see section 3.3).



(a) Original image **(b)** Regional maxima **(c)** Profile of gray levels

Figure 5.5.: *Regional maxima on a grayscale image. (a) Original image with a line ROI superimposed. (b) Result of regional maxima, showing many spurious maxima. (c) Gray-level profile along the line ROI shown in (a).*

MorphoLibJ uses the algorithm described in Breen & Jones (1996) for computing regional minima and maxima.

5.2.2. Extended minima and maxima

One problem arising with regional minima or maxima is that they are very sensitive to noise. It is often more convenient to use so-called **extended extrema**, that allows filtering spurious extrema.

Extended minima and maxima are based on the **H-extrema transformations**, that remove extrema based on a contrast criterium denoted by h. For example, the H-maxima transformation of a function f is obtained by performing a morphological reconstruction by dilation of the function $f - h$ (the marker) constrained by the original function f (the mask). Extended maxima are obtained by computing the regional maxima of the reconstruction result. An illustration on a 1D signal is provided on Figure 5.6.

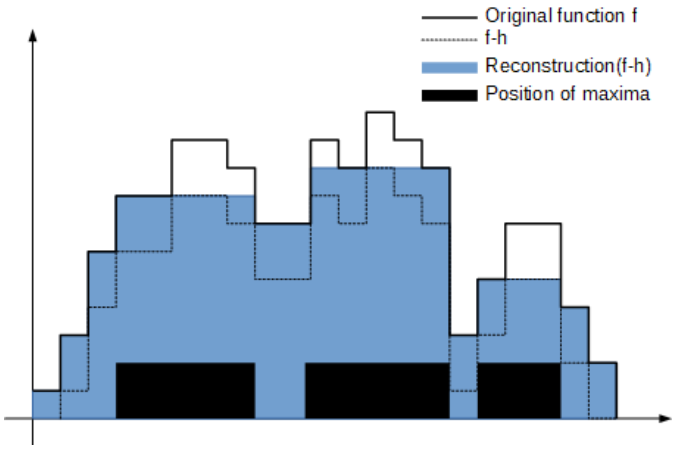


Figure 5.6.: *Computation of extended maxima on a 1D function.*

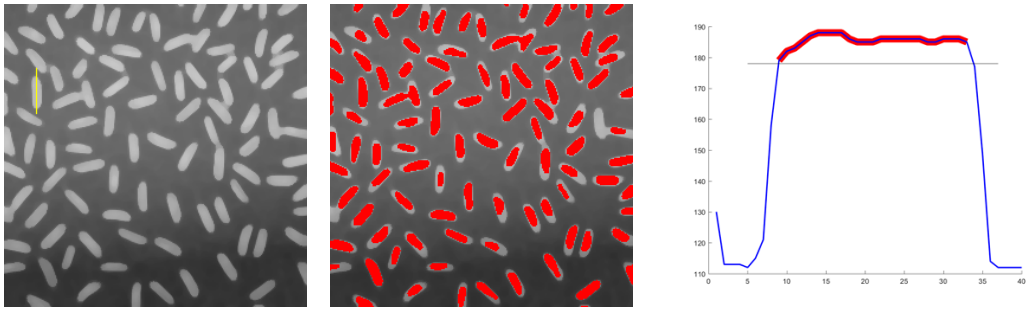
For images, **extended maxima** are defined as a connected region containing elements such that the difference of the value of each element within the region with the maximal value within the region is lower than the tolerance, and such that the neighbors of the regions all have values smaller than the maximum within the region minus the tolerance. This definition allows the identification of larger extrema, that better takes into account the noise within the image (Fig. 5.7). The **extended minima** are defined in a similar way, and are efficiently used as pre-processing step for watershed segmentation.

Both extended maxima and minima are computed using the morphological reconstruction algorithm. More details can be found in the book of Soille (2003).

5.2.3. Minima or maxima imposition

It is sometimes useful to transform an input grayscale image by **imposing minima or maxima** specified by a binary image. Such process is at the basis of watershed-based morphological segmentation (Section 6.4).

The principle is to perform a morphological reconstruction using as a marker image a combination of the input image and the minima or maxima, and as mask image the original



(a) Original image **(b)** Extended maxima **(c)** Profile of gray levels

Figure 5.7.: *Extended maxima on a grayscale image. (a) Original image with a line ROI superimposed. (b) Result of extended maxima computed with a dynamic of 10. (c) Gray-level profile along the line ROI shown in (a).*

input image. The result is a grayscale image whose regional minima or maxima are the same as the specified ones.

5.2.4. Plugins usage

The following operations are available in the "Plugins ▸ MorphoLibJ ▸ Minima and Maxima" menu :

Regional Min / Max
Computes regional minima or extrema in grayscale or binary image, with specified connectivity

Regional Min / Max 3D
Computes regional minima or extrema in 3D grayscale or binary image, with specified connectivity

Extended Min / Max
Computes extended minima or extrema in grayscale or binary image, with specified connectivity

Extended Min / Max 3D
Computes extended minima or extrema in 3D grayscale or binary image, with specified connectivity

Impose Min / Max
Imposes binary minima or maxima on a grayscale image.

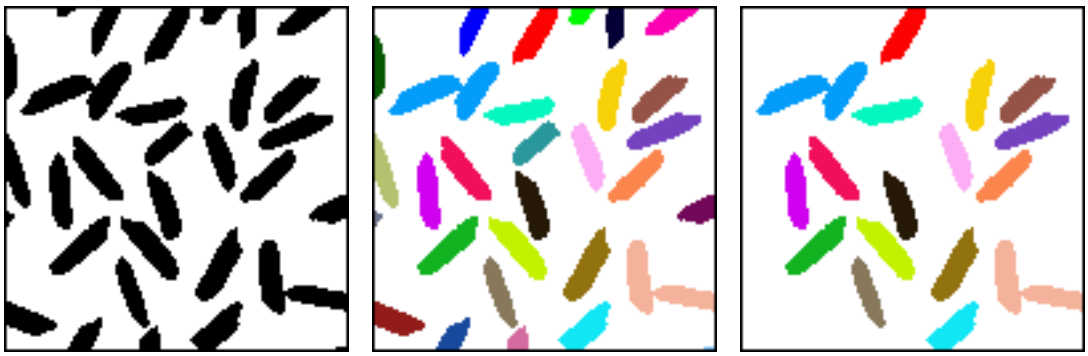
Impose Min / Max 3D
Imposes binary minima or maxima on a 3D grayscale image.

5.3. Attribute filtering

Attribute filters aim at removing components of an image based on a certain size criterion, rather than on intensity. The most common and useful criterion is the number of pixels/voxels (i.e., the area or volume). For example, a size opening operation with a threshold value equal to 20 will remove all blobs containing fewer than 20 voxels. The length of the diagonal of the bounding box can also be of interest to discriminate elongated versus round component shapes.

5.3.1. Application to binary images

When applied to a binary image, attribute opening consists in identifying each connected component, computing the attribute measurement of each component, and retain only the connected components whose measurement is above a specified value (Figure 5.8). This kind of processing is often used to clean-up segmentation results.



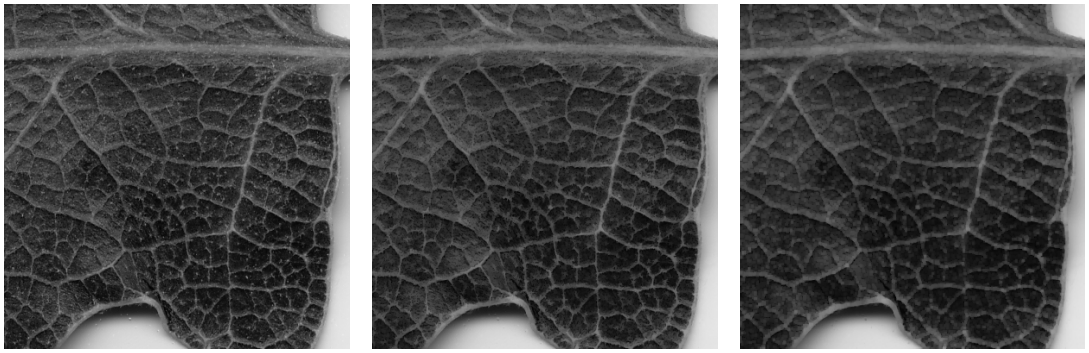
(a) Original image **(b)** Connected components **(c)** Size opening

Figure 5.8.: *Example of area opening on a binary image. (a) Original binary image. (b) Identification of connected components. (c) Only the connected components with a sufficient size (defined by the area), have been retained.*

5.3.2. Application to grayscale images

When applied to a grayscale image, attribute opening consists in generating a series of binary images by thresholding at each distinct gray level in the image. The binary attribute opening described above is then applied independently to each binary image and the grayscale output is computed as the union of the binary results. The final output is a grayscale image whose bright structures with the attribute below a given value have disappeared. A great advantage of this filter is that the contours of the structures area better preserved than opening with a structuring element (Figure 5.9).

As for classical morphological filters, grayscale attribute closing or tophat can be defined. Grayscale attribute closing consists in removing dark connected components whose size is



(a) Original image **(b)** Area opening **(c)** Morphological opening

Figure 5.9.: *Example of area opening on a grayscale image. (a) Original grayscale image of a leaf (image courtesy of Eric Biot, INRA Versailles). (b) Grayscale size opening making bright spots disappear. (c) Comparison with morphological closing using a square structuring element of radius 1: bright spots are removed, but some veins also disappear.*

smaller than a specified value. White [resp. Black] Attribute Top-Hat considers the difference of the attribute opening [resp. closing] with the original image, and can help identifying bright [resp. dark] structures with small size.

5.3.3. Usage

So far, the following attribute filtering plugins are available within MorphoLibJ (under "Plugins ▸ MorphoLibJ"):

Gray Scale Attribute Filtering

Opens a dialog to perform between attribute opening, closing, and black or white top-hat on a planar (2D) grayscale image. Two size criteria can be used: the area (number of pixels), or the diameter (length of the diagonal of the bounding box).

Gray Scale Attribute Filtering 3D

Opens a dialog to perform between attribute opening, closing, and black or white top-hat on a 3D grayscale image. The size criterion is the number of voxels.

6. Watershed segmentation

The **watershed algorithm** assimilates the grayscale image to a digital elevation model, and aims at detecting the different catchment basins. In the grayscale image, the catchment basins correspond to dark regions surrounded by bright structures (the "crests"). It is a very popular technique specially used to segment touching objects. The MorphoLibJ suite contains several implementations and applications of the algorithm, described in the following sections.

Contents

6.1. Classic Watershed plugin **35**
6.1.1. Introduction 35
6.1.2. Usage 35
6.1.3. Over-segmentation 36
6.2. Marker-controlled Watershed plugin **37**
6.2.1. Introduction 37
6.2.2. Usage 38
6.3. Interactive Marker-controlled Watershed plugin **38**
6.3.1. Introduction 38
6.3.2. Usage 39
6.4. Morphological Segmentation plugin **42**
6.4.1. Introduction 42
6.4.2. Usage 42
6.4.3. Macro language compatibility 45
6.5. Distance Transform Watershed **47**
6.5.1. Introduction 47
6.5.2. Distance Transform Watershed 47
6.5.3. Distance Transform Watershed (3D) 49

6.1. Classic Watershed plugin

6.1.1. Introduction

Classic Watershed is an ImageJ/Fiji plugin to perform watershed segmentation of grayscale 2D/3D images using flooding simulations as described by Soille & Vincent (1990); Vincent & Soille (1991).

The basic idea consists of considering the input image as topographic surface and placing a water source in each regional minimum of its relief. Next the entire relief is flooded from the sources and dams are placed where the different water sources meet.

All points in the surface at a given minimum constitute the **catchment basin** associated with that minimum. The **watersheds** are the zones dividing adjacent catchment basins.

The first image points that are reached by water are the points at the lowest grayscale value h_{min}, then all image pixels are progressively reached up to the highest level h_{max} (see Fig. 6.1).

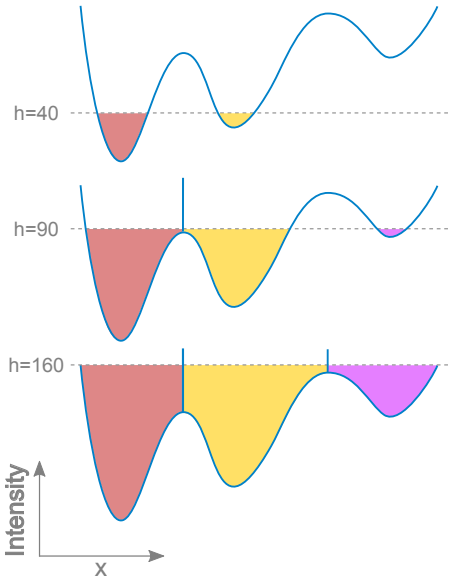
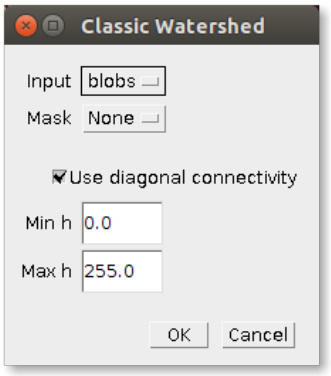


Figure 6.1.: *Schematic overview of watershed flooding in 1D*

6.1.2. Usage

The Classic Watershed plugin runs on any grayscale image (8, 16 and 32-bit) in 2D and 3D. At least one image needs to be open in order for the plugin to run. If that's the case, a dialog like the following will pop up:



Let's have a look at the different parameters:

- Image parameters:
  - **Input image**: grayscale image to flood, usually the gradient of an image.
  - **Mask image** (optional): binary image of the same dimensions as the input image which can be used to restrict the areas of application of the algorithm. Set to "None" to run the method on the whole input image.
- Morphological parameters:
  - **Use diagonal connectivity**: select to allow the flooding in diagonal directions (8-connectivity in 2D and 26-connectivity in 3D, see Section 3.3).
  - **Min h**: minimum grayscale value to start flooding from (by default, set to the minimum value of the image type).
  - **Max h**: maximum grayscale value to reach with flooding (by default, set to the maximum value of the image type).

Output:

- **Labeled image** containing the resulting catchment basins (with integer values 1, 2, 3...) and watershed lines (with 0 value).

6.1.3. Over-segmentation

Normally, Classic Watershed will lead to an over-segmentation of the input image, especially for noisy images with many regional minima. In that case, it is recommended to either **pre-process the image before** running the plugin, **or merge regions based on a similarity criterion afterwards**. Several de-noising methods are available in Fiji/ImageJ, namely: median filtering, Gaussian blur, bilateral filtering, etc.

Example: This short macro runs the plugin twice in the blobs sample, first without pre-processing and then after applying a Gaussian blur of radius 3:

```
// load the Blobs sample image
run("Blobs (25K)");
// invert LUT and pixel values to have dark blobs
run("Invert LUT");
run("Invert");
// run plugin on image
run("Classic Watershed", "input=blobs mask=None use min=0 max=150");
// apply LUT to facilitate result visualization
run("3-3-2 RGB");
// pre-process image with Gaussian blur selectWindow("blobs.gif");
run("Gaussian Blur...", "sigma=3");
rename("blobs-blur.gif");
// apply plugin on pre-processed image
run("Classic Watershed","input=blobs-blur mask=None use min=0 max=150");
// apply LUT to facilitate result visualization
run("3-3-2 RGB");
```

Figure 6.2.: *Input and result images from the macro example.*

6.2. Marker-controlled Watershed plugin

6.2.1. Introduction

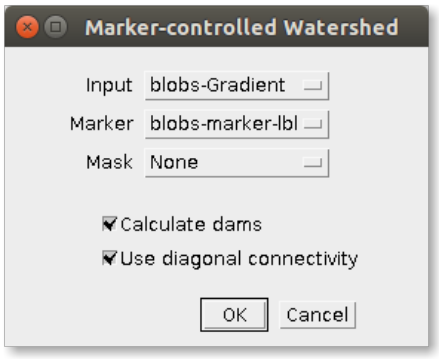
Marker-controlled Watershed is an ImageJ/Fiji plugin to segment grayscale images of any type (8, 16 and 32-bit) in 2D and 3D based on the marker-controlled watershed algorithm by Meyer & Beucher (1990). As the previous method, this algorithm considers the input image as a topographic surface (where higher pixel values mean higher altitude) but it simulates its flooding from **specific seed points or markers**. A common choice for the markers are the local minima of the gradient of the image, but the method works on any specific marker, either selected manually by the user or determined automatically by another algorithm (see Fig. 6.3).



Figure 6.3.: *Example of marker-controlled watershed segmentation on nucleus of* Arabidopsis thaliana *(image by courtesy of Kaori Sakai and Javier Arpon, INRA-Versailles).*

6.2.2. Usage

Marker-controlled Watershed needs at least two images to run. If that's the case, a dialog like the following will pop up:



Let's have a look at the different parameters:

- Image parameters:
  - The **Input image**: a 2D or 3D grayscale image to flood, usually the gradient of an image.
  - The **Marker image**: an image of the same dimensions as the input containing the seed points or markers as connected regions of voxels, each of them with a different label. They correspond usually to the local minima of the input image, but they can be set arbitrarily.
- And it can optionally admit a third image:
  - The **Mask image**: a binary image of the same dimensions as input and marker which can be used to restrict the areas of application of the algorithm. Set to "None" to run the method on the whole input image.
- Rest of parameters:
  - **Calculate dams**: select to enable the calculation of watershed lines.
  - **Use diagonal connectivity**: select to allow the flooding in diagonal directions (more rounded objects are usually obtain by unchecking this option).

Output:

- **Labeled image** containing the catchment basins and (optionally) watershed lines (dams).

6.3. Interactive Marker-controlled Watershed plugin

6.3.1. Introduction

Similar to the Marker-controlled Watershed plugin, this ImageJ/Fiji plugin segments grayscale images of any type (8, 16 and 32-bit) in 2D and 3D using the marker-controlled watershed

algorithm by Meyer & Beucher (1990). It floods the image from specific seed points, but this time **the points are introduced interactively by the user**.

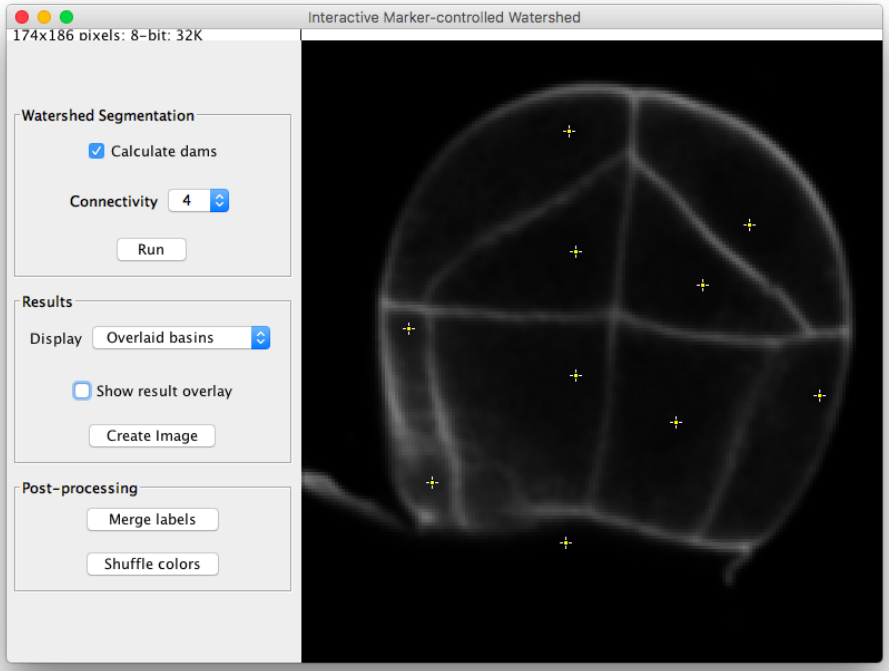


Figure 6.4.: *Overview of the Interactive Marker-controlled Watershed plugin.*

6.3.2. Usage

Interactive Marker-controlled Watershed runs on any open grayscale image, single 2D image or (3D) stack. If no image is open when calling the plugin, an Open dialog will pop up.

The user can pan, zoom in and out, or scroll between slices (if the input image is a stack) in the main canvas as if it were any other ImageJ window. On the left side of the canvas there are three panels of parameters, one with the watershed parameters, one for the output (result) options and one for post-processing the result. All buttons, checkboxes and panels contain a short explanation of their functionality that is displayed when the cursor lingers over them.

6.3.2.1. Interactive markers

In this plugin the markers are introduced interactively by the user using any of the selection tools. By default, the point selection tool will be enabled in the main ImageJ toolbar. To select markers on different slices, one option is to use the point selection tool and keep the

SHIFT key pressed each time you click to set a new marker. Another possibility is to use the ROI Manager. In that case, all selected ROIs in the manager will be used as markers (see Fig. 6.3for examples of selections used as markers).

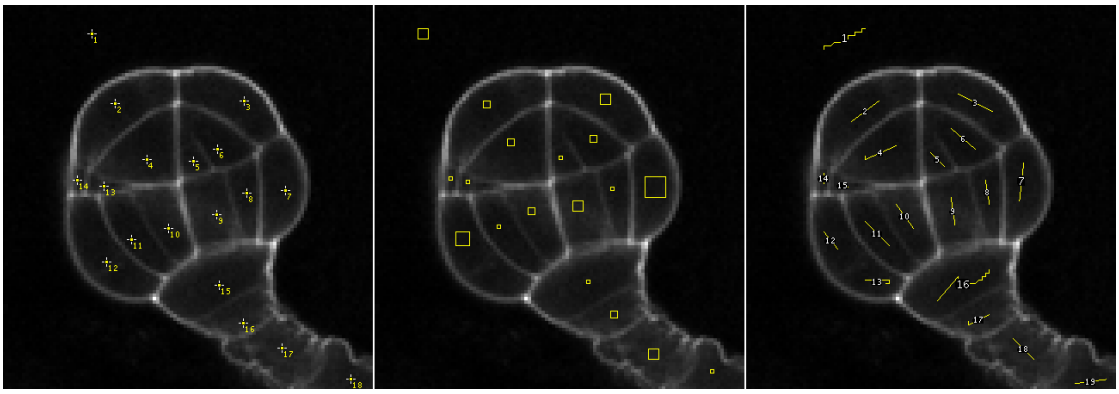
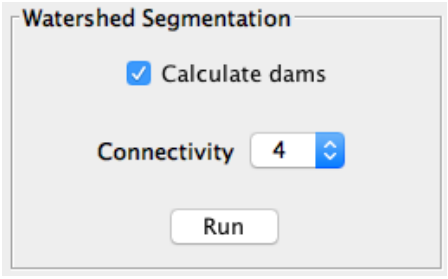


Figure 6.5.: *Example of interactive markers introduced by the user. From left to right: point selections, rectangular selections and free-hand selections (stored in the ROI Manager).*

6.3.2.2. Watershed Segmentation panel



This panel is reserved to the parameters involved in the segmentation pipeline:

- **Calculate dams**: un-check this option to produce segmentations without watershed lines.

- **Connectivity**: voxel connectivity (4-8 in 2D, and 6-26 in 3D). Selecting non-diagonal connectivity (4 or 6) usually provides more rounded objects.

Finally, **click on "Run" to launch the segmentation**. If your segmentation is taking too long or you want **to stop it** for any reason, you can do so by clicking on the same button (which should read "STOP" during that process).

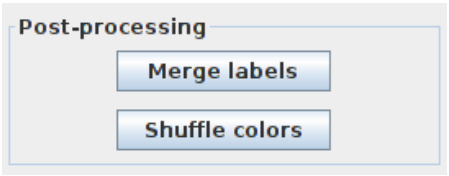
6.3.2.3. Results panel



Only enabled after running the segmentation.

- **Display**: list of options to display the segmentation results (see Fig. 6.7).
  - **Overlaid basins**: colored objects overlaying the input image (with or without dams depending on the selected option in the Watershed Segmentation panel).
  - **Overlaid dams**: overlay the watershed dams in red on top of the input image (only works if "Calculate dams" is checked).
  - **Catchment basins**: colored objects.
  - **Watershed lines**: binary image showing the watershed lines in black and the objects in white (only works if "Calculate dams" is checked).
- **Show result overlay**: toggle result overlay.
- **Create image button**: create a new image with the results displayed in the canvas.

6.3.2.4. Post-processing panel



Similarly to the Results panel, this panel only gets enabled after running the segmentation pipeline.

- **Merge labels**: merge together labels selected by either the **"freehand" selection tool** (on a single slice) **or the point tool** (on single or multiple slices). The zero-value label belongs to the watershed dams, therefore it will ignored in case of being selected. The first selected label value will be assigned to the rest of selected labels, which will share its color.
  Note: to select labels on different slices, use the point selection tool and keep the SHIFT key pressed each time you click on a new label.
- **Shuffle colors**: randomly re-assign colors to the labels. This is a very handy option whenever two adjacent labels present a similar color.

6.4. Morphological Segmentation plugin

6.4.1. Introduction

Morphological Segmentation is an ImageJ/Fiji plugin that combines morphological operations, such as extended minima and morphological gradient, with watershed flooding algorithms to segment grayscale images of any type (8, 16 and 32-bit) in 2D and 3D.

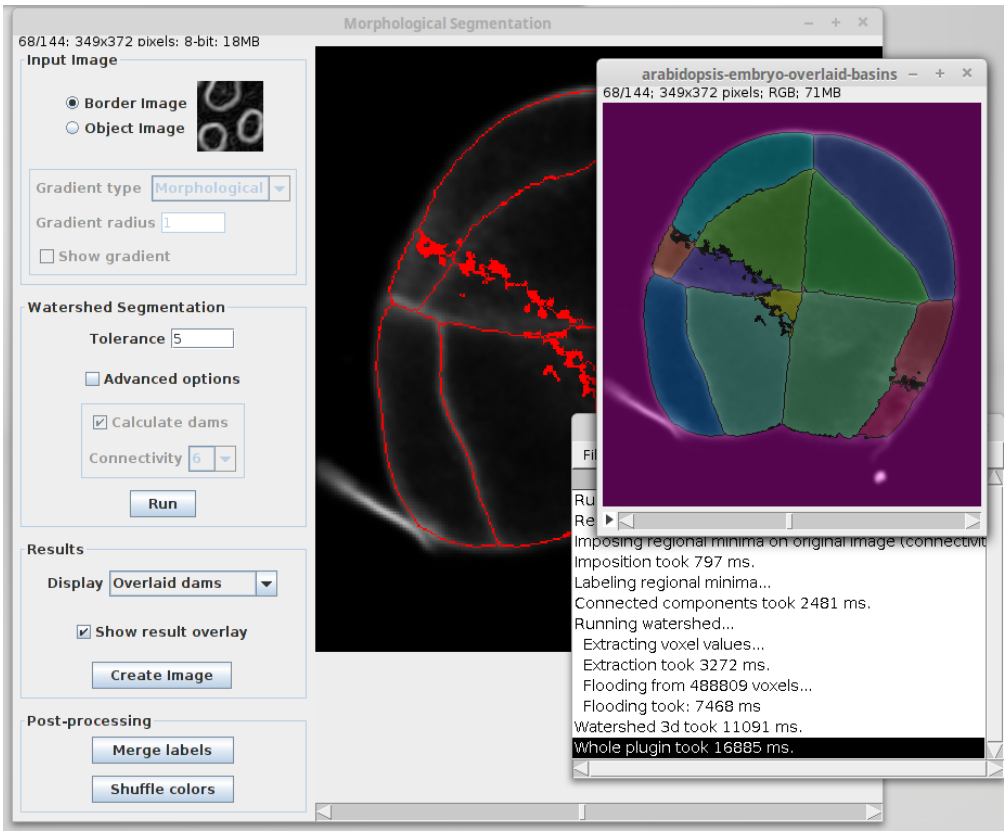


Figure 6.6.: *Overview of the Morphological Segmentation plugin.*

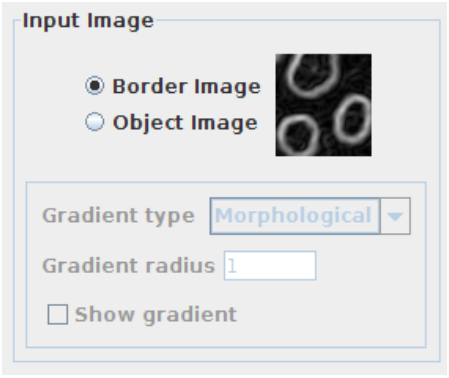
6.4.2. Usage

Morphological Segmentation runs on any open grayscale image, single 2D image or (3D) stack. If no image is open when calling the plugin, an Open dialog will pop up.

The user can pan, zoom in and out, or scroll between slices (if the input image is a stack) in the main canvas as if it were any other ImageJ window. On the left side of the canvas there are four panels of parameters, one for the input image, one with the watershed parameters, one for the output options and one for post-processing the resulting labels. All buttons, checkboxes and panels contain a short explanation of their functionality that is displayed when the cursor lingers over them.

Image pre-processing: some pre-processing is included in the plugin to facilitate the segmentation task. However, other pre-processing may be required depending on the input image. It is up to the user to decide what filtering may be most appropriate upstream.

6.4.2.1. Input Image panel



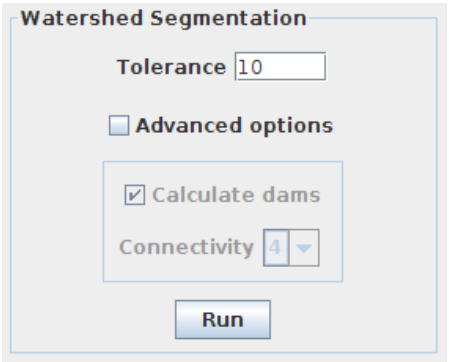
First, you need to indicate the nature of the input image to process. This is a **key parameter** since the watershed algorithm is expecting an image where the boundaries of objects present high intensity values (usually as a result of a gradient or edge detection filtering).

You should select:

- **Border Image**: if your input image has highlighted object boundaries.
- **Object Image**: if the borders of the objects do not have higher intensity values than the rest of voxels in the image.

When selecting "Object Image", an additional set of options is enabled to choose the type of gradient and radius (in pixels) to apply to the input image before starting the morphological operations. Finally, a checkbox allows displaying the gradient image instead of the input image in the main canvas of the plugin (only after running the watershed segmentation).

6.4.2.2. Watershed Segmentation panel



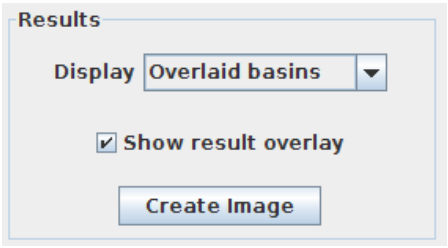
This panel is reserved to the parameters involved in the segmentation pipeline. By default, only the tolerance can be changed. Clicking on "Advanced options" enables the rest of options.

- **Tolerance**: dynamic of intensity for the search of regional minima (in the extended-minima transform, which is the regional minima of the H-minima transform, value of h). Increasing the tolerance value reduces the number of segments in the final result, while decreasing its value produces more object splits.
  Note: since the tolerance is an intensity parameter, it is sensitive to the input image type. A tolerance value of 10 is a good starting point for 8-bit images (with 0-255 intensity range) but it should be drastically increased when using image types with larger intensity ranges. For example to ~2000 when working on a 16-bit image (intensity values between 0 and 65535).
- **Calculate dams**: un-check this option to produce segmentations without watershed lines.
- **Connectivity**: voxel connectivity (4-8 in 2D, and 6-26 in 3D). Selecting non-diagonal connectivity (4 or 6) usually provides more rounded objects.

Finally, **click on "Run" to launch the segmentation**.

If your segmentation is taking too long or you want **to stop it** for any reason, you can do so by clicking on the same button (which should read "STOP" during that process).

6.4.2.3. Results panel



Only enabled after running the segmentation.

- **Display**: list of options to display the segmentation results.
  - **Overlaid basins**: colored objects overlaying the input image (with or without dams depending on the selected option in the Watershed Segmentation panel).
  - **Overlaid dams**: overlay the watershed dams in red on top of the input image (only works if "Calculate dams" is checked).
  - **Catchment basins**: colored objects.
  - **Watershed lines**: binary image showing the watershed lines in black and the objects in white (only works if "Calculate dams" is checked).

- **Show result overlay**: toggle result overlay.
- **Create image button**: create a new image with the results displayed in the canvas.

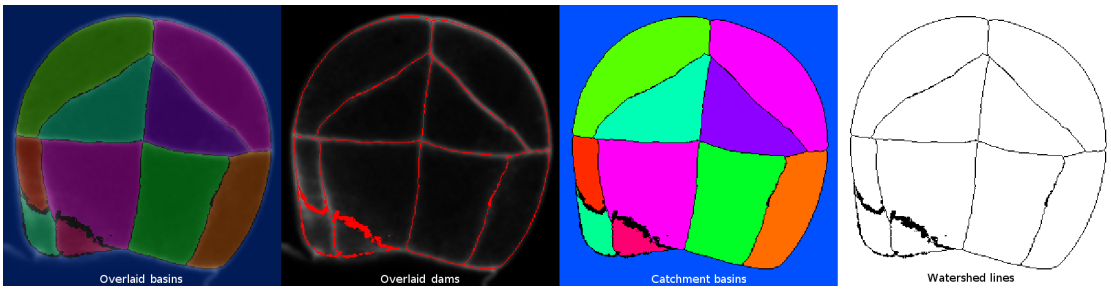
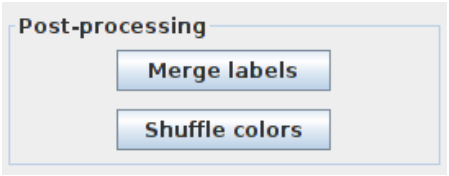


Figure 6.7.: *Examples of the 4 different display options.*

6.4.2.4. Post-processing panel



Similarly to the Results panel, this panel only gets enabled after running the segmentation pipeline.

- **Merge labels**: merge together labels selected by either the **"freehand" selection tool** (on a single slice) **or the point tool** (on single or multiple slices). The zero-value label belongs to the watershed dams, therefore it will ignored in case of being selected. The first selected label value will be assigned to the rest of selected labels, which will share its color.
  Note: to select labels on different slices, use the point selection tool and keep the SHIFT key pressed each time you click on a new label.
- **Shuffle colors**: randomly re-assign colors to the labels. This is a very handy option whenever two adjacent labels present a similar color.

6.4.3. Macro language compatibility

Morphological Segmentation is completely compatible with the popular ImageJ macro language[1]. Each of the buttons in the GUI are macro-recordable and their commands can be reproduced later from a simple macro file.

The complete list of commands is as follows:

[1] http://imagej.net/developer/macro/macros.html

- Start the plugin:

```
run("Morphological Segmentation");
```

- Select input image:

```
// select as object image
call("inra.ijpb.plugins.MorphologicalSegmentation.setInputImageType", "object");
// select as border image
call("inra.ijpb.plugins.MorphologicalSegmentation.setInputImageType", "border");
```

- Run segmentation with specific options:

```
call("inra.ijpb.plugins.MorphologicalSegmentation.segment","tolerance=10", "
    calculateDams=true","connectivity=6");
```

- Toggle result overlay:

```
call("inra.ijpb.plugins.MorphologicalSegmentation.toggleOverlay");
```

- Set option to display gradient image:

```
call("inra.ijpb.plugins.MorphologicalSegmentation.setShowGradient", "true");
```

- Select display format:

```
// Overlaid basins
call("inra.ijpb.plugins.MorphologicalSegmentation.setDisplayFormat", "Overlaid
    basins");
// Overlaid dams
call("inra.ijpb.plugins.MorphologicalSegmentation.setDisplayFormat", "Overlaid dams"
    );
// Catchment basins
call("inra.ijpb.plugins.MorphologicalSegmentation.setDisplayFormat", "Catchment
    basins");
// Watershed lines
call("inra.ijpb.plugins.MorphologicalSegmentation.setDisplayFormat", "Watershed
    lines");
```

- Create new image with the current result:

```
call("inra.ijpb.plugins.MorphologicalSegmentation.createResultImage");
```

Complete macro example

The following macro opens the Blobs sample image, applies the plugin with a tolerance value of 32 and displays the result as overlaid dams.

```
// load the Blobs sample image
run("Blobs (25K)");
// run the plugin
run("Morphological Segmentation");
// wait for the plugin GUI to load
wait(1000);
// select input image as "object"
call("inra.ijpb.plugins.MorphologicalSegmentation.setInputImageType", "object");
// set gradient radius as 1
call("inra.ijpb.plugins.MorphologicalSegmentation.setGradientRadius", "1");
// run segmentation with tolerance 32, calculating the watershed dams,
// 4-connectivity
call("inra.ijpb.plugins.MorphologicalSegmentation.segment", "tolerance=32", "
    calculateDams=true", "connectivity=4");
// display the overlaid dams
call("inra.ijpb.plugins.MorphologicalSegmentation.setDisplayFormat", "Overlaid dams");
```

6.5. Distance Transform Watershed

6.5.1. Introduction

A classic way of separating touching objects in binary images makes use of the distance transform and the watershed method. The idea is to create a border as far as possible from the center of the overlapping objects. This strategy works very well on rounded objects and it is called Distance Transform Watershed. It consists on calculating the distance transform of the binary image, inverting it (so the darkest parts of the image are the centers of the objects) and then applying watershed on it using the original image as mask (see Figure 6.8). In our implementation, we include an option to use watershed with extended minima so the user can control the number of object splits and merges.

MorphoLibJ provides two plugins under the "Plugins ▸ MorphoLibJ ▸ Binary Images..." menu to apply this strategy on 2D and 3D images:

6.5.2. Distance Transform Watershed

Distance Transform Watershed needs one 2D 8-bit binary image to run. If that's the case, a dialog like the following will pop up:

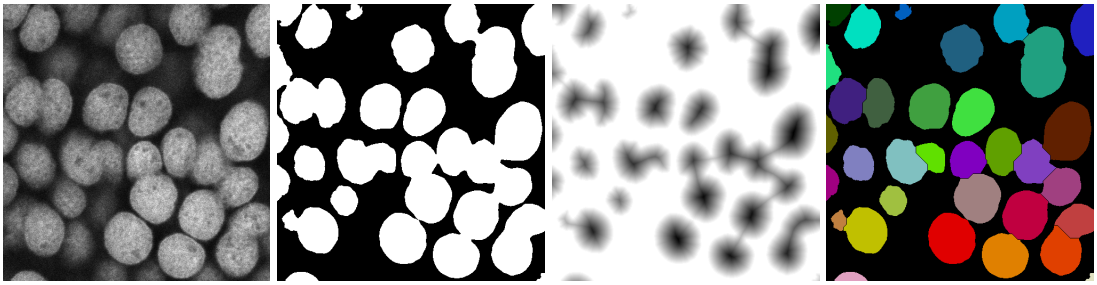


***Figure 6.8.: Basics of the Distance Transform Watershed algorithm**. From left to right: sample image of touching DAPI stained cell nuclei from a confocal laser scanning microscope, binary mask calculated after filtering and thresholding input image, inverse of the distance transform applied to the binary mask (Chamfer distance map using normalized Chessknight weights and 32-bit output) and resulting labeled image after applying watershed to the inverse distance image using the binary mask (dynamic of 1 and 4-connectivity).*



The plugin parameters are divided between the distance transform and the watershed options:

- **Distance map options**:
  - **Distances**: allows selecting among a pre-defined set of weights that can be used to compute the distance transform using Chamfer approximations of the Euclidean metric (see section 8.1.1). They affect the location but specially the shape of the border in the final result. The options are:
    - **Chessboard (1,1)**: weight equal to 1 for all neighbors.
    - **City-Block (1,2)**: weights 1 for orthogonal neighbors and 2 for diagonal neighbors.
    - **Quasi-Euclidean (1,1.41)**: weights 1 for orthogonal neighbors and $\sqrt{2}$ for diagonal neighbors.

        - **Borgefors (3,4)**: weights 3 for orthogonal neighbors and 4 for diagonal neighbors (best approximation of Euclidean distance for 3-by-3 masks).
        - **Weights (2,3)**: weights 2 for orthogonal neighbors and 3 for diagonal neighbors.
        - **Weights (5,7)**: weights 5 for orthogonal neighbors and 7 for diagonal neighbors.
        - **Chessknight (5,7,11)**: weights 5 for orthogonal neighbors and 7 for diagonal neighbors, and 11 for chess-knight moves (best approximation for 5-by-5 masks).
    - **Output type**: 16 or 32-bit, to calculate distance with short or float precision.
    - **Normalize weights**: indicates whether the resulting distance map should be normalized (divide distances by the first Chamfer weight).

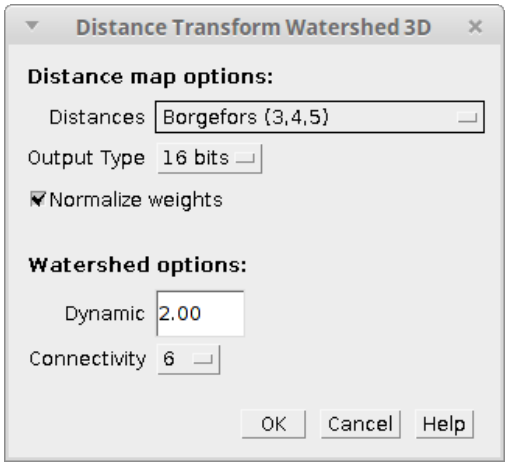
- **Watershed options**:
    - **Dynamic**: same as in the Morphological Segmentation plugin, this is the dynamic of intensity for the search of regional minima in the inverse of the distance transform image. Basically, by increasing its value there will be more object merges and by decreasing it there will be more object splits.
    - **Connectivity**: pixel connectivity (4 or 8). Selecting non-diagonal connectivity (4) usually provides more rounded objects.

Finally, the result with the current plugin configuration can be visualized by clicking on the **Preview** option.

Result: 2D 32-bit label image (one index value per object).

6.5.3. Distance Transform Watershed (3D)

Distance Transform Watershed 3D needs one 3D 8-bit binary image to run. If that's the case, a dialog like the following will pop up:



The parameters are the same as in the 2D version but some of them are adapted for 3D images:

- **Distance map options**:
  - **Distances**: Now the available options are:
    - **Chessboard (1,1,1)**: weight equal to 1 for all neighbors.
    - **City-Block (1,2,3)**: weights 1 for orthogonal neighbors, 2 for diagonal neighbors and 3 for cube-diagonals.
    - **Quasi-Euclidean (1,1.41,1.73)**: weights 1 for orthogonal neighbors, $\sqrt{2}$ for diagonal neighbors and $\sqrt{3}$ for cube-diagonals.
    - **Borgefors (3,4,5)**: weights 3 for orthogonal neighbors, 4 for diagonal neighbors and 5 for cube-diagonals (best approximation of Euclidean distance for 3-by-3-by-3 masks).
  - **Output type**: 16 or 32-bit, to calculate distance with short or float precision.
  - **Normalize weights**: indicates whether the resulting distance map should be normalized (divide distances by the first Chamfer weight).
- **Watershed options**:
  - **Dynamic**: same as in the 2D version, this is the dynamic of intensity for the search of regional minima in the inverse of the distance transform image. Basically, by increasing its value there will be more object merges and by decreasing it there will be more object splits.
  - **Connectivity**: voxel connectivity (6 or 26). Selecting non-diagonal connectivity (6) usually provides more rounded objects.

As it is usual in ImageJ, no preview is provided here since we are dealing with 3D images.

Result: 3D 32-bit label image (one index value per object).

7. Measurements

MorphoLibJ contains several tools for quantifying the **size**, the **shape**, the **intensities**, or the **spatial organization** of biological structures observed within images 2D or 3D images.

Sections 7.1 and refer 7.2 to the analysis of 2D and 3D regions, respectively. Most region analysis operators of the MorphoLibJ library assume the input image to be either a binary image representing a single region, or a label image representing a collection of disjoint regions (see section 9). The aim is to facilitate the management of label images, contrary to the built-in "Analyze Particles..." function that operates directly on a grayscale image.

The library also provides plugins for intensity measurements (section 7.3), the comparison of label images (section 7.4), and the quantification of spatial organization (section 9.5.1).

Contents

7.1. Region Analysis 52
7.1.1. Intrinsic volumes 52
7.1.2. Geometric moments and equivalent ellipse 55
7.1.3. Feret diameter and oriented box 56
7.1.4. Geodesic measurements 57
7.1.5. Thickness 57
7.1.6. Quantification by shape indices 58
7.1.7. Plugins 59
7.2. Region Analysis 3D 65
7.2.1. Intrinsic volumes 65
7.2.2. Geometric moments and equivalent Ellipsoid 68
7.2.3. 3D Shape indices 69
7.2.4. Plugins 70
7.3. Intensity measurements 70
7.4. Label Overlap measures 71
7.5. Microstructure analysis 72

7.1. Region Analysis

Region analysis usually refers to the quantification of features related to the **size** and the **shape** of regions identified within images (Arganda-Carreras & Andrey, 2017).

Many region features have been proposed to describe morphology of regions. The MorphoLibJ library proposes plugins to quantify some of the most common ones:

- **intrinsic volumes** (section 7.1.1) encompass the area, the perimeter, and the Euler number, that quantifies the topology of the region
- the computation of **moments of a region** can be used to determine the centroid or equivalent ellipse (section 7.1.2)
- dimension extent may be determined by use of **Feret diameters** (section 7.1.3)
- **geodesic features** allow to investigate the complexity of the region, in particular the tortuosity (section 7.1.4)
- the use of the skeleton provide derived features such as the **average thickness** (section 7.1.5)
- features describing the size may be combined to obtain **shape indices** that describe the morphology of the regions independently of their size and location (section 7.1.6)

The following sections will present these features. For each family of features, we first provide mathematical definitions of the implemented features, eventually completed by the methods used to measure them from discrete images. A final section (section 7.1.7) presents the different plugins that allow to measure features from images.

7.1.1. Intrinsic volumes

The **intrinsic volumes** are a set of features with interesting mathematical properties that are commonly used for describing individual particles as well as binary microstructures. In 2D, they correspond to the **area**, the **perimeter** and the **Euler number**.

7.1.1.1. Area

The **area** of a set X can be defined using integrals over the set:

$$A(X) = \int_X dx \tag{7.1}$$

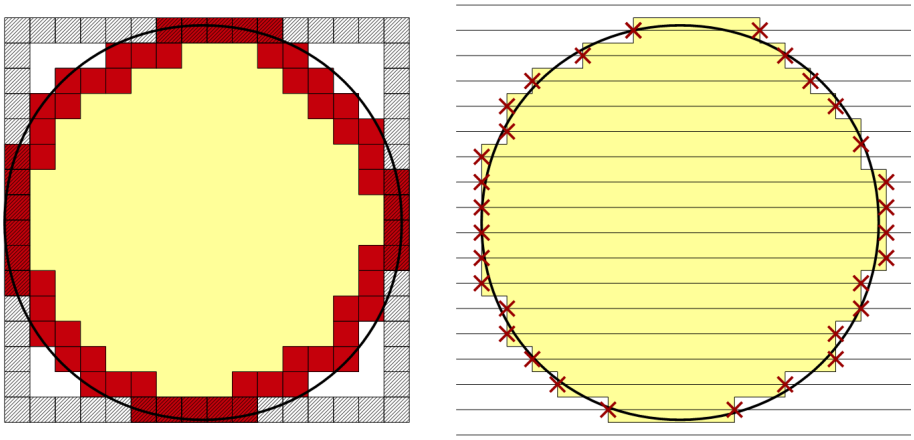
In image analysis, the **measurement of area** of a region simply consists in counting the number of pixels that constitute it, and to multiply by the area of a single pixel.

7.1.1.2. Perimeter

The **perimeter** of a region corresponds to the length of its boundary. Its formal definition involves computing an integral over its boundary ∂X:

$$P(X) = \int_{\partial X} dx \tag{7.2}$$

An intuitive approach for **measuring the perimeter** of a binary region is to count the number of boundary pixels (Figure 7.1). However, this approach is rather inaccurate. For example, when counting the boundary pixels of a discretized disk, one obtain the same perimeter value as the length of the bounding rectangle, leading to a relative error of more than 25%[1].



(a) Boundary pixels count **(b)** Crofton method

Figure 7.1.: *Measurement of perimeter within a discrete image of a disk. (a) Counting the number of boundary pixels. (b) Counting the number of intersection with parallel lines.*

The discrete version of Crofton formula can provide a better estimate of the perimeter than traditional boundary pixel count. The principle is to consider a set of parallel lines with a random orientation, and to count the number of intersections with the region of interest (Figure 7.1). The number of intersections is proportional to the line density and to the perimeter (Serra, 1982; Legland et al., 2007; Ohser & Schladitz, 2009). By averaging over all possible directions, an unbiased estimate can be obtained.

Perimeter can be measured using either two directions (horizontal and vertical), or four directions (by adding the diagonals). Restricting the number of directions introduces an estimation bias, with known theoretical bounds (Moran, 1966; Legland et al., 2007). The error made by counting intersections is usually better than boundary pixel count (Lehmann & Legland, 2012).

[1] If d is the diameter of the circle, the expected perimeter is πd. The number of boundary pixels measure the perimeter of the enclosing square, equal to $4d$. The relative error is $(4d - \pi d)/\pi d \simeq 27\%$

7.1.1.3. Euler Number

The **Euler number** is a feature that describes the **topology** of a region. It corresponds to the number of connected components, minus the number of holes (Fig. 7.2-a).

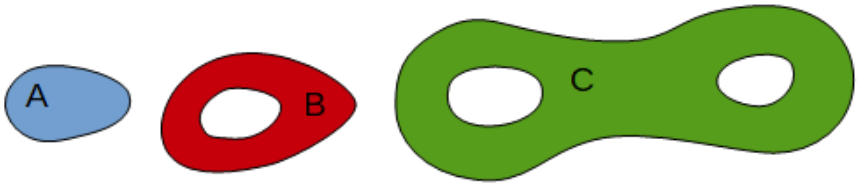
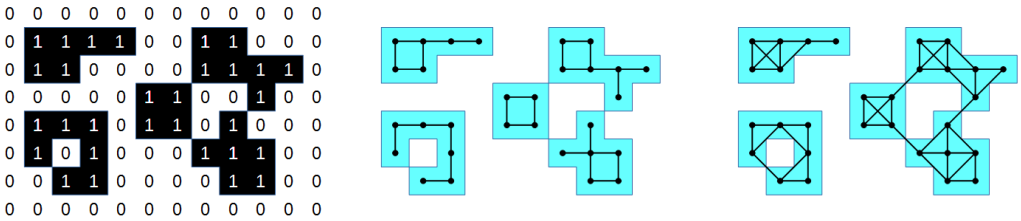


Figure 7.2.: *Euler Number for 2D regions. Particle A is composed of a single component, its Euler number is 1. Particles B and C present one and two holes respectively. Their corresponding Euler numbers are equal to* $0 = 1 - 1$ *and* $-1 = 1 - 2$.

In 2D, the Euler number of a region with smooth boundary also equals the **integral of the curvature** over the boundary of the set:

$$\chi(X) = \frac{1}{2\pi} \int_{\partial X} \kappa(x) dx \tag{7.3}$$

The measurement of Euler number from binary images is usually performed by considering a reconstruction of the **connected components** that connects adjacent pixels or voxels (Fig. 7.3). The resulting reconstruction depends on the choice of the connectivity (see Section 3.3). For planar images, typical choices are the 4-connectivity, corresponding to the orthogonal neighbors (Fig. 7.3-b), and the 8-connectivity, that also consider the diagonal neighbors (Fig. 7.3-b). The connectivity is also considered for computing morphological reconstructions (section 5.1), or for computing the connected component labeling of a binary image (section 8.3).



(a) Binary image **(b)** 4-connectivity **(c)** 8-connectivity

Figure 7.3.: *Determination of the Euler number from binary images. (a) Original binary images. (b) Euler number computed with 4-connectivity: five components are detected. (c) Euler number computed using the 8-connectivity: three components are detected, two of them having holes.*

Depening on the chosen connectivity, the resulting Euler number may differ. For example, the reconstruction presented in Figure 7.3 has Euler number equal to five (the number of connected components), whereas the Euler number of the reconstruction using the 8-connectivity is equal to 1: three connected components are obtained, minus two holes within them (Fig. 7.3-c).

7.1.2. Geometric moments and equivalent ellipse

A binary region X may be described mathematically by its **geometric moments** m_{pq} of order (p,q), which correspond to an integral of its indicator function I_X, with various degrees along the directions:

$$m_{pq}(X) = \int \int I_X(x,y) x^p y^q \cdot dx \cdot dy \tag{7.4}$$

The moment of order $(0,0)$ simply corresponds to the area of X:

$$\begin{aligned} m_{00} &= \int \int I_X(x,y) \cdot dx \cdot dy \\ &= A(X) \end{aligned}$$

The coordinates of the centroid of X can be expressed from the first-order moments:

$$\begin{aligned} x_c &= \frac{m_{10}}{m_{00}} = \frac{1}{Area(X)} \int \int I_X(x,y) \cdot x \cdot dx \cdot dy \\ y_c &= \frac{m_{01}}{m_{00}} = \frac{1}{Area(X)} \int \int I_X(x,y) \cdot y \cdot dx \cdot dy \end{aligned}$$

It is often more convenient to work with **centered moments**, given by:

$$\mu_{pq} = \int \int I_X(x,y) (x - x_c)^p (y - yc)^q \cdot dx \cdot dy \tag{7.5}$$

The second order centered moments can be arranged into a matrix:

$$\begin{bmatrix} \mu_{20} & \mu_{11} \\ \mu_{11} & \mu_{02} \end{bmatrix}$$

By computing the parameters of the ellipse that produces the same moments up to the second order, one obtains the **equivalent ellipse** (Burger & Burge, 2008). In particular, the orientation θ of the equivalent ellipse is given by $\theta = \frac{1}{2}\text{atan2}(2 \cdot \mu_{11}, \mu_{20} - \mu_{02})$.

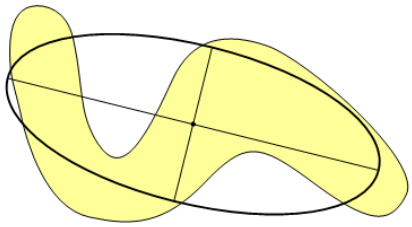


Figure 7.4.: *Equivalent ellipse of a binary region.*

7.1.3. Feret diameter and oriented box

A popular way to assess the size of a region is to measure its **largest Feret diameter**. The maximum Feret diameter of a region is simply the maximum distance computed over all the pairs of points belonging to the region:

$$F_{\max} = \max_{\boldsymbol{x},\boldsymbol{y} \in X} d(\boldsymbol{x},\boldsymbol{y}) \tag{7.6}$$

It is easy to realize that the search can be performed on the set of boundary points (Fig. 7.5). In practice, the computation of Maximum Feret diameter can be accelerated by first computing the convex hull of the region. In MorphoLibJ, the maximal Feret diameter is computed over the corners of the regions.

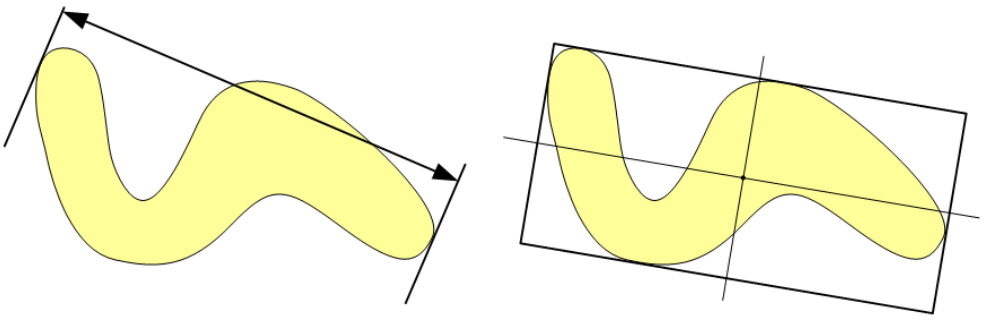


Figure 7.5.: *Feret diameter and oriented box*

The Feret diameter can be performed on any direction. Typical choice is to measure the Feret diameter in the direction perpendicular to the direction of the maximum Feret diameter. The ratio of the largest Feret diameter and of the Feret diameter measured in the perpendicular direction gives an indication on the elongation of the region.

The notion of **object-oriented bounding box** (sometimes denoted as “OOBB”) is closely related to that of Feret diameter. One possible definition of oriented bounding box is the rectangle with smallest width that totally encloses the region. An example is illustrated on

Figure 7.5. Another definition is the rectangle with smallest area: the two resulting rectangles are often similar, but may differ in some cases. Note that in general, the orientation of the largest axis of the oriented box differs from both the direction of the largest Feret diameter, and the direction of the equivalent ellipse.

7.1.4. Geodesic measurements

For particles with complex shapes, the **geodesic diameter** may be of interest. It corresponds of the largest geodesic distance between two points within a region, the geodesic distance being the length of the shortest path joining the two points while staying inside the region (Lantuéjoul & Beucher, 1981). An example is given on Figure 7.6.

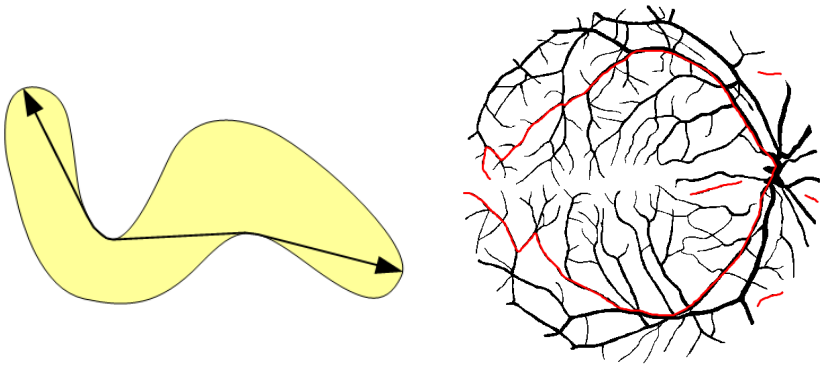


Figure 7.6.: *Geodesic diameter. Left: illustration of the geodesic diameter on a simple particle. Right: computation of the geodesic diameter on a segmented image from the DRIVE database (Staal et al., 2004). Each connected component was associated to a label, then the longest geodesic path within each connected component was computed and displayed as red overlay.*

The implementation of geodesic diameter within MorphoLibJ relies on Chamfer distances propagation (see section 8.2). Hence, the actual value may be somewhat over-estimated due to the approximation in the measurements.

7.1.5. Thickness

The thickness is also a convenient measure to describe a region. It may be evaluated by the diameter of the largest inscribed circle plugin (page 63, Fig. 7.7). This method however often over-estimates the thickness, as it measures thickness at the largest point within the region.

The MorphoLibJ library also proposes to measure **average thickness** by first computing the skeleton of the region, and then to measure for each pixel of the skeleton the distance to the closest background point in the original image (Fig. 7.7). This way, the thickness measure is integrated over the whole region skeleton and is more representative.

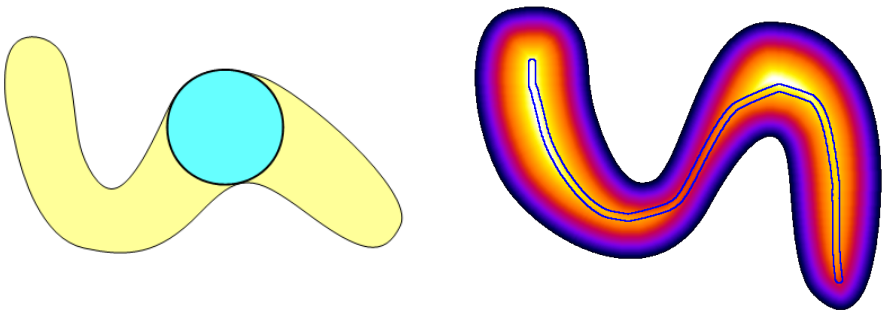


Figure 7.7.: *Evaluation of the thickness using two methods. On the left, computation of the largest inscribed circle. On the right, computation of the average thickness over the skeleton.*

7.1.6. Quantification by shape indices

It is often convenient to describe and **quantify the shape** of regions independently of their location, orientation, or relative scaling. Several indices are commonly used to describe the shape of the particles, independently of their size. We present here the definitions used within the MorphoLibJ plugins.

7.1.6.1. Circularity

The **circularity** (or "shape factor", or "isperimetric deficit index") is defined as the ratio of area over the square of the perimeter, normalized such that the value for a disk equals one:

$$\text{circularity} \quad = \quad 4\pi\frac{A}{P^2} \tag{7.7}$$

While the values of isoperimetric deficit index theoretically range within the interval $[0;1]$, the measurements errors of the perimeter may produce circularity values above 1 (Lehmann & Legland, 2012).

7.1.6.2. Convexity

The **convexity** (also known as "solidity") is defined as the ratio of the area of the region over the area of its convex hull.

$$\text{convexity} = \frac{A(X)}{A(\text{ConvHull}(X))}$$

The Convexify Plugin (Section 8.4.1) also allows to compute the convex image corresponding to a binary region.

7.1.6.3. Elongation

The MorphoLibJ library provides several shape indices describing the elongation of the regions. Their values is 1 for a disk, and increase for elongated regions.

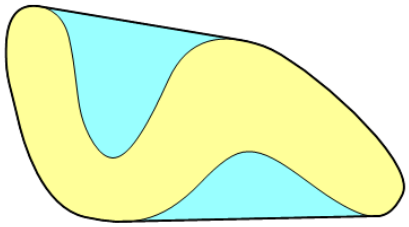


Figure 7.8.: *Convexity of a planar region. The convex area is the union of the blue and yellow regions.*

The ellipse elongation factor, defined as the ratio of the largest over the smallest axis lengths, can be used to quantify the shape of the region.

The oriented box elongation, defined as the ratio of the length of the oriented bounding box over its width

The geodesic elongation, defined as the ratio of the geodesic diameter of the diameter of the largest inscribed disk.

7.1.6.4. Tortuosity

The **tortuosity** is defined as the ratio of the geodesic diameter over the largest Feret diameter.

It describes the complexity of the region. Its theroretical value is 1 for convex regions, and increases for regions with complex geometries.

7.1.7. Plugins

Most MorphoLibJ plugins consider the current image as input, that must be either binary (only one region is considered), or label (typically the result of a connected components labeling, see section 8.3). The output is a results table (ImageJ *ResultsTable*) containing one row for each label actually present within the image. The spatial calibration of the image is taken into account for most measurements. All plugins can be found under the "Plugins ▸ MorphoLibJ ▸ Analyze" menu.

7.1.7.1. Analyze Regions

The **global geometry** of particles in 2D images can be characterized with the **Analyze Regions** plugin (under "Plugins ▸ MorphoLibJ ▸ Analyze ▸ Analyze Regions"). For 2D particles, the area, the perimeter and derived features are implemented.

The options of the dialog correspond to the parameters that can be computed:

Area the area of the regions

Perimeter the perimeter, measured using discrete version of Crofton formula (see section 7.1.1)

Circularity the normalized ratio of the area by the square of the perimeter: $4\pi \cdot A/p^2$. Values superior to 1 may appear due to discretization effect.

Equivalent Ellipse the position, size and orientation of the equivalent ellipse (see section 7.1.2)

Ellipse_Elong. the elongation of the equivalent ellipse (ratio of axis lengths)

Convexity the convexity, as defined in section 7.1.6.2

Max. Feret Diameter the value of the largest Feret diameter

Oriented Box the position, size and orientation of the oriented box with smallest width

Oriented Box Elong. the elongation of the bounding box (ratio of length over width)

Geodesic Diameter the length of the geodesic diameter

Tortuosity the tortuosity, as the ratio of geodesic diameter over largest Feret diameter

Max inscribed disc the position and the radius of the largest circle that can be inscribed within the region

Geodesic Elong. the ratio of geodesic diameter over diameter of inscribed circle.

The first columns of the resulting ResultsTable contain the label of the regions, to facilitate their identification.

7.1.7.2. Bounding Box

The **Bounding Box plugin** computes the minimum and maximum x and y coordinates of each region within a binary or label image. The result of the plugin comprises the following features:

Label the label of the region measured on the current line (equal to 255 for binary images)

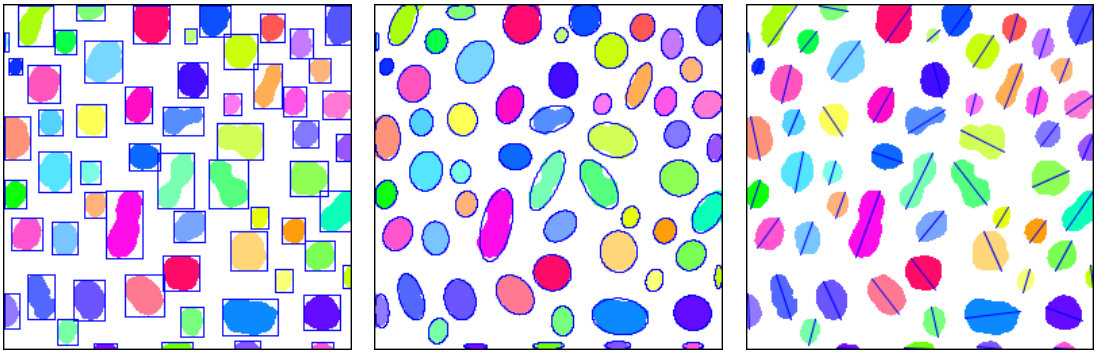
Box2D.XMin the minimum x coordinate

Box2D.XMax the maximum x coordinate

Box2D.YMin the minimum y coordinate

Box2D.YMax the maximum y coordinate

It is also possible to draw the resulting bounding box(es) onto another image (Fig. 7.9-a).



(a) Bounding Box **(b)** Equivalent Ellipse **(c)** Feret Diameter

Figure 7.9.: *Examples of graphical outputs of region analysis plugins. (a) Bounding box. (b) Equivalent ellipse. (c) Max Feret diameter.*

7.1.7.3. Equivalent Ellipse

The **Equivalent Ellipse plugin** computes the equivalent ellipse for each region within a binary or label image. The result of the plugin comprises the following features:

Label the label of the region measured on the current line (equal to 255 for binary images)

Ellipse.Center.X the x coordinate of ellipse center

Ellipse.Center.X the y coordinate of ellipse center

Ellipse.Radius1 the length of the largest semi-axis

Ellipse.Radius2 the length of the smallest semi-axis

Ellipse.Orientation the orientation of the ellipse, in degrees from the horizontal.

It is also possible to draw the resulting ellipse(s) onto another image (Fig. 7.9-b).

7.1.7.4. Max Feret Diameter

The **Max Feret Diameter plugin** computes the length of the largest Feret diameter of each region within a binary or label image, as well as the position of the extreme points. The result of the plugin comprises the following features:

Label the label of the region measured on the current line (equal to 255 for bionary images)

Diameter the length of the Feret diameter

Orientation the orientation of the diameter, in degrees from the horizontal

P1.X the x-coordinate of the first extremity

P1.Y the y-coordinate of the first extremity

P2.X the x-coordinate of the second extremity

P2.Y the y-coordinate of the second extremity

It is also possible to draw the resulting diameter as a line segment onto another image (Fig. 7.9-c).

7.1.7.5. Oriented Bounding Box

The **Oriented Bounding Box plugin** computes the minimal width oriented bounding box of each region within a binary or label image. The result of the plugin comprises the following features:

Label the label of the region measured on the current line (equal to 255 for bionary images)

OBox.Center.X the x coordinate of oriented box center

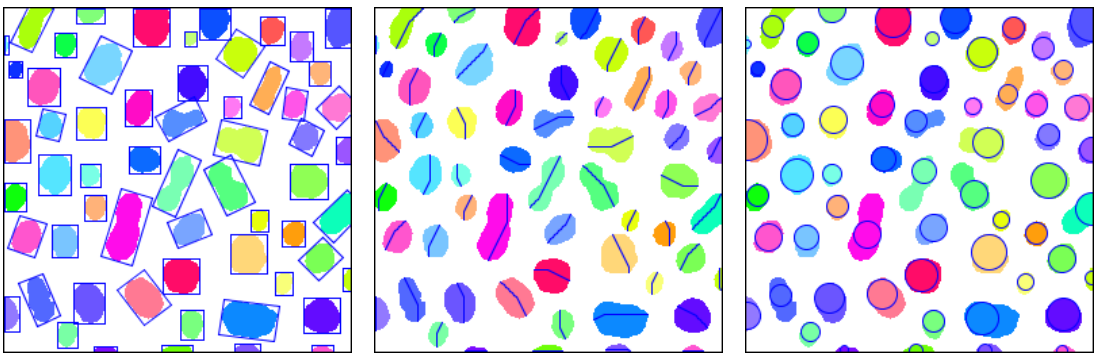
OBox.Center.X the y coordinate of oriented center

OBox.Length the length of the oriented box

OBox.Width the width of the oriented box

OBox.Orientation the orientation of the box, in degrees from the horizontal.

It is also possible to draw the resulting oriented boxe(s) onto another image (Fig. 7.10-a).



(a) Oriented box **(b)** Geodesic diameter **(c)** Inscribed circle

Figure 7.10.: *Examples of graphical output of region analysis plugins. (a) Oriented bounding box. (b) Largest geodesic diameter. (c) Largest inscribed circle.*

7.1.7.6. Geodesic Diameter

The **Geodesic Diameter plugin** computes several geodesic measures for each particle in a label image. The result of the plugin comprises the following features:

Label — the label of the region measured on the current line (equal to 255 for binary images)

Geod. Diam. — the value of the geodesic diameter.

Radius — the radius of the largest inscribed circle, which is computed during the algorithm.

Geod. Elong. — the ratio of geodesic diameter over the diameter of the largest inscribed circle. The values range from 1 for nearly round particles and increases for elongated particles.

xi, yi — coordinates of the largest inscribed circle.

x1, y1 — coordinates of one of the geodesic extremities of the particle.

x2, y2 — coordinates of another geodesic extremity of the particle.

It is also possible to draw the resulting geodesic paths onto another image (Fig. 7.10-b).

7.1.7.7. Largest inscribed circle

The **Max. Inscribed Circle plugin** computes for each label the largest disk that can be enclosed within the corresponding particle. The plugin opens a dialog that allows to choose the label image to characterize, the choice of the method for computing distance, and eventually the image on which overlaid circles can be drawn. The output of the plugin includes the following information:

Label — the label of the region measured on the current line (equal to 255 for binary images)

inscrCircle.Center.X — the x-coordinate of the inscribed circle.

inscrCircle.Center.Y — the y-coordinate of the inscribed circle.

inscrCircle.Radius — the radius of the inscribed circle.

It is also possible to draw the resulting circle(s) onto another image (Fig. 7.10-c).

7.1.7.8. Average Thickness

The Average Thickness plugin computes the average thickness of each region within a label map. The Plugin computes both the skeleton and the distance map within each label, and combines the two to compute the average of the distance map values within the skeleton pixels.

7.1.7.9. Skeleton Geodesic Diameter

The Skeleton Geodesic Diameter Plugin consists in computing the geodesic diameter of the skeleton of each region. The results may be used to assess the geodesic length of the largest path within the region. Contrary to the geodesic diameter, the path is computed such that is stays close to the medial axis of the region.

The Plugin provides several options:

Input Label Image The label map containing the image of the regions to analyze

Dilate Skeleton Can be used to allow more freedom on the way the geodesic path is computed. If set to false, only orthogonal and diagonal steps will be considered.

Chamfer mask The chamfer mask used to compute the geodesic path

Overlay Options Allows to choose to overlay the result over an image, and/or to export the computed geodesic paths to the ROI Manager.

7.2. Region Analysis 3D

Region analysis may also be performed on 3D images. As for 2D images, input images may be either binary (corresponding to a single region), or contain the labels of the different regions to analyze.

7.2.1. Intrinsic volumes

For 3D particles, intrinsic volumes correspond to the **volume**, the **surface area**, the **mean breadth** and the **3D Euler number**. While the volume and the surface area are rather common, the latter two are less intuitive. Both the mean breadth and the 3D Euler number can be related to the curvatures that can be measured on smooth surfaces.

7.2.1.1. Volume

The **volume** and the **surface area** of a set X with smooth surface ∂X can be defined using integrals over the set, or over its boundary:

$$V(X) = \int_X dx \tag{7.8}$$

The measurement of **volume of 3D region** is as straightforward as in 2D: it consists in counting the voxels that belong to the 3D region, multiplied by the volume of an individual voxel.

7.2.1.2. Surface area

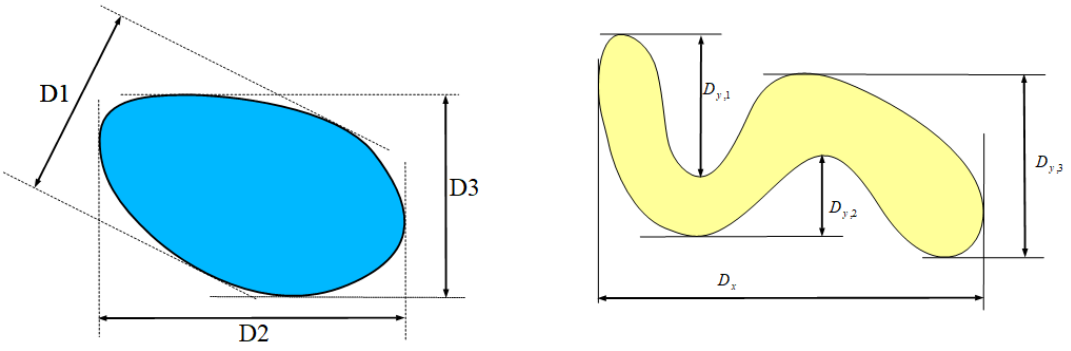
The **surface area** of a set X with smooth surface ∂X can be defined using integral over its boundary:

$$S(X) = \int_{\partial X} dx \tag{7.9}$$

The **measurement of surface area** from 3D binary images follows the same principle as the estimation of perimeter. One considers the intersections of the region with straight lines, and averages over all possible directions (Lang et al., 2001; Legland et al., 2007). The number of directions is typically chosen equal to 3 (the three main axes in image), or 13 (by considering also diagonals). As for perimeter estimation, surface area estimation is usually biased, but is usually more precise than measuring the surface area of the polygonal mesh reconstructed from binary images (Lehmann & Legland, 2012).

7.2.1.3. Mean breadth

The **mean breadth**, also known as the mean width or the mean diameter, is a feature that quantifies the size of a set as an average diameter. In the case of a convex set, the average breadth corresponds to the average of the caliper diameter (or Feret diameter) measured over all directions (Fig. 7.11-a).



(a) Convex set. **(b)** Non-Convex Set

Figure 7.11.: *Mean breadth of a convex (a) and of a non convex (b) set. In the case of a non-convex set, the concavities are taken into account for measuring the diameters.*

In the case of a non-convex set, the concavities are taken into account for measuring the diameters, resulting in "**total projected diameter**" (Fig. 7.11-b). For 3D sets, the measurements of total projected diameter consists in measuring the (2D) Euler number of the intersection of the set with planes orthogonal to the direction (Serra, 1982). The average of total projected diameter over Ω, the set of all directions, results in the mean breadth $\bar{b}$:

$$\bar{b}(X) = \int_{\Omega} D_{\omega}(X)\, d\omega \tag{7.10}$$

For a three-dimensional set X with smooth boundary ∂X, an alternative definition of the mean breadth is based on integral geometry (Serra, 1982; Ohser & Schladitz, 2009). For each point x of the boundary ∂X, two curvatures $\kappa_1(x)$ and $\kappa_2(x)$ can be defined. The integral of their average over the surface, known as the **integral of the mean curvature**, is proportional to the mean breadth:

$$\bar{b}(X) = \frac{1}{2\pi} \int_{\partial X} \frac{\kappa_1(x) + \kappa_2(x)}{2} dx \tag{7.11}$$

In practice, the mean breadth may be computed from digital binary images by considering elementary configurations of 2×2×2 voxels, identifying the contribution of this configuration to the total mean breadth, and combining with the histogram of binary configuration within the image (Ohser & Schladitz, 2009; Legland et al., 2007).

7.2.1.4. Euler number

As in 2D, the **3D Euler number** also quantifies the topology of a set. It corresponds to the number of connected components, minus the number of "handles" or "tunnels" through the structure, plus the number of bubbles within the particles(Serra, 1982; Ohser & Schladitz, 2009), see Figure 7.12.

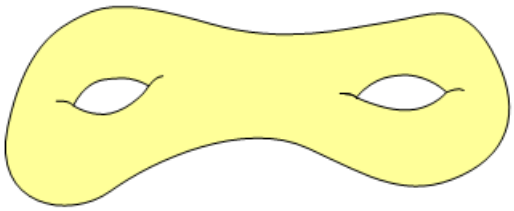


Figure 7.12.: *Euler Number of a 3D particle. The Euler number equals -1, corresponding to the subtraction of 1 connected components minus two handles.*

For a set X with smooth boundary ∂X, the 3D Euler number is proportionnal to the integral of the **gaussian curvature**, corresponding to the product of the curvatures:

$$\chi(X) = \frac{1}{4\pi} \int_{\partial X} \kappa_1(x) \cdot \kappa_2(x) dx \tag{7.12}$$

As for 2D images, the measurement of Euler number from 3D images also relies on the choice of a connectivity between adjacent voxels. In 3D, the 6-connectivity considers the neighbors in the three main directions within the image, whereas the 26 connectivity also considers the diagonals. Other connectivities have been proposed but are not implemented in MorphoLibJ (Ohser & Schladitz, 2009). Note that depending on the choice of connectivity, very different results may be obtained. Such differences may result from the small irregularities arising in images after segmentation. A typical work-around is to regularize the binary or label image, for example by applying morphological opening and / or closing.

7.2.2. Geometric moments and equivalent Ellipsoid

The mathematical definition of the **geometric moments** for 3D regions is similar to the 2D case:

$$m_{pqr}(X) = \int\int\int I_X(x,y,z)x^p y^q z^r \cdot dx \cdot dy \cdot dz \tag{7.13}$$

The first moment $m_{000}(X)$ corresponds to the volume of the particle. The normalization of the first-order moments by the volume leads to the 3D centroid of the particle. The second-order moments can be used to compute an equivalent ellipsoid, defined as the ellipsoid with the same moments up to the second order as the region of interest (see appendix B).

Equivalent Ellipsoid plugin

Returns the parameters of the equivalent ellipsoid into a ResultsTable with the following columns:

Ellipsoid.Center.X the x-coordinate of the center of gravity

Ellipsoid.Center.Y the y-coordinate of the center of gravity

Ellipsoid.Center.Z the z-coordinate of the center of gravity

Ellipsoid.Radius1 the length of the largest semi-axis

Ellipsoid.Radius2 the length of the medium semi-axis

Ellipsoid.Radius3 the length of the smallest semi-axis

Ellipsoid.Phi the azimut of the projection of the major axis on the XY plane, in degrees ("yaw")

Ellipsoid.Theta the elevation of the major axis on the XY plane, in degrees ("pitch")

Ellipsoid.Psi the rotation of the ellipsoid around the main axis ("roll"), in degrees

The three angles correspond to a succession of three rotations (see Figure 7.13):

1. a rotation $R_x(\psi)$ about the x-axis by ψ degrees (positive when the y-axis moves towards the z-axis)
2. a rotation $R_y(\theta)$ about the y-axis by θ degrees (positive when the z-axis moves towards the x-axis)
3. a rotation $R_z(\varphi)$ about the z-axis by φ degrees (positive when the x-axis moves towards the y-axis)

The global rotation matrix is assimilated to the product $R_z(\varphi) \cdot R_y(\theta) \cdot R_x(\psi)$.

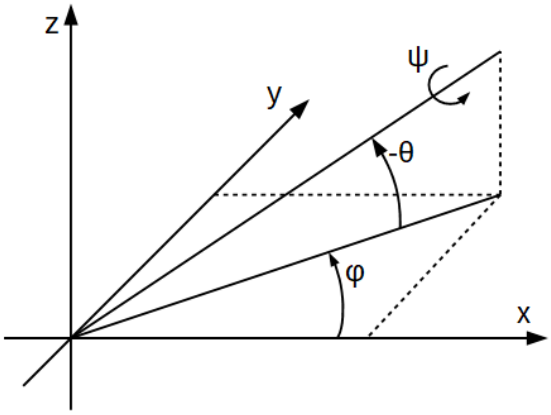


Figure 7.13.: *Definition of angles for representing the orientation of equivalent ellipsoid.*

7.2.3. 3D Shape indices

In 3D, shape indices can be also defined to quantify the shape of the regions independently of their size.

Sphericity

The **sphericity index** is the generalisation of the circularity to 3D. It can be defined as the ratio of the squared volume over the cube of the surface area, normalized such that the value for a ball equals one:

$$\text{sphericity} \quad = \quad 36\pi\frac{V^2}{S^3} \tag{7.14}$$

Other shape factors may be obtained by computing normalized ratios of volume with mean breadth, or surface with mean breadth. Their interpretation is however often not obvious.

Elongation

It is also possible to compute elongation factors from ratio of the radiuses if the equivalent ellipsoid. With three radius values, three ratios can be computed. In MorphoLibJ, the largest radius is used as numerator, resulting in the following ratios:

$$e_{12} = \frac{r_1}{r_2},\ e_{23} = \frac{r_2}{r_3},\ e_{13} = \frac{r_1}{r_3},\ \text{with } r_1 \geq r_2 \geq r_3$$

7.2.4. Plugins

Analyze Regions 3D

The plugin "Analyze Regions 3D" gathers most 3D measures implemented within the MorphoLibJ library. It can be found under "Plugins ▸ MorphoLibJ ▸ Analyze ▸ Analyze Regions 3D". The results are provided in an ImageJ *ResultsTable*, whose name contains the name of the original image.

Interface surface area

The **Interface Surface Area** plugin allows to measure the surface area of the interface between two labels within a 3D label image. The different options are:

Label 1 the first label

Label 2 the second label

Method the number of directions to use for the computation (either 3 or 13).

One of the two labels can have the value 0. In this case, the interface of the other label with the background is measured.

If the two regions do not touch anywhere, the resulting value will be 0. The region adjacency graph (section 9.5.1) can be used to identify neighbor regions.

7.3. Intensity measurements

Other measurements are provided for pairs of grayscale and label 2D or 3D images ("Plugins ▸ MorphoLibJ ▸ Analyze ▸ Intensity Measurements 2D/3D"). The label image can correspond to a segmented particle, or to a more generic region of interest.

The plugin calculates the mean, standard deviation, maximum, minimum, median, mode, skewness and kurtosis of the intensity value distribution of each labeled region in the grayscale image as well as the same statistics of the neighboring (adjacent) regions of each of the labels. If a regions has no adjacent labels, their measurements will appear as NaN (not a number). The results are displayed in an ImageJ *ResultsTable*.

7.4. Label Overlap measures

Given two label images, there are different measures that allow us to evaluate the overlap agreement (or error) between the labels. Following Tustison & Gee (2009), and given a source image S and a target image T, this plugin provides the following overlap measurements in two different result tables (one with the total values for all labels and one with values for individual labels):

- Target Overlap for each individual labeled region r:

$$TO_r = \frac{|S_r \cap T_r|}{|T_r|}$$

- Total Overlap (for all regions):

$$TO = \frac{\sum_r |S_r \cap T_r|}{\sum_r |T_r|}$$

- Jaccard Index or Union Overlap for each individual labeled region r:

$$UO_r = \frac{|S_r \cap T_r|}{|S_r \cup T_r|}$$

- Jaccard Index or Union Overlap for all regions:

$$UO = \frac{\sum_r |S_r \cap T_r|}{\sum_r |S_r \cup T_r|}$$

- Dice Coefficient or Mean Overlap for each individual labeled region r:

$$MO_r = 2\frac{|S_r \cap T_r|}{|S_r| + |T_r|}$$

- Dice Coefficient or Mean Overlap for all regions:

$$MO = 2\frac{\sum_r |S_r \cap T_r|}{\sum_r (|S_r| + |T_r|)}$$

- Volume Similarity for each individual labeled region r:

$$VS_r = 2\frac{|S_r| - |T_r|}{|S_r| + |T_r|}$$

- Volume Similarity for all regions:

$$VS = 2\frac{\sum_r (|S_r| - |T_r|)}{\sum_r (|S_r| + |T_r|)}$$

- False Negative Error for each individual labeled region r:

$$FN_r = \frac{|T_r \setminus S_r|}{|T_r|}$$

- False Negative Error for all regions:

$$FN = \frac{\sum_r |T_r \setminus S_r|}{\sum_r |T_r|}$$

- False Positive Error for each individual labeled region r:

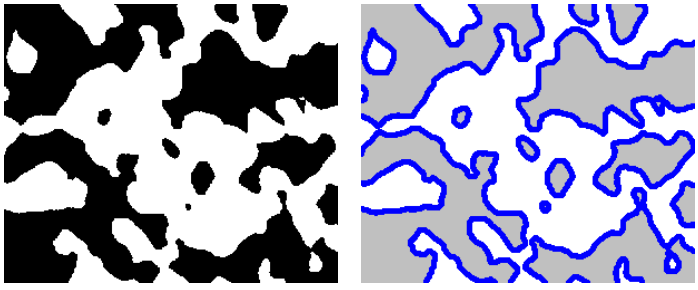
$$FP_r = \frac{|S_r \setminus T_r|}{|S_r|}$$

- False Positive Error for all regions:

$$FP = \frac{\sum_r |S_r \setminus T_r|}{\sum_r |S_r|}$$

7.5. Microstructure analysis

In some cases, the content of the image to analyze is better seen as a representative observation of a stationnary binary process. This context arise frequently in material sciences or in the study of porous media such as bone material (Lang et al., 2001; Ohser & Schladitz, 2009; Doube et al., 2010). An example of such image (obtained from a dairy gel) is represented in Figure 7.14-a.



(a) Sample image **(b)** Perimeter density

Figure 7.14.: *Analysis of binary microstrucure. (a) Original binary image. (b) Estimation of perimeter density.*

To analyze such images, the notion of particle analysis is not relevant anymore. One possibility is to adapt the definitions of intrinsic volumes, by considering that the image corresponds to a sampling window. This induces the two modifications:

1. the integral over boundaries are restricted to the boundary **within the sampling window** (Fig. 7.14-b)

2. the result is normalized by the area (or the volume) of the sampling window.

For the planar case, one obtains the following definitions:

$$A_A(X) = \frac{\int_{X \cap W} dx}{A(W)} \tag{7.15}$$

$$P_A(X) = \frac{\int_{\partial X \cap W} dx}{A(W)} \tag{7.16}$$

$$\chi_A(X) = \frac{\int_{\partial X \cap W} \kappa(x) dx}{A(W)} \tag{7.17}$$

The plugins are available in "Plugins ▸ MorphoLibJ ▸ Analyze ▸ Microstructure Analysis" and "Plugins ▸ MorphoLibJ ▸ Analyze ▸ Microstructure 3D".

8. Binary images

Binary and label images are a convenient way of representing the result of segmentation algorithms. The MorphoLibJ library provides several plugins for the processing and the management of binary and label images.

Contents

8.1. Distance transforms 75
8.1.1. Distance transform 75
8.1.2. Euclidean distance transform 75
8.1.3. Chamfer distances 76
8.1.4. Case of label images 79
8.1.5. Plugin usage 80
8.2. Geodesic distance transform 81
8.2.1. Plugins 82
8.2.2. Results interpretation 82
8.2.3. Algorithms 83
8.3. Connected component labeling 84
8.4. Binary images processing 85
8.4.1. Convexification 85
8.4.2. Selection of binary regions 85
8.4.3. Utilities 86

8.1. Distance transforms

8.1.1. Distance transform

When analyzing images, it is often necessary to compute distances to a particular structure or position. A convenient operator for binary images is the **distance transform**. Its principle is to compute, for each foreground pixel, the distance to the nearest background pixel (Fig. 8.1). The result is commonly referred to as **Distance Map**.

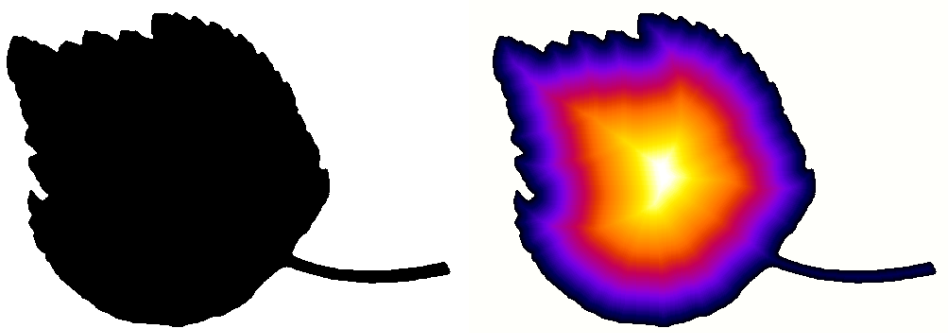


Figure 8.1.: *Binary image, and result of computation of the distance transform.*

Different algorithms exist for computing distance transforms, mostly depending on the choice of the metric for the distance. MorpholLibJ provides implementations for both Euclidean and chamfer masks distance transforms.

8.1.2. Euclidean distance transform

The most natural choice for computing distance map is to consider the **Euclidean distance** to the background. Several algorithms have been proposed, with various performance (Borgefors, 1984; Cuisenaire, 1999; Maurer et al., 2003; Fabbri et al., 2008; Wang & Tan, 2013). The MorphoLibJ library implements Euclidean distance transforms based on the algorithm from Saito & Toriwaki (1994). This algorithm has the advantage of being rather simple to implement, and to generalize easily to arbitrary dimension.

8.1.3. Chamfer distances

Chamfer distance transform is an alternative to Euclidean distance transform that is often simpler to compute (Rosenfeld & Pfalz, 1968; Borgefors, 1984, 1986). The principle is to update the distance associated to each pixel by considering the distances within a small neighborhood, and adding a weight depending on the relative location of the neighbor (Figure 8.2).

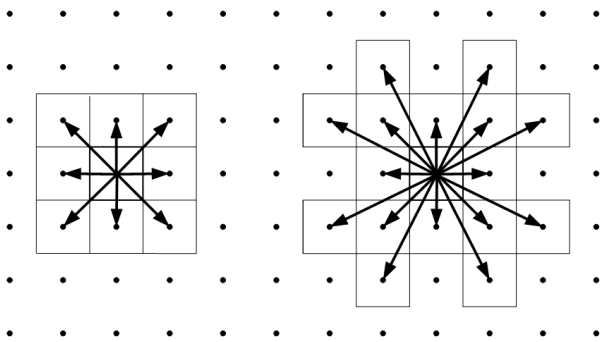


Figure 8.2.: *Computation of distance maps using chamfer masks. The distance associated to each pixel is updated according to the distances associated to the neighbor pixels. Left: 3x3 neighborhood. Right: 5x5 neighborhood.*

In practice, using integer weights may result in faster computations. This allows to store the resulting distance map within 16-bits grayscale images, resulting in more efficient memory usage than 32-bits floating point computation, and may be computationnally more efficient.

Several typical choices for chamfer weights are illustrated on Figure 8.3:

- The simplest one is the **"Chessboard" mask**, that considers distance equal to 1 for both orthogonal and diagonal neighbors (Fig. 8.3-a). The propagation of distances from a center pixel follow a square pattern.
- The **City-block** or **Manhattan mask** uses a weight equal to 2 for diagonal neighbord, resulting in diamond-like propagation of distance (Fig. 8.3-b).
- The "**Borgefors**" distance uses the weight 3 for orthogonal pixels and 4 for diagonal pixels (Borgefors, 1986). It was claimed to be best integer approximation when considering the 3-by-3 neighborhood (Fig. 8.3-c).
- The "**Chess-knight**" distance (Das & Chatterji, 1988) also takes into account the pixels located at a distance from $(\pm 1, \pm 2)$ in any direction (Fig. 8.3-d). It is usually the best choice, as it considers a larger neighborhood.
- The MorphoLibJ library also provides chamfer masks with four weights, as proposed by Verwer et al. (1989). The fourth weight corresponds to pixels located at a distance of $(\pm 1, \pm 3)$ from the reference pixel, in any direction.



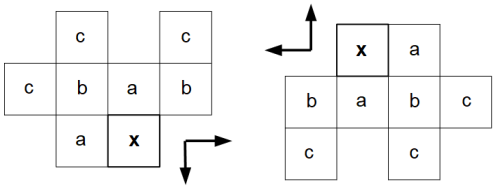
(a) Chessboard (1,1) **(b)** City-block (1,2) **(c)** Borgefors (3,4) **(d)** Chess-knight (5,7,11)

Figure 8.3.: *Several examples of distance maps computed with different chamfer masks. For each image, the distance to the lower-left pixel is represented both with numeric value and with color code. Numbers in brackets correspond to the weights associated with orthogonal and diagonal directions. For chess-knight weights, the weight 11 corresponds to a shift of* $(\pm 1, \pm 2)$ *pixels in any direction.*

As chamfer masks are only an approximation of the real Euclidean distance, some differences are expected compared to the actual Euclidean distance map. Using larger weights reduces the relative error, but the largest possible distance (that depends on the bit-depth of the output image) may be reached faster, leading to "saturation" of the distance map.

8.1.3.1. Chamfer distance computation

A great advantage of chamfer masks is that they allow fast computation of distance maps by dividing the process into two passes. During the first pass (in "forward" direction), pixels are iterated from left to right and from top to bottom. The distance associated to each pixel is updated based on pixels already updated, located upper and on the left (Figure 8.4-a). During the second pass, pixels are iterated "backward" (from right to left and from bottom to top), and the distance map is updated based on the pixels located bottom and on the right (Figure 8.4-b).

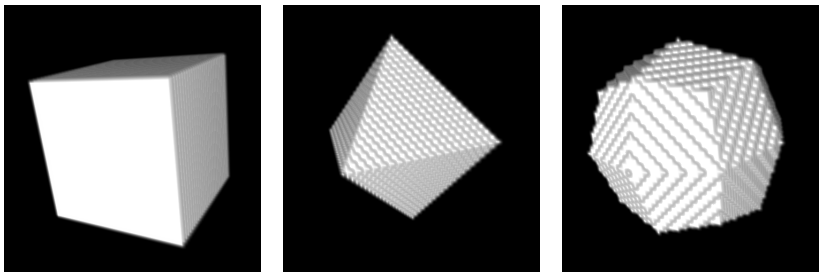


(a) Forward mask **(b)** Backward mask

Figure 8.4.: *Implementation of two-passes chamfer distances. The 5x5 chamfer mask is decomposed into two "half-masks", one for each pass.*

8.1.3.2. 3D chamfer masks

For 3D images, chamfer distance maps can be computed using the same principle. Three different chamfer weights can be associated to the 6 orthogonal neighbors, to the 12 neighbor voxel sharing an edge with reference voxel, and to the 8 neighbor voxel sharing only a corner with reference voxel. The results can be represented via the discrete ball, obtained by applying a threshold on the distance map computed from a single voxel located in the center of the image (Fig. 8.5).



(a) Chessboard (1,1,1) **(b)** City-block (1,2,3) **(c)** Borgefors (3,4,5)

Figure 8.5.: *Discrete 3D balls resulting from different chamfer distances. The distance map is computed to the center voxel, then the result is thresholded and represented in 3D.*

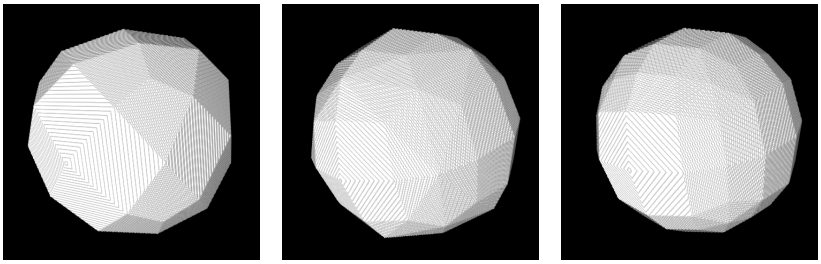
Several 3D chamfer distances can be used:

- The (3D) chessboard distance uses the same weight for all the 26 neighbors. This results in distance propagation in a cubic shape pattern (Fig. 8.5-a).
- The (3D) Manhattan distance considers weights triplet equal to $(1, 2, 3)$, resulting in octagon-like distance propagation (Fig. 8.5-b).
- The Borgefors weights (3,4,5) usually give a good approximation of the Euclidean distance when considering only the 3-by-3-by-3 neighborhood (Fig. 8.5-c).

Larger neighborhood may be considered, resulting in a better approximation (Fig. 8.6):

- Svensson and Borgefors found that adding a fourth weight for voxels shifted by $(\pm 1, \pm 1, \pm 2)$ in any direction results in a substantial error reduction while preserving a low complexity of the algorithm (Svensson & Borgefors, 2002b,a).
- Starting from version 1.5, MorphoLibJ allows using up to six weights (corresponding to the complete 5–by–5-by–5 neighborhood).

As for the 2D case, the choice of the chamfer distance is a compromise between the accuracy (larger neighborhood and weights are usually more accurate) and the largest distance that can be represented.



(a) Svensson (3,4,5,7) **(b)** Five weights **(c)** Six weights

Figure 8.6.: *Enhancement of 3D distance maps when using larger chamfer masks. Masks are based on (a) four, (b) five, or (c) six weights. All images represent the threshold for the distance 100 from image center.*

8.1.4. Case of label images

The distance transform algorithms may also be applied to label images. However, as regions corresponding to each label may be continuous, the algorithms need to be adapted.

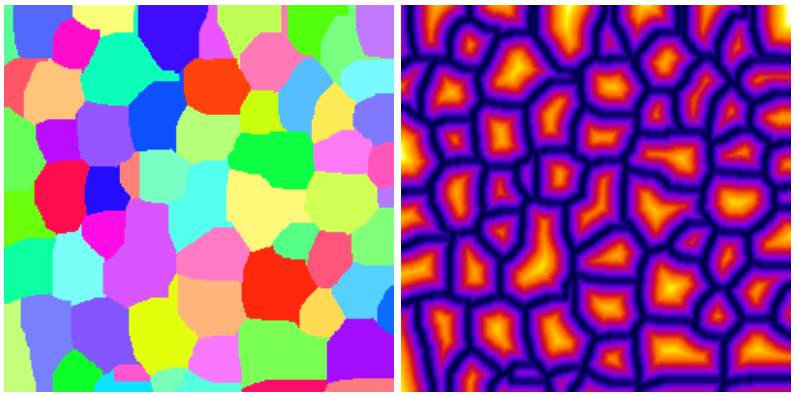


Figure 8.7.: *An example of a label image containing contiguous regions, and result of the distance transform applied on the label image.*

For label image, distances are computed between each foreground (greater than zero) pixel and the nearest pixel with a different value : either 0 (the background) or another label. Distances are propagated as for binary images. An example of result is provided on Figure 8.7.

8.1.5. Plugin usage

The MorphoLibJ library provides two plugins for computing chamfer distance maps for 2D or 3D binary images:

Euclidean Distance Map
Computes the Euclidean distance map from a binary or label image, between each foreground pixel to the nearest pixel with a different value (either background or another label).

Euclidean Distance Map 3D
Computes the Euclidean distance map from a 3D binary or label image, between each foreground pixel to the nearest pixel with a different value (either background or another label).

Chamfer Distance Map
Computes a chamfer distance map from a binary or label image, between each foreground pixel to the nearest pixel with a different value (either background or another label).

Chamfer Distance Map 3D
Computes a chamfer distance map from a 3D binary image between each foreground voxel to the nearest voxel with a different value (either background or another label). Input image is assumed to have cubic voxels.

The following options may be adjusted:

Distances — the weights of the chamfer mask used for computing the distance map

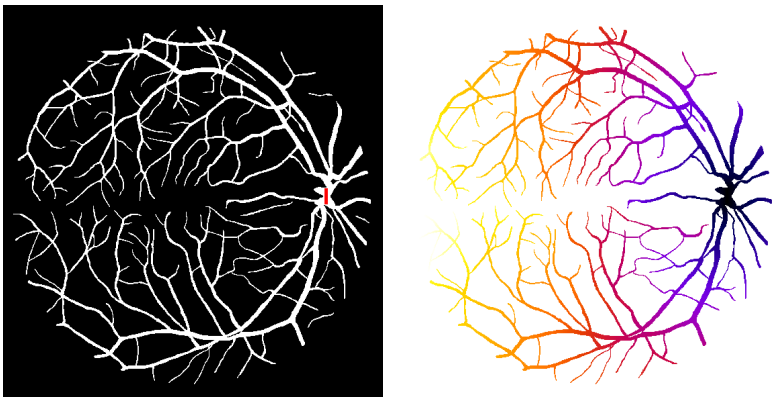
Output Type — specify if the result should be stored in 16-bits image (uses less memory), or 32-bits image (larger distances can be computed).

Normalize — specify if the resulting map should be divided by the weight associated with orthogonal pixels or voxels. This may be useful for masks with large weights to have resulting distances comparable to Euclidean ones.

Preview — (2D only) enable the preview of the result

8.2. Geodesic distance transform

In some cases it may be useful to restrict the propagation of distances to a specific region or mask. For example, one may be interested in the distance between two points in a vasculature network, while staying within the network. The **geodesic distance transform** consists in computing the distance from a given binary marker, while constraining the propagation of the distance within a binary mask. An illustration is given in Figure 8.8, and a 3D example is shown in Figure 8.9.



(a) Original image **(b)** Geodesic distance map

Figure 8.8.: *Computation of the geodesic distance map on a binary image from the DRIVE database (Staal et al., 2004). (a) Original image with marker superimposed in red. (b) Color representation of the geodesic distance map: hot colors correspond to large distances, cold colors correspond to small distances.*

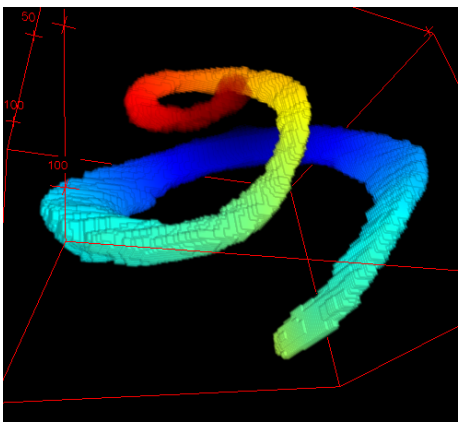


Figure 8.9.: *Computation of the geodesic distance map on a 3D binary image.*

8.2.1. Plugins

Geodesic Distance Map

Computes the geodesic distance between each foreground pixel of a binary mask image to the closest pixel of a marker image, while staying within the particle represented by the mask image (See Figure 8.8).

Interactive Geodesic Distance Map

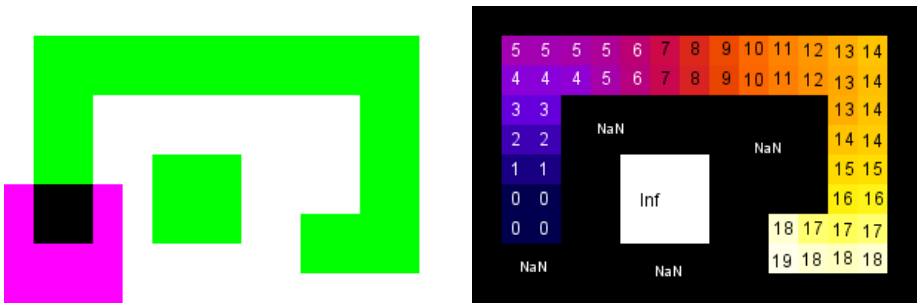
Computes the geodesic distance between each foreground pixel of a the currently selected image (considered the mask image) to the closest pixel of a marker image defined by the user ROIs, while staying within the particle represented by the mask image.

Geodesic Distance Map 3D

Computes the 3D geodesic distance between each foreground pixel of a binary mask image to the closest pixel of a marker image, while staying within the particle represented by the mask image (See Figure 8.9). Binary image is assumed to have cubic voxels.

8.2.2. Results interpretation

Geodesic distance transforms plugins result in images that contains the geodesic distance of foreground pixels or voxels, but also background pixels/voxels of the mask. Also, some regions may not be reacheable by the propagation algorithm.



(a) Input marker and mask images. **(b)** Geodesic Distance Map.

Figure 8.10.: *Interpretation of geodesic distance maps. (a) Input marker (magenta) and mask images (green). Overlay of marker and mask is shown in black. (b) Geodesic Distance Map. Numeric values correspond to distances (values are rounded for readability). Non-reacheable components are set to +infinity. Background of original mask is set to NaN.*

Values in geodesic distance maps can be interpreted this way:

- 0: the pixel is contained both within the original marker and mask images (distance is zero by definition)
- numerical value: the value corresponds to the length of the path that joins the current pixel or voxel to the closest pixel or voxel belonging to the mask.

- NaN: the pixel/voxel does not belong to the mask, and is not considered during propagation.
- "Infinity", or very large value: the pixel does belong to the mask, but can not be reached by propagation. This corresponds to the case where the mask is composed of several components.

Geodesic distance maps are computed by propagating chamfer weights, as for the computation of distance maps. As chamfer weights are only an approximation of the real Euclidean distance, some differences are expected compared to the actual geodesic distance.

When computation are performed with integers, special values like Inf or Nan can not be represented. In these cases, they are replaced by the largest possible value for 16-bits unsigned integer type.

8.2.3. Algorithms

Various algorithms are decribed in Soille (2003). The simplest one consists in iterating forward and backward passes that update the result map according to the neighbors pixels, until no more modification occurs. An alternative is to propagate the distance maps from the pixels within the markers using a priority queue.

The MorphoLibJ plugins use an hybrid algorithm, that initializes the distance map by a single sequence of forward and backward passes, and initializes the queue during the backward pass. This reduces the amount of memory required by the priority queue.

8.3. Connected component labeling

The different structures within a binary images can be individualised by using a connected **component labeling algorithm**. The result is an image containing integer label, each label corresponding to a set of connected pixels or voxels (Figure 8.11). Label images can be represented with colored LUT to help distinguish adjacent labels (see section 9.1).

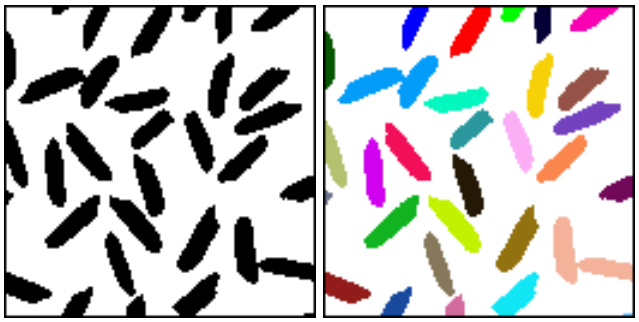
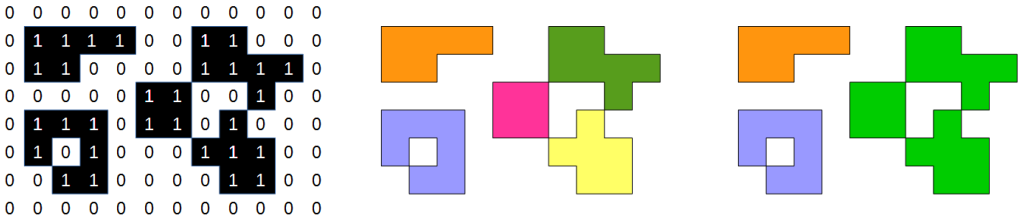


Figure 8.11.: *Binary image, and result of connected components labeling.*

The result of a connected component labeling depends on the chosen **connectivity**, that corresponds to the convention used to decide wether two pixels or voxels are connected or not (Figure 8.12). See also Section 3.3.

For planar images, a typical choice is the 4-connectivity, that considers only the orthogonal neighbors of a given pixel (Figure 8.12-b). When two components touch only via a corner, they are not considered as the same one. The 8-connectivity is an alternative that also considers the diagonal pixels as neighbors (Figure 8.12-c).



(a) Binary image **(b)** 4-connectivity **(c)** 8-connectivity

Figure 8.12.: *Impact of the connectivity on the result of connected component labeling. (a) Original binary images. (b) Connected component labeling using the 4-connectivity. (c) Connected component labeling using the 8-connectivity.*

For 3D images, the 6-connectivity considers only orthogonal neighbors in the three main directions, whereas the 26-connectivity considers all the neighbors of a given voxel in its 3-by-3-by-3 surrounding. The connectivity is also considered for computing morphological reconstructions (section 5.1), or for computing the Euler number of binary or label images (section 7.1.1.3).

The number of labels that can be represented within a label map depends on the image type: 255 for byte images, 65535 for short images, and around 16 millions for 32-bit floating point images (only the 24 bits of the mantissa are used for label representation, leading to a maximum number of labels equal to 2^{24}).

Connected Components Labeling

transforms the binary image into a label image by assigning a specific number (label) to each connected component

8.4. Binary images processing

8.4.1. Convexification

When processing a binary region, it can sometimes be useful to consider the binary region corresponding to the convex hull of the original region (Fig. 8.13). For example, the convexity measure (section 7.1.6.2) is based on the convex hull. The Convexify plugin allows to generate the smallest convex region containing a binary region.



(a) Binary image. **(b)** Convexification. **(c)** Original image over its convex hull.

Figure 8.13.: *Convexification of a binary image containing several disjoint components.*

Convexify

for a binary image, generate the smallest convex region containing a region in the image.

8.4.2. Selection of binary regions

The MorphoLibJ library offers several tools for automatically selecting binary regions based on size or position criteria. Some of them are illustrated on Figure 8.14. As the algorithms usually requires connected component labeling, it is often more convenient to use corresponding methods for label images (Section 9.4.1).

Keep Largest Region

Identifies the largest connected component, and remove all other regions.



Figure 8.14.: *Utilities for binary images. From left to right: original image, keep largest region, remove largest region, apply size opening for keeping only regions with at least 150 pixels.*

Remove Largest Region

Identifies the largest connected component, and remove it from image.

Area Opening

Computes the size of each region (as the number of voxels), and removes all regions whose size is below the specified threshold. The plugin works only for 2D images, but provides a preview.

Size Opening 2D/3D

computes the size (area in 2D, volume in 3D) of each connected component, and remove all particles whose size is below the value specified by the user. Algorithms work for both 2D or 3D images. Default connectivity 4 (resp. 6) is used for 2D (resp. 3D) images.

8.4.3. Utilities

Binarize Image

Converts the current image into a binary image, by setting all values greater than 0 to 255, and the values (lower than or) equal to zero to 0. Can be used to quickly convert a label map into a binary image, with display options consistent within the MorphoLibJ library.

Binary Overlay

Computes the color image obtained by combining a reference image, a binary image, and an overlay color. If a pixel is set to true in the binary image, it is set to the overlay color. Otherwise, it is set to the value of the reference image. See also the Binary/Labels Overlay plugin (section 9.1.1).

9. Label images

When several structures or components are present within an image, it may be convenient to work with **label images**, or **label maps**. Within a label map, each pixel or voxel of a label image corresponds to the integer index of the particle it belongs to. By convention, the value 0 corresponds to the background.

Label images are typically computed from binary images by **connected component algorithms** (section 8.3). Segmentation algorithms such as the watershed may also return their result as a label image.

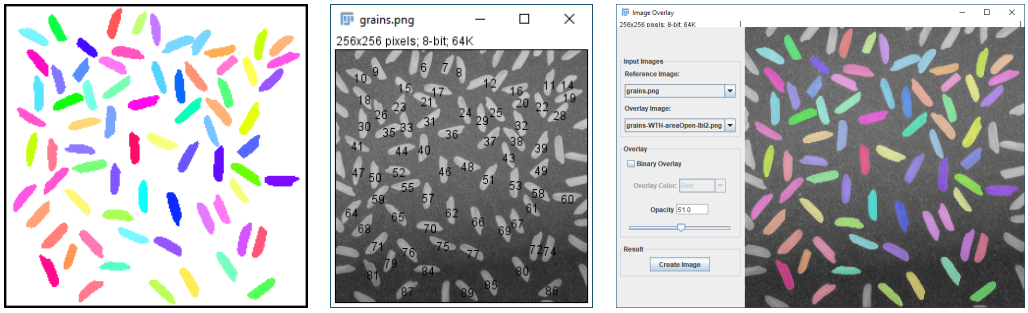
This chapter presents several utilities for managing label maps, in particular for facilitating their visualisation, to convert them into ROI, or to visualize the result of region features. Several plugins allows for selecting or filtering label maps, some of them being implemented into the Label Edition Plugin.

Contents

9.1. Visualization of label images **88**
9.1.1. Plugins 88
9.1.2. Color maps 89
9.2. Conversion to ROI **90**
9.3. Visualization of region features **91**
9.4. Label image processing **92**
9.4.1. Region and label selection 92
9.4.2. Morphological filtering 93
9.4.3. Edition of label maps 95
9.5. Region adjacencies **96**
9.5.1. Region Adjacency Graph 96
9.5.2. Adjacent region boundaries 97
9.5.3. Region adjacencies 98
9.6. Label Edition plugin **99**

9.1. Visualization of label images

Label images can be represented either using shades of gray, or using color maps to better distinguish the different regions. It may also be convenient to overlay the region label on the image to associate it to values within a table, or to convert the boundary of each region into a Region of Interest (ROI) that can be managed within the ROI Manager dialog.



(a) Using Colormap **(b)** Draw Labels as Overlay **(c)** Overlay Label Image

Figure 9.1.: *Various visualization modes of label images. (a) Display regions using appropriate colormap. (b) The label associated to each region is overlaid on another image. (c) Label map or binary images can be overlaid on another grayscale image using user-defined opacity.*

9.1.1. Plugins

Several plugins allow to control the appearance of label images. It is possible to choose a given color map, or to transform a label image into a color image. In both cases, the background color can be specified, and the color order can be shuffled to facilitate the discrimination of neighbor regions with similar labels.

Set Label Map

Allows to choose the color map used to display a label image (Fig. 9.1-a). In particular, shuffling the color map and/or choosing a specific color for background allows better visualization than with only gray levels. Note that when the number of regions is large, regions with small labels will be associated to the same color as background, and will therefore not be visible!

Labels To RGB

Converts a label image to a true RGB image. Similar to ImageJ native conversion, but this plugin avoids confusion between background pixels and labels with small values when the number of labels is large.

Draw Labels As Overlay

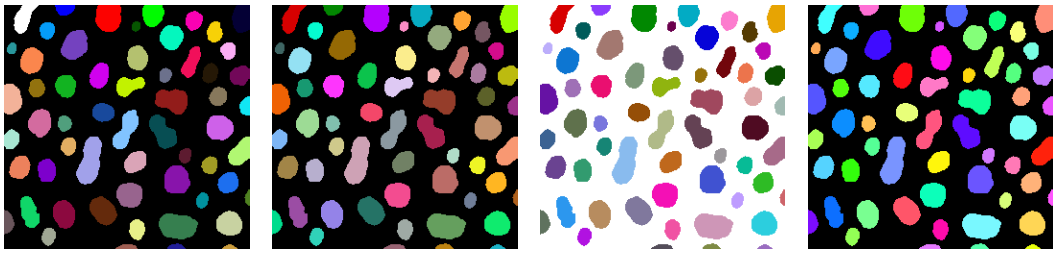
Choose another image, and display the label associated to each region as a text overlay (Fig. 9.1-b). The position of the overlay is the centroid of each region, and a small shift can be specified.

Binary/Labels Overlay

Opens a dialog to choose a reference/background image, and an image to overlay, that can be either a binary image or a label map (Fig. 9.1-c). User can specify the opacity of the overlay, and generate a new Color image for the result. See also the Binary Overlay plugin (section 8.4.3).

9.1.2. Color maps

When the number of labels within a label image is large, it may be difficult to differentiate regions whose labels are close. The Figure 9.2 presents various combinations of colormaps and background colors.



(a) Glasbey, Black **(b)** GlasbeyBright, Black **(c)** GlasbeyDark, White **(d)** Golden Angle, Black

Figure 9.2.: *Several choices of colormap for visualization of a label image.*

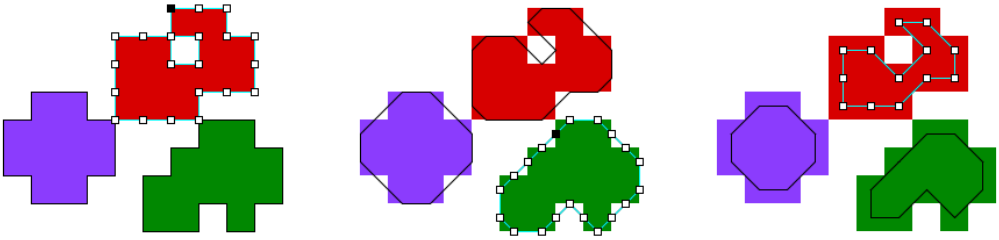
The "Glasbey" colormap, included within ImageJ, generates colors that are perceptually contrasted (Figure 9.2-a). It however generates black and white colors, making them difficult to distinguish from background (on the example from Figure 9.2-a, the region in the top-right corner is not visible anymore). Therefore, the "Glasbey Bright" and "Glasbey Dark" colormaps were made available, to provide good contrast both between labels and with background (Glasbey et al., 2007; Kovesi, 2015). The choice of these colormaps was inspired by the colorcet library[1]. The Golden Angle colormap is an alternative to the Glasbey family, that provides more saturated colors (Figure 9.2-d). The contrast between colors is less than for Glasbey colormaps, making adjacent region potentially more difficult to distinguish.

[1] https://colorcet.holoviz.org/index.html

9.2. Conversion to ROI

When working with label maps, it is often convenient to convert one or several regions into a Region Of Interest (ROI) that can be managed by the ROI Manager from ImageJ. Several macros and plugins have been developed for this, see for example the BIOP plugin "Label image to ROIs"[2].

MorphoLibJ also provides a plugin "Label Map To ROI". Its principle is based on the joint labelingand contour tracing algorithm introduced by Chang et al. (2004) (see also Burger & Burge, 2008). Image pixels are iterated starting from top-left corner. When a non-background pixel is encountered, a new contour polygon is initialized. The next contour pixel is identified from the neighbor pixels, and taking into the connectivity. Contour pixels are tracked along the boundary until we come back to the starting point.



(a) Corners **(b)** Edge Middles **(c)** Pixel Centers

Figure 9.3.: *Differerent results of the "Label Map To ROIs" plugin, using different vertex locations. Boundary polygons are shown in black. For each image, one of the polygons is selected and displayed in light blue. For the three examples, labels were obtained with connectivity 4, and the ROI polygons were computed with the same connectivity.*

Label Map To ROIs

Converts regions of a label map into a series of ROI, by traveling along the boundary pixels of the regions. For each region, one or several polygonal ROI are created, depending on the topology of the region. Different options can be chosen:

Connectivity the connectivity to use for iterating over the boundary pixels. It is advised to use the same connectivity as the one that was used for computing connected components.

Vertex Location determines the location of the vertices used for creating the polygons, with respect to the position of the boundary pixels. The different choices are illustrated on Figure 9.3.

Name_Pattern the string pattern used to name each ROI. When a region boundary is composed of several polygons, they are identified by a trailing dash and index. Example: "r003-2" corresponds to the second boundary polygon of the region 3.

[2] https://github.com/BIOP/ijp-LaRoMe

9.3. Visualization of region features

When images contain many labels, it is may be difficult to interpret together the localisation and the features describing the regions. The “Assign Measure To Label” and “Draw Label Values” plugins provide some help to facilitate this exploration.

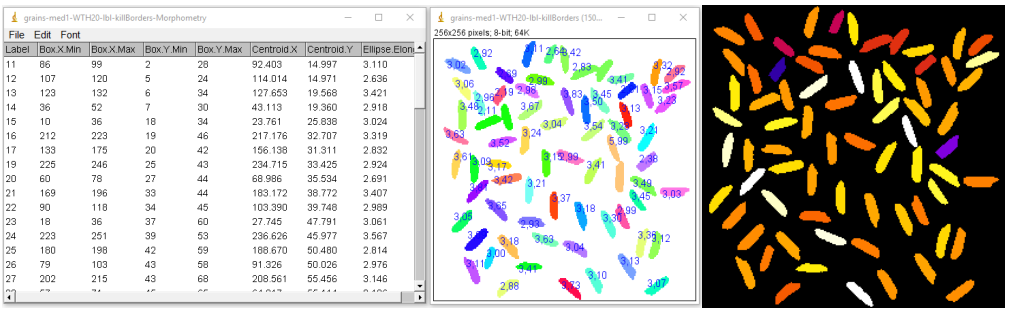


Figure 9.4.: *Various possibilities to graphically visualize region feature. On the left, the table used for display. Center, overlay of the elongation feature using the “Draw Table Values” plugin. On the right, color representation of the elongation using a color code: between dark purple (circular, compact) to white (very elongated).*

Draw Table Values

draws the value of a feature measured on each label of an image using two coordinates (bounding box, centroid...) also stored in a ResultsTable. An example is shown on Figure 9.4.

Assign Measure To Label

combines a label image with a results table, and creates a new 32-bits image for which each pixel/voxel is assigned the measurement value corresponding to the region it belongs to (Figure 9.4). Background pixels/voxels are associated to NaN (Not-A-Number) value.

9.4. Label image processing

9.4.1. Region and label selection

It is often convenient to select regions in a label image based on their size or their location (e.g. removing the regions touching the border). The MorphoLibJ library offers several utility tools for automatically selecting or removing specific regions, or to edit the label images. Some of them are illustrated on Figure 9.5. The Label Edition Plugin (Section 9.6) provides some of these operators integrated into a graphical user interface.

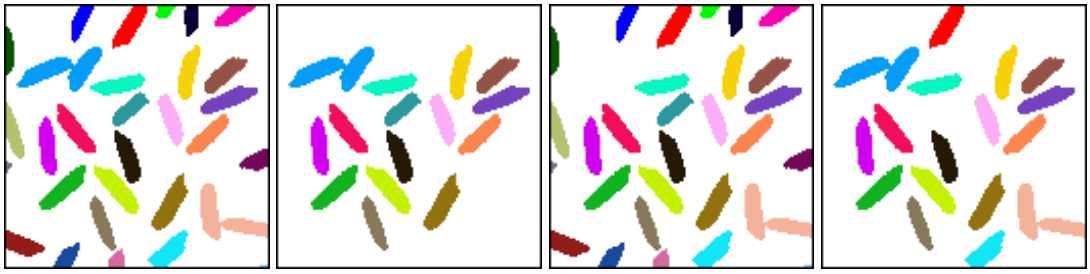


Figure 9.5.: *Utilities for label images. From left to right: original label image, remove border labels, remove largest region, apply size opening for keeping only regions with at least 150 pixels.*

Remove Border Labels

Similar to the "kill borders" function, but operates faster as no morphological reconstruction is required.

Select Label(s)

Enters a set of labels, and creates a new label image containing only the selected labels.

Keep Largest Label

Identifies the largest region within the label, and creates a new label map containing only this region.

Remove Largest Label

Identifies the largest region within the label, and creates a new label map containing all the labels but the largest one.

Label Size Filtering

Filters the labels according to their size (area in 2D, volume in 3D), by specifying a comparison operator and a threshold value.

9.4.2. Morphological filtering

Several morpological filters have been adapted to process label maps.

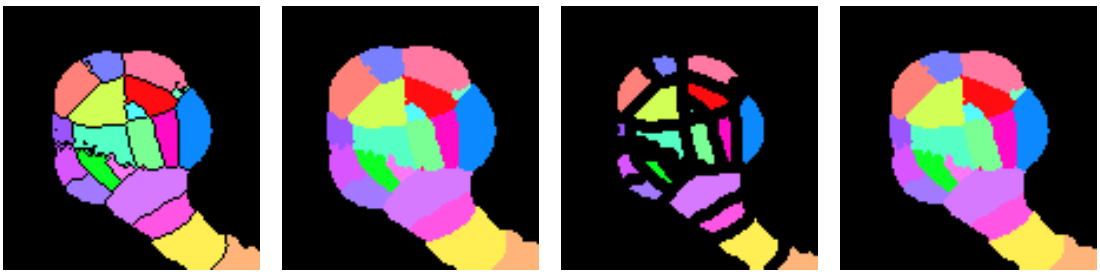
Label Morphological Filtering

Applies morphological filtering operators on the label map, by choosing the type of operation (dilation, erosion, opening or closing), and the radius of the operation. The principle is the same as morphological filtering on grayscale images, but using label values instead of gray levels, and avoiding "collisions" between labels:

- when applying a dilation, only background pixels or voxels are updated, according to the closest label within the neighborhood (Figure 9.6-b). When two regions are adjacent without background pixel between them, the application of the dilation does not transform nor move their boundary.
- when applying an erosion, label pixels or voxels are removed depending on the labels within the neighborhood and on the "any Label" option:
  - if true, current label is removed if any other label (background or another region) is contained within the neighborhood
  - if false, current label is removed if and only if a background pixel or voxel is contained within the neighborhood (Figure 9.6-c).

When the "Any Label" option is set to false, performing a morphological closing results in a label map with more regular regions, with less gaps between regions, and preserved boundaries between adjacent regions (Figure 9.6-d).

The computation of the neighborhood, and of the closest label, is based on chamfer masks (see Section 8.1.1).



(a) Label image **(b)** Dilation **(c)** Erosion **(d)** Closing

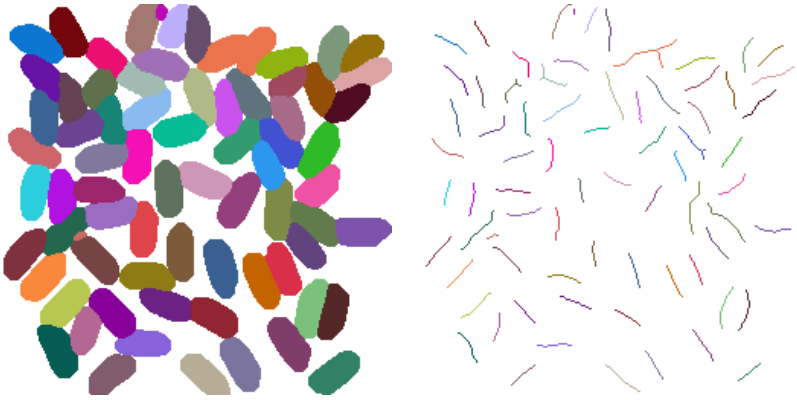
Figure 9.6.: *Morphological filtering of label images. Morphological dilation (b) and erosion (c) take care of adjacent regions.*

Fill Label Holes

Fills the holes within regions. Only the holes that are adjacent to a single region are filled, using a fill corresponding to the label of the adjacent region.

Skeletonize Labels

Computes the skeleton of each region within a 2D label map, and returns the result within a new label map.

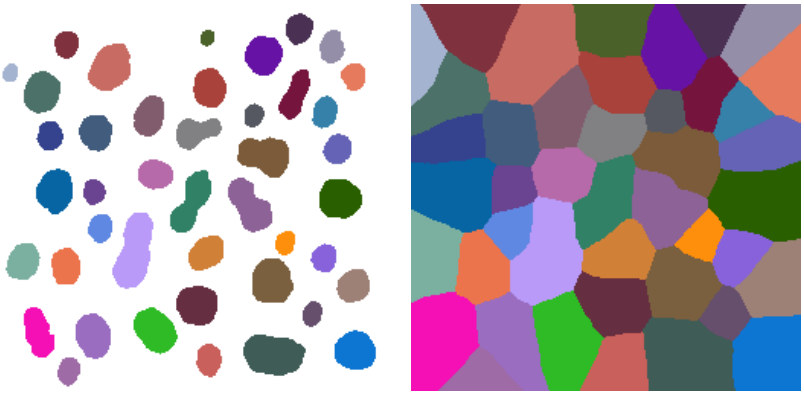


(a) Original Label Map **(b)** Skeleton of label Map

Figure 9.7.: *Skeletonization of a label map with adjacent regions.*

Region Inflence Zones

Computes the **influence zone** of each region within a label map, and returns the result within a new label map. The influence zone of a region is comprises all the pixels within the region, plus the background pixels that are closer to this region than any other region. The result of this plugin may be seen as a special case of label dilation, using an arbitrary large dilation size.



(a) Original Label Map **(b)** Influence Zones

Figure 9.8.: *Computation of influence zones of a label map.*

9.4.3. Edition of label maps

Several plugins allow for basic edition of filtering of label images. They result in another label image.

Crop Label

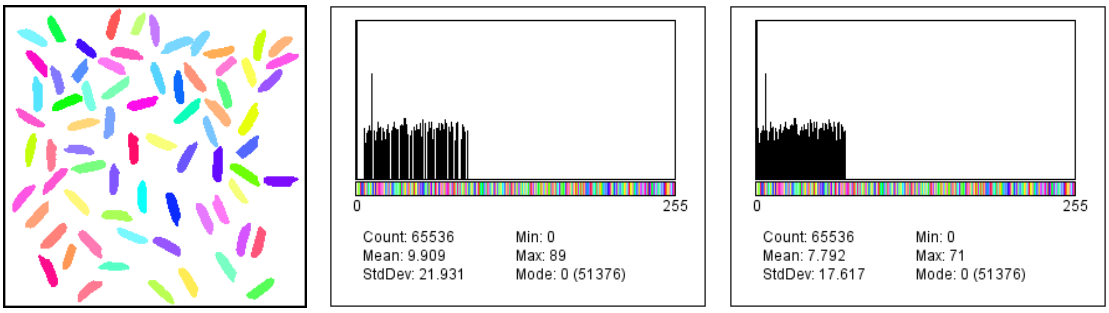
creates a new binary image containing only the label specified by the user. The size of the new image is fitted to the region.

Replace Value

replaces the value of a region by another value. Can be used to "clear" a label, by replacing its value by 0, or to merge to adjacent regions, by replacing the value of a label by the value of its neighbor.

Remap Labels

re-computes the label of each region such that the largest label is equal to the number of labels (Figure 9.9). This tool can be useful to remove "empty labels" that could occur after removal of one or several labels (merge labels, remove labels...).



(a) Label image **(b)** Original histogram **(c)** Remapped histogram

Figure 9.9.: *Illustration of the "Remap Labels" plugin. (a) Original label image. (b) Histogram of original label image, showing missing labels. (c) Histogram of the remapped label image, without missing labels.*

The Figure 9.9 presents a use case of the **Remap Labels** plugin. The left image is a label image in which small labels as well as labels touching image borders have been removed (Fig. 9.9-a). The histogram of the image presents some "holes", corresponding to the labels that have been removed (Fig. 9.9-b). The application of the plugin results in a similar images, without missing labels (Fig. 9.9-c). The number of labels in the image corresponds to the maximal value in the image (in this case, 71).

9.5. Region adjacencies

It may sometimes be useful to consider the adjacency relationships between regions. This can be useful for post-processing of segmentation results (using e.g. graph processing algorithms), or for exploring collections of cells within cellular tissues (Florindo et al., 2016). Several plugins are available within the library. The definition of adjacency is not totally consistent between the plugins, and results may greatly differ when applied on the same input label image.

9.5.1. Region Adjacency Graph

The **region adjacency graph** plugin gives access to the neighborhood relationship between adjacent regions (Figure 9.10).

Region Adjacency Graph

The plugin works for both 2D and 3D images, and requires a label image as input. The output of the plugin is a results table with as many rows as the number of pairs of adjacent regions, containing the labels of the two adjacent regions.

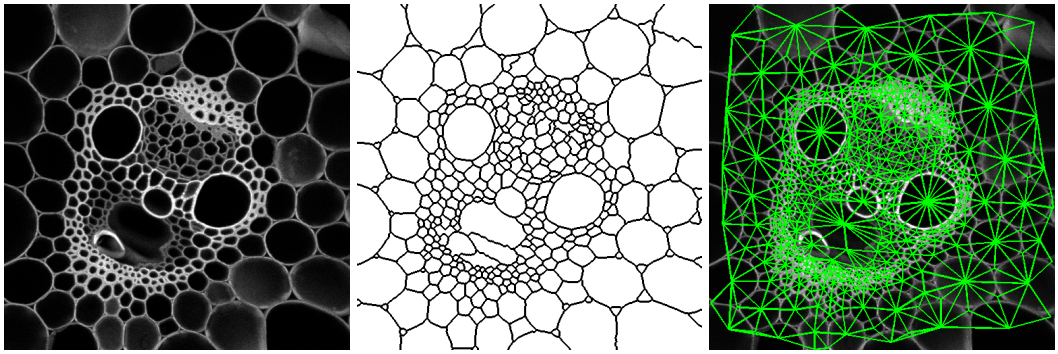


Figure 9.10.: *Computation of the Region Adjacency Graph (RAG) on a microscopy image of plant tissue. Left: original image. Middle: result of watershed segmentation. Right: overlay of edges representing adjacent regions.*

In the current implementation, MorphoLibJ considers that two regions are adjacent if they are separated by 0 or 1 pixel (or voxel) in one of the two (or three) main directions.

9.5.2. Adjacent region boundaries

It may sometimes be useful to visualize the boundaries between adjacent regions (Fig. 9.11).

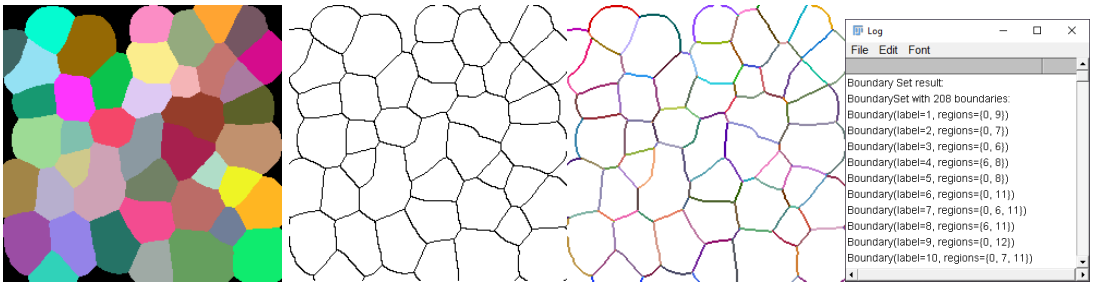


Figure 9.11.: *Example use of the plugin "Label Region Boundaries" plugin. Left: a label image containing regions with various adjacencies. Right: a new label map was created, where each label corresponds to the boundary between two or more regions within the original image.*

The "Label Boundaries" plugin generates a binary image of the boundaries between regions. The "Region Boundaries Labeling" considers similar boundaries, but in addition it associates a label to each boundary.

Label Boundaries

Creates a new binary image containing value 255 for pixels/voxels having a neighbor with a different value. The background is taken into account for computing boundaries. Only the bottom-left neighborhood is checked during image iteration, to generate a thinner boundary image.

Region Boundaries Labeling

Computes a new label map, where each label corresponds to the boundary between two (or more) regions. The background is also considered as a region. The list of identified boundaries together with their label and bounding regions is displayed in the log window. Most boundaries are adjacent to two regions, but the junctions between boundaries may be adjacent to three or more regions. Label map of boundaries is represented using floating point numbers, leading to a maximum number of 2^{23} (around 8.3 millions) boundaries. The number of boundaries is usually much larger than the number of regions, and the maximum number of label may be reached quickly.

See also the Interface Surface Area plugin (section 7.2.4), that measures the surface area between adjacent 3D regions.

9.5.3. Region adjacencies

Another usage of adjacencies is to select regions around a given one.

Select Region Neighbors

Extracts the neighbor regions within a label image from a region label, or a list of region labels (See Fig. 9.12).

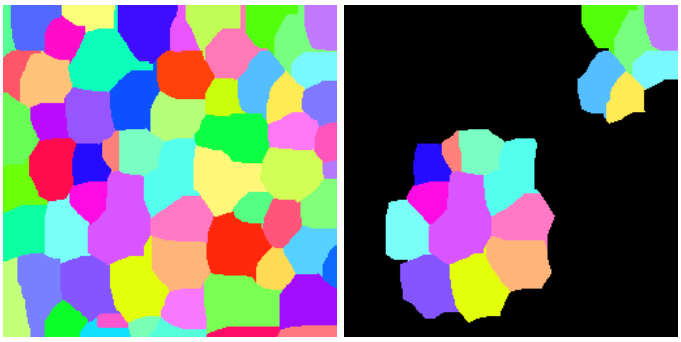


Figure 9.12.: *An example of a label image containing contiguous regions, and result of the selection of the neighbors of two regions (keeping the original regions).*

9.6. Label Edition plugin

To ease the processing of label images, MorphoLibJ provides the Label Edition plugin (available under "Plugins ▸ MorphoLibJ ▸ Labels ▸ Label Edition"). This plugin contains a graphical user interface (GUI) where the users can perform the following set of editing tasks (Figure 9.13):

- Manually merge labels after their selection using the point selection tool (in 2D and 3D).
- Apply morphological erosion, dilation, opening and closing with a square/cube of radius 1 as structuring element.
- Remove labels by area or volume (size opening operation), largest size, manual selection or border location.

All operations are performed "in place", i.e., the input image gets directly modified. However, the initial status of the label image can be recovered by clicking on "Reset".

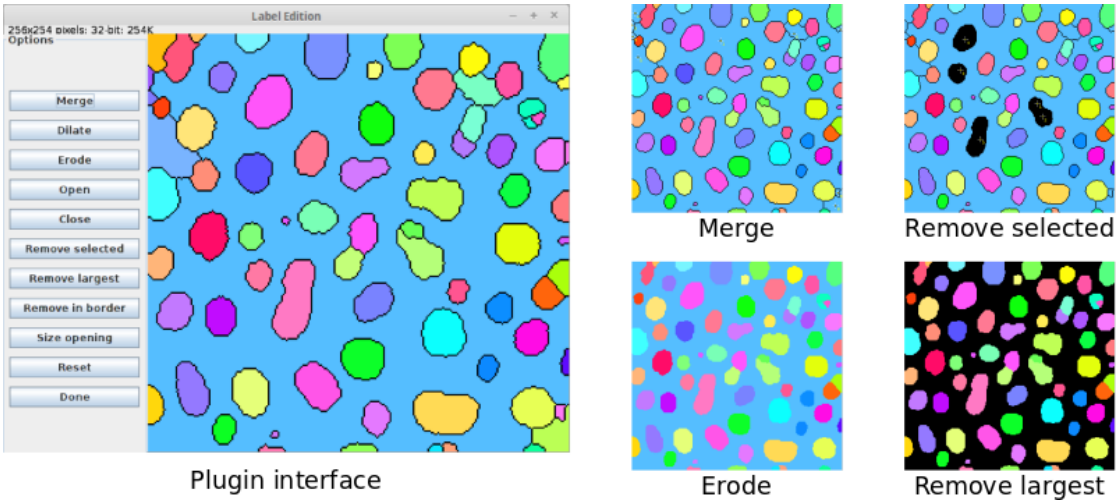


Figure 9.13.: *Label Edition plugin. From left to right and from top to bottom: overview of the plugin GUI on a 2D label image, result of label merging by manual selection, removal of selected labels, label erosion and removal of largest label (in this case, the largest label corresponds to the background).*

10. Library interoperability

A key design concept was the modularity of the implementation to facilitate its reusability. Three layers with different programming abstraction can be identified:

- For final users, plugins provide graphical display and intuitive tuning of parameters. Such plugins can easily be incorporated into a macro:

```
1 // Calls the Regional Min/Max plugin on current ImagePlus instance
2 run("Regional Min & Max", "operation=[Regional Maxima] connectivity=4");
```

- For plugins developers, operators are available through collections of static methods, making it possible to apply most operations with a single line of code. Example:

```
1 // Computes regional maxima using the 4-connectivity
2 ImageProcessor maxima = MinimaAndMaxima.regionalMaxima(image, 4);
```

- For core developers, algorithms are implementations of abstract interfaces, making it possible to choose or develop the most appropriate one, and to monitor execution events.

```
1 // choose and setup the appropriate algorithm
2 RegionalExtremaAlgo algo = new RegionalExtremaByFlooding();
3 algo.setExtremaType(ExtremaType.MAXIMA);
4 algo.setConnectivity(4);
5 // add algorithm monitoring
6 algo.addAlgoListener(new DefaultAlgoListener());
7 // compute result on a given ImageProcessor
8 ImageProcessor result = algo.applyTo(image);
```

In total, the library provides nearly two hundred classes and interfaces.

10.1. Library organization

The library follows a logic structure of folders divided by topics aiming at their re-usability from other plugins or scripts, among others:

inra.ijpb.data contains generic data structures for manipulating 2D or 3D images

inra.ijpb.binary contains the set of utilities for working on binary images (connected component labeling, distance transform, geodesic distance transform...)

inra.ijpb.geometry contains utility functions for geometric computing, and several classes for representing geometric primitives (ellipse, point pair...)

inra.ijpb.label contains the utilities for label images (cropping, size opening, remove border labels, etc)

inra.ijpb.measure contains the tools for geometric and gray level characterization of 2D or 3D images

inra.ijpb.morphology contains the collection of mathematical morphology operators

inra.ijpb.watershed contains the classes implementing the different versions of the watershed algorithm

inra.ijpb.plugins contains the set of plugins that is accessible from ImageJ/Fiji Plugins menu

All major methods have a general class with static methods that allow calling the methods on 2D and 3D images in a transparent way. This modularity permitted to develop other plugins devoted to the analysis of nucleus images (Poulet et al., 2015), gray level granulometries (Devaux & Legland, 2014) or the description of binary microstructures (Silva et al., 2015).

10.2. Scripting MorphoLibJ

One advantage of this organization of the library and the use of public static methods is that it allows very easy and fast prototyping of morphological algorithms and pipelines.

10.2.1. Segmentation pipeline prototype

Let's see an example in a complete Beanshell script that takes the active 2D or 3D image and finds a reasonable segmentation combining a set of morphological operations (gradient, extended minima and watershed):

```
// @ImagePlus imp

// ImageJ imports
import ij.IJ;
import ij.ImagePlus;
import ij.ImageStack;

// MorphoLibJ imports
import inra.ijpb.binary.BinaryImages;
import inra.ijpb.morphology.MinimaAndMaxima3D;
import inra.ijpb.morphology.Morphology;
import inra.ijpb.morphology.Strel3D;
import inra.ijpb.watershed.Watershed;

// create structuring element (cube of radius 1)
strel = Strel3D.Shape.CUBE.fromRadius( 1 );
// apply morphological gradient to input image
image = Morphology.gradient( imp.getImageStack(), strel );
// find regional minima on gradient image with dynamic value of 30 and 6-connectivity
regionalMinima = MinimaAndMaxima3D.extendedMinima( image, 30, 6 );
// impose minima on gradient image
imposedMinima = MinimaAndMaxima3D.imposeMinima( image, regionalMinima, 6 );
// label minima using connected components (32-bit output)
```

```
labeledMinima = BinaryImages.componentsLabeling( regionalMinima, 6, 32 );
// apply marker-based watershed using the labeled minima on the minima-imposed
// gradient image (the "true" value indicates the use of dams in the output)
resultStack = Watershed.computeWatershed( imposedMinima, labeledMinima, 6, true );

// create image with watershed result
resultImage = new ImagePlus( "watershed", resultStack );
// assign right calibration
resultImage.setCalibration( imp.getCalibration() );
// and show it
resultImage.show();
```

10.2.2. Label visualization in 3D viewer

Making use of MorphoLibJ's label methods and the ImageJ 3D Viewer's visualization tools it is quite simple to create a script to display each label of an image as 3D surfaces of the corresponding colors provided by the image look-up table:

```
// @ImagePlus imp
import inra.ijpb.label.LabelImages;
import ij3d.Image3DUniverse;
import ij3d.ContentConstants;
import org.scijava.vecmath.Color3f;
import ij.IJ;
import isosurface.SmoothControl;

// set to true to display messages in log window
verbose = false;

// set display range to 0-255 so the displayed colors
// correspond to the LUT values
imp.setDisplayRange( 0, 255 ); imp.updateAndDraw();

// calculate array of all labels in image
labels = LabelImages.findAllLabels( imp );
// create 3d universe
univ = new Image3DUniverse();
univ.show();
// read LUT from input image
lut = imp.getLuts()[0];
// add all labels different from zero (background)
// to 3d universe
for( i=0; i<labels.length; i++ )
{
  if( labels[ i ] > 0 )
  {
    labelToKeep = new int[ 1 ];
    labelToKeep[ 0 ] = labels[ i ];
    if( verbose )
    IJ.log( "Reconstructing label " + labels[ i ] + "..." );

    // create new image containing only that label
    labelImp = LabelImages.keepLabels( imp, labelToKeep );
    // convert image to 8-bit
    IJ.run( labelImp, "8-bit", "" );
    // use LUT label color
    color = new Color3f( new java.awt.Color( lut.getRed( labels[ i ] ),
              lut.getGreen( labels[ i ] ),
              lut.getBlue( labels[ i ] ) ) );
```

```
42      if ( verbose )
43        IJ.log( "RGB( " + lut.getRed( labels[ i ] ) +", "
44                + lut.getGreen( labels[ i ] )
45                + ", " + lut.getBlue( labels[ i ] ) + ")" );
46      channels = new boolean[3];
47      channels[ 0 ] = false;
48      channels[ 1 ] = false;
49      channels[ 2 ] = false;
50      // add label image with corresponding color as an isosurface
51      univ.addContent( labelImp, color, "label-"+labels[i], 0, channels, 2, ContentConstants
            .SURFACE);
52    }
53  }
54
55  // launch smooth control
56  sc = new SmoothControl( univ );
```

At the end of the script a dialog is shown to smooth the surfaces at will. Each label is added to the 3D scene independently with the name "label-X" where X is its label value.

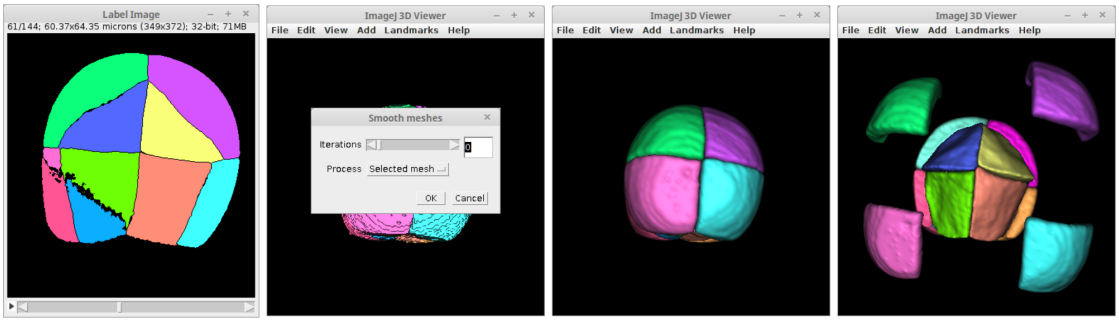


Figure 10.1.: *From left to right: input label image, script output, smoothed label surfaces and example of individually translated surfaces in the 3D viewer.*

Index

A
attribute filtering, 32

B
boundary, 18, 97

C
catchment basin, 35
closing, 17
connected component algorithm, 84
connectivity, 13, 54, 84
convex hull, 58, 85

D
dilation, 16
    conditional, 26
    grayscale, 16
    label map, 93
directional filtering, 20, 23
distance transform, 47, 75
    chamfer, 76
    Euclidean, 75
    geodesic, 81
    label image, 79

E
ellipse, 55
erosion, 16
    grayscale, 17
    label map, 93
Euler number, 54, 67
extended maxima, 30
extended minima, 30

F
fill holes, 26

G
geodesic distance transform, 81
gradient, 18, 37

I
imposition of minima/maxima, 30
influence zone, 94
inhomogeneous background, 19

K
kill borders, 26

O
opening, 18
    size, 32

R
reconstruction, 26, 30
region adjacency graph, 96
region analysis, 52
    3D, 65
regional maxima, 29
regional minima, 29

S
size opening, 32
skeleton, 64
    label map, 94

T
top-hat, 19, 32
tortuosity, 59

W
watershed, 34
    interactive, 38
    marker-controlled, 37
    morphological segmentation, 42

Plugins Index

A
Analyze Regions, 59
Analyze Regions 3D, 70
Area Opening, 86
Assign Measure To Label, 91
Average Thickness, 63

B
Binarize Image, 86
Binary Overlay, 86
Binary/Labels Overlay, 89
Bounding Box, 60

C
Chamfer Distance Map, 80
Chamfer Distance Map 3D, 80
Classic Watershed, 35
Connected Components Labeling, 85
Convexify, 85
Crop Label, 95

D
Directional Filtering, 23
Draw Labels As Overlay, 88
Draw Table Values, 91

E
Equivalent Ellipse, 61
Euclidean Distance Map, 80
Euclidean Distance Map 3D, 80
Extend Image Borders, 24
Extended Min / Max, 31
Extended Min / Max 3D, 31

F
Fill Holes, 29
Fill Label Holes, 93

G
Geodesic Diameter, 63
Geodesic Distance Map, 82
Geodesic Distance Map 3D, 82
Gray Scale Attribute Filtering, 33
Gray Scale Attribute Filtering 3D, 33

I
Impose Min / Max, 31
Impose Min / Max 3D, 31
Intensity Measurements 2D/3D, 70
Interactive Geodesic Distance Map, 82
Interactive Marker-controlled Watershed, 39
Interactive Morphological Reconstruction, 28
Interactive Morphological Reconstruction 3D, 28
Interface Surface Area, 70

K
Keep Largest Label, 92
Keep Largest Region, 85
Kill Borders, 29

L
Label Boundaries, 97
Label Edition, 99
Label Map To ROIs, 90
Label Morphological Filtering, 93
Label Overlap Measures, 71
Label Size Filtering, 92
Labels To RGB, 88

M
Marker-controlled Watershed, 37
Max Feret Diameter, 61

Max. Inscribed Circle, 63
Microstructure 3D, 73
Microstructure Analysis, 73
Morphological Filters, 22
Morphological Filters (3D), 23
Morphological Reconstruction, 28
Morphological Reconstruction 3D, 28
Morphological Segmentation, 42

O
Oriented Bounding Box, 62

R
Region Adjacency Graph, 96
Region Boundaries Labeling, 97
Region Influence Zones, 94
Regional Min / Max, 31
Regional Min / Max 3D, 31
Remap Labels, 95
Remove Border Labels, 92
Remove Largest Label, 92
Remove Largest Region, 86
Replace Value, 95

S
Select Label(s), 92
Select Region Neighbors, 98
Set Label Map, 88
Size Opening 2D/3D, 86
Skeleton Geodesic Diameter, 64
Skeletonize Labels, 94

Bibliography

Arganda-Carreras, I. & Andrey, P. (2017). *Designing Image Analysis Pipelines in Light Microscopy: A Rational Approach*, (pp. 185–207). Springer New York: New York, NY.

Borgefors, G. (1984). Distance transformation in arbitrary dimensions. *Comp. Vis. Graph. Im. Proc.*, 27, 321–145.

Borgefors, G. (1986). Distance transformations in digital images. *Comp. Vis. Graph. Im. Proc.*, 34, 344–371.

Breen, E. J. & Jones, R. (1996). Attribute opening, thinnings, and granulometries. *Computer Vision and Image Understanding*, 64(3), 377–389.

Burger, W. & Burge, M. J. (2008). *Digital Image Processing, An algorithmic introduction using Java*. Springer.

Chang, F., Chen, C.-J., & Lu, C.-J. (2004). A linear-time component-labeling algorithm using contour tracing technique. *Computer Vision and Image Understanding*, 93(2), 206–220.

Cuisenaire, O. (1999). *Distance transformations: fast algorithms and applications to medical image processing*. PhD thesis, Université Catholique de Louvain.

Das, P. P. & Chatterji, B. N. (1988). Knight's distance in digital geometry. *Pattern Recognition Letters*, 7(4), 215–226.

Devaux, M.-F. & Legland, D. (2014). *Microscopy: advances in scientific research and education*, chapter Grey level granulometry for histological image analysis of plant tissues, (pp. 681–688). Formatex Research Center.

Doube, M., Kłosowski, M. M., Arganda-Carreras, I., Cordelières, F. P., Dougherty, R. P., Jackson, J. S., Schmid, B., Hutchinson, J. R., & Shefelbinea, S. J. (2010). BoneJ: free and extensible bone image analysis in ImageJ. *Bone*, 47(6), 1076–1079.

Fabbri, R., Bruno, O. M., Costa, L. d. F., & Torelli, J. C. (2008). 2D euclidean distance transform algorithms: a comparative survey. *ACM Computing Surveys*, 40, 1–44.

Florindo, J. B., Bruno, O. M., Rossatto, D. R., Kolb, R. M., Gómez, M. C., & Landini, G. (2016). Identifying plant species using architectural features in leaf microscopy images. *Botany*, 94(1), 15–21.

Glasbey, C., van der Heijden, G., Toh, V. F. K., & et al. (2007). Colour displays for categorical images. *Color Research and Application*.

Heneghan, C., Flynn, J., Keefe, M. O., & Cahill, M. (2002). Characterization of changes in blood vessel width and tortuosity in retinopathy of prematurity using image analysis. *Medical Image Analysis*, 6(4), 407 – 429.

Kong, T. Y. & Rosenfeld, A. (1989). Digital topology: Introduction and survey. *Computer Vision, Graphics, and Image Processing*, 48(3), 357–393.

Kovesi, P. (2015). Good colour maps: how to design them.

Lang, C., Ohser, J., & Hilfer, R. (2001). On the analysis of spatial binary images. *Journal of Microscopy*, 203(3), 303–313.

Lantuéjoul, C. & Beucher, S. (1981). On the use of geodesic metric in image analysis. *Journal of Microscopy*, 121(1), 39–40.

Legland, D., Arganda-Carreras, I., & Andrey, P. (2016). MorphoLibJ: integrated library and plugins for mathematical morphology with ImageJ. *Bioinformatics*, 32(22), 3532–3534.

Legland, D. & Devaux, M.-F. (2009). Détection semi-automatique de cellules de fruits charnus observés par microscopie confocale 2d et 3d. *Cahiers Techniques de l'INRA*, numéro spécial imagerie, 7–16.

Legland, D., Kiêu, K., & Devaux, M.-F. (2007). Computation of Minkowski measures on 2D and 3D binary images. *Image Analysis and Stereology*, 26(6), 83–92.

Lehmann, G. & Legland, D. (2012). *Efficient N-Dimensional surface estimation using Crofton formula and run-length encoding*. Technical report.

Luengo Hendriks, C. L. & van Vliet, L. J. (2003). Discrete morphology with line structuring elements. In M. W. N. Petkov (Ed.), *Computer Analysis of Images and Patterns - CAIP 2003 (Proc. 10th Int. Conf., Groningen, NL, Aug.25-27)*, volume 2756 of *Lecture Notes in Computer Science* (pp. 722–729).: Springer Verlag, Berlin.

Maurer, C. R., Rensheng, Q., & Raghavan, V. (2003). A linear time algorithm for computing exact Euclidean distance transforms of binary images in arbitrary dimensions. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 25(2), 265–270.

Meyer, F. & Beucher, S. (1990). Morphological segmentation. *Journal of Visual Communication and Image Representation*, 1(1), 21–46.

Moran, P. A. P. (1966). Measuring the length of a curve. *Biometrika*, 53(3), 359–364.

Ohser, J. & Schladitz, K. (2009). *3D Images of Materials Structures: processing and analysis*. WILEY-VCH Verlag GmbH & Co. KGaA, Weinheim.

Poulet, A., Arganda-Carreras, I., Legland, D., Probst, A. V., Andrey, P., & Tatout, C. (2015). NucleusJ: an ImageJ plugin for quantifying 3D images of interphase nuclei. *Bioinformatics*, 31(7), 1144–1146.

Robinson, K. & Whelan, P. F. (2004). Efficient morphological reconstruction: a downhill filter. *Pattern Recognition Letters*, 25(15), 1759 – 1767.

Rosenfeld, A. (1970). Connectivity in digital pictures. *J. ACM*, 17(1), 146–160.

Rosenfeld, A. & Pfalz, J. L. (1968). Distance functions on digital pictures. *Pattern Recognition*, 1, 33–61.

Saito, T. & Toriwaki, J.-I. (1994). New algorithms for euclidean distance transformation of an n-dimensional digitized picture with applications. *Pattern Recognition*, 27(11), 1551–1565.

Schindelin, J., Arganda-Carreras, I., Frise, E., Kaynig, V., Longair, M., Pietzsch, T., Preibisch, S., Rueden, C., Saalfeld, S., Schmid, B., et al. (2012). Fiji: an open-source platform for biological-image analysis. *Nature methods*, 9(7), 676–682.

Schneider, C. A., Rasband, W. S., Eliceiri, K. W., Schindelin, J., Arganda-Carreras, I., Frise, E., Kaynig, V., Longair, M., Pietzsch, T., Preibisch, S., et al. (2012). NIH Image to ImageJ: 25 years of image analysis. *Nature methods*, 9(7).

Serra, J. (1982). *Image Analysis and Mathematical Morphology. Volume 1*. London: Academic Press.

Serra, J. & Vincent, L. (1992). An overview of morphological filtering. *Circuits, Systems and Signal Processing*, 11(1), 47–108.

Shoemake, K. (1994). Euler Angle Conversion. In P. S. Heckbert (Ed.), *Graphics Gems* (pp. 222 – 229). Academic Press.

Silva, J. V., Legland, D., Cauty, C., Kolotuev, I., & Floury, J. (2015). Characterization of the microstructure of dairy systems using automated image analysis. *Food Hydrocolloids*, 44(0), 360 – 371.

Slabaugh, G. (1999). Computing Euler Angles from a Rotation Matrix.

Soille, P. (2003). *Morphological Image Analysis*. Springer, 2nd edition.

Soille, P. & Talbot, H. (2001). Directional morphological filtering. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 23(11), 1313–1329.

Soille, P. & Vincent, L. M. (1990). Determining watersheds in digital pictures via flooding simulations. In *Lausanne-DL tentative* (pp. 240–250).: International Society for Optics and Photonics.

Staal, J., Abramoff, M., Niemeijer, M., Viergever, M., & van Ginneken, B. (2004). Ridge based vessel segmentation in color images of the retina. *IEEE Transactions on Medical Imaging*, 23, 501–509.

Svensson, S. & Borgefors, G. (2002a). Digital distance transforms in 3d images using information from neighbourhoods up to 5 x 5 x 5. *Computer Vision and Image Understanding*, 88(1), 24 – 53.

Svensson, S. & Borgefors, G. (2002b). Distance transforms in 3D using four different weights. *Pattern Recognition Letters*, 23, 1407–1418.

Tustison, N. & Gee, J. (2009). Introducing dice, jaccard, and other label overlap measures to itk. *Insight J*, (pp. 1–4).

Verwer, B. J. H., Verbeek, P. W., & Dekker, S. T. (1989). An efficient uniform cost algorithm applied to distance transforms. *IEEE Trans. Pattern Anal. Mach. Intell.*, 11, 425–429.

Vincent, L. (1993). Morphological Grayscale Reconstruction in Image analysis: Applications and Efficient Algorithm. *IEEE Transaction on Image Processing*, 2(2), 176–201.

Vincent, L. & Soille, P. (1991). Watersheds in digital spaces : An efficient algorithm based on immersion simulation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 13(6), 583–598.

Wang, J. & Tan, Y. (2013). Efficient Euclidean distance transform algorithm of binary images in arbitrary dimensions. *Pattern Recognition*, 46(1), 230 – 242.

A. List of operators

Plugin Name	bin.	8	16	32	RGB	2D	3D	GUI	Page
Image filtering									
Morphological filters	*	*	*	*	*	*		*	22
Morphological filters (3D)	*	*	*	*	*		*	*	23
Directional filters	*	*	*	*		*		*	23
Grayscale Attribute Filtering	*	*	*	*		*		*	32
Grayscale Attribute Filtering 3D	*	*	*	*			*	*	32
Morphological Reconstruction	*	*	*	*		*		*	26
Morphological Reconstruction (3D)	*	*	*	*			*	*	26
Interactive Morphological Reconstruction	*	*	*	*		*		*	28
Interactive Morphological Reconstruction 3D	*	*	*	*			*	*	28
Kill Borders	*	*	*	*		*	*		27
Fill Holes (Binary / Gray)	*	*	*	*		*	*		27
Segmentation									
Classic Watershed		*	*	*		*	*	*	35
Marker-controlled Watershed		*	*	*		*	*	*	37
Interactive Marker-controlled Watershed		*	*	*		*	*	*	38
Morphological Segmentation		*	*	*		*	*	*	42
Minima and Maxima									
Regional Min & Max	*	*	*	*		*		*	29
Regional Min & Max 3D	*	*	*	*			*	*	29
Extended Min & Max	*	*	*	*		*		*	30
Extended Min & Max 3D	*	*	*	*			*	*	30
Impose Min & Max	*	*	*	*		*		*	30
Impose Min & Max 3D	*	*	*	*			*	*	30
Utilities									
Extend Borders	*	*	*	*	*	*	*	*	
Binary Overlay	*	*	*	*	n.a.	*	*	*	
Binary/Labels Overlay	*	*	*	*	n.a.	*	*	*	89
Draw Label Values	*	*	*	*	n.a.	*			88

Table A.1.: *Image processing operators for grayscale and intensity images.*

Plugin Name	bin.	8	16	32	RGB	2D	3D	GUI	Page
Binary Images									
Connected Components Labeling	*	n.a.	n.a.	n.a.	n.a.	*	*	*	84
Chamfer Distance Map	*	*	*	*	n.a.	*		*	80
Chamfer Distance Map 3D	*	*	*	*	n.a.		*	*	80
Geodesic Distance Map	*	*	*	*	n.a.	*		*	82
Interactive Geodesic Distance Map	*	n.a.	n.a.	n.a.	n.a.	*		*	82
Geodesic Distance Map 3D	*	*	*	*	n.a.		*	*	82
Distance Transform Watershed	*	n.a.	n.a.	n.a.	n.a.	*		*	47
Distance Transform Watershed 3D	*	n.a.	n.a.	n.a.	n.a.		*	*	49
Convexify	*	n.a.	n.a.	n.a.	n.a.	*			85
Remove Largest Region	*	n.a.	n.a.	n.a.	n.a.	*	*		85
Keep Largest Region	*	n.a.	n.a.	n.a.	n.a.	*	*		85
Area Opening	*	n.a.	n.a.	n.a.	n.a.	*		*	86
Size Opening 2D/3D	*	n.a.	n.a.	n.a.	n.a.	*	*	*	86
Label Images									
Set Label Map	n.a.	*	*	*	n.a.	*	*	*	88
Labels To RGB	n.a.	*	*	*	n.a.	*	*	*	88
Draw Labels As Overlay	n.a.	*	*	*	n.a.	*			88
Assign Measure To Label	*	*	*	*	n.a.	*	*	*	88
Label Edition	n.a.	*	*	*	n.a.	*	*	*	99
Label Morphological Filters	*	*	*	*	n.a.	*	*	*	93
Remap Labels	*	*	*	*	n.a.	*	*		95
Fill Label Holes	*	*	*	*	n.a.	*	*		95
Expand Labels	n.a.	*	*	*	n.a.	*	*	*	95
Remove Border Labels	n.a.	*	*	*	n.a.	*	*	*	92
Replace / Remove Label(s)	n.a.	*	*	*	n.a.	*	*	*	92
Merge Labels	n.a.	*	*	*	n.a.	*	*	*	
Select Label(s)	n.a.	*	*	*	n.a.	*	*	*	92
Crop Label	n.a.	*	*	*	n.a.	*	*	*	95
Remove Largest Label	*	*	*	*	n.a.	*	*		95
Keep Largest Label	*	*	*	*	n.a.	*	*		95
Label Size Filtering	*	*	*	*	n.a.	*	*	*	92
Region Adjacency Graph	n.a.	*	*	*	n.a.	*	*		96
Region Boundaries Labeling	n.a.	*	*	*	n.a.	*	*		96
Label Boundaries	n.a.	*	*	*	n.a.	*	*		96
Select Neighbor Labels	n.a.	*	*	*	n.a.	*	*	*	98

Table A.2.: *Image processing operators for binary and label images.*

B. Computation of equivalent ellipsoid coefficients

It is often convenient to represent the results of second order moments (section 7.2) as an **equivalent ellipsoid** that has the same orientation and the same eigen values as the structure of interest. The equivalent ellipsoid can be completely characterised by nine coefficients:

- the three coordinates of the centroid
- the three radius r_1, r_2, r_3, with $r_1 > r_2 > r_3$, corresponding to the lengths of each semi-axis
- a 3D orientation, that can be represented by three Euler angles.

B.1. Notations

We consider euclidean spaces of dimension 3. Let X be the a 3D set representing the structure of interest.

B.1.1. Moments

In 3D, the **moments** m_{pqr} of order (p,q,r) are defined as:

$$m_{pqr} = \int\int\int I_X(x,y,z) x^p y^q z^r \cdot dx \cdot dy \cdot dz \tag{B.1}$$

where $I_X(x,y,z)$ is the indicator function of the set X, taking value 1 if the specified point is within the set X, and 0 otherwise. The moment m_{000} corresponds to the **volume** of the structure. The **centered moments** are expressed as:

$$\mu_{pqr} = \int\int\int I_X(x,y,z) (x-x_c)^p (y-y_c)^q (z-z_c)^r \cdot dx \cdot dy \cdot dz \tag{B.2}$$

where $(x_c, y_c, z_c) = (\frac{m_{100}}{m_{000}}, \frac{m_{010}}{m_{000}}, \frac{m_{001}}{m_{000}})$ is the **centroid** of X. The matrix of second order moments is the symmetric 3×3 matrix that combines all the centered second order moments:

$$\boldsymbol{M} = \begin{bmatrix} \mu_{200} & \mu_{110} & \mu_{101} \\ \mu_{110} & \mu_{020} & \mu_{011} \\ \mu_{101} & \mu_{011} & \mu_{002} \end{bmatrix}$$

B.1.2. Eigen values decomposition

The matrix $\boldsymbol{M}$ can be factorized using eigen value decomposition:

$$\boldsymbol{M} = \boldsymbol{Q}\boldsymbol{\Lambda}\boldsymbol{Q}^t$$

where

- $\boldsymbol{Q}$ is a 3×3 orthogonal matrix whose i-th column is the eigenvector q_i of $\boldsymbol{M}$
- $\boldsymbol{\Lambda}$ is a diagonal 3×3 matrix whose diagonal elements are the corresponding eigen values, $\Lambda_{ii} = \lambda_i$.

Eigen values are ordered in decreasing order. The matrix $\boldsymbol{Q}$ can be used to extract the orientation of the main axes of the structure (section B.2), whereas the eigen values λ_i can be related to the particle dimensions along these axes (section B.3).

B.2. Computation of angles

This section largely follows the document of Greg Slabaugh (Slabaugh, 1999) for obtaining Euler angles from a rotation matrix. We mostly follow his notations. Another useful reference may be Shoemake (1994).

B.2.1. Rotation matrices

A rotation of ψ radians about the x-axis is noted by:

$$\boldsymbol{R}_x(\psi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\psi & -\sin\psi \\ 0 & \sin\psi & \cos\psi \end{bmatrix}$$

Similarily, a rotation of θ radians about the y-axis is noted by:

$$\boldsymbol{R}_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix}$$

Finally, a rotation of φ radians about the z-axis is noted by:

$$\boldsymbol{R}_z(\varphi) = \begin{bmatrix} \cos\varphi & -\sin\varphi & 0 \\ \sin\varphi & \cos\varphi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

A general rotation matrix may have the following form:

$$\boldsymbol{R} = \begin{bmatrix} R_{11} & R_{12} & R_{13} \\ R_{21} & R_{22} & R_{23} \\ R_{31} & R_{32} & R_{33} \end{bmatrix}$$

Such a matrix can be represented by a sequence of three successive rotations around the main axes. As matrix multiplication does not commute, the order of the axes one rotates about will affect the result. In MorphoLibJ, we follow the choice of G. Slabaugh and consider rotation first about the x-axis, then about the y-axis, and finally about the z-axis. Then, the angles ψ, θ and φ correspond to Euler angles. Angle ψ corresponds to the "roll", angle θ to the "pitch", and angle φ to the "yaw".

The global rotation matrix can be written as follow:

$$\begin{aligned}\boldsymbol{R} &= \boldsymbol{R}_z(\varphi)\cdot\boldsymbol{R}_y(\theta)\cdot\boldsymbol{R}_x(\psi)\\ &= \begin{bmatrix} \cos\theta\cos\varphi & \sin\psi\sin\theta\cos\varphi-\cos\psi\sin\varphi & \cos\psi\sin\theta\cos\varphi+\sin\psi\sin\varphi \\ \cos\theta\sin\varphi & \sin\psi\sin\theta\sin\varphi+\cos\psi\cos\varphi & \cos\psi\sin\theta\sin\varphi-\sin\psi\cos\varphi \\ -\sin\theta & \sin\psi\cos\theta & \cos\psi\cos\theta \end{bmatrix}\end{aligned}$$

The problem is now to identify the three Euler angles ψ, θ and φ from the matrix coefficients. This results in nine equations.

B.2.2. Elevation

We start considering the elevation θ, or "pitch". From element R_{31} of the matrix, one finds

$$R_{31} = -\sin\theta$$

One identifies θ with the following

$$\theta = -\sin^{-1}(R_{31})$$

by keeping the value of $\theta \in [-\frac{\pi}{2};\frac{\pi}{2}]$ to remove ambiguity. The case $R_{31} = \pm 1$ will be considered later. In practice, we replace the $\sin^{-1}$ function by the atan2 function for better numerical stability.

Compared to the document of Slabaugh, keeping $\theta \in [-\frac{\pi}{2};\frac{\pi}{2}]$ removes the sign ambiguity of $\cos\theta$, and therefore simplifies the remaining computations.

B.2.3. Roll

The possible values of the roll ψ around the x-axis can be found by

$$\frac{R_{32}}{R_{33}} = \tan\psi$$

This lead to the value of ψ as

$$\psi = \text{atan2}(R_{32}, R_{33})$$

where $\text{atan2}(y,x)$ is the arc tangent function of the two variables y and x that extends the atan function to the all four quadrants. The function atan2 is available in most programming languages.

B.2.4. Azimut

The azimut φ, or "yaw", can be obtained from

$$\frac{R_{21}}{R_{11}} = \tan\varphi$$
$$\varphi = \text{atan2}(R_{21}, R_{11})$$

B.2.5. Special case of $\cos\theta = 0$

When the matrix element $R_{31} = \pm 1$, we have $\theta = \pm\pi/2$: the main axis of the ellipsoid is aligned with the z-axis. Developping the terms R_{12}, R_{13}, R_{22}, and R_{23} shows that the values of ψ and φ are linked, and an infinite number of solutions exists (Slabaugh, 1999). This phenomenon is called *Gimbal lock*. A convenient way to obtain a single valid solution is to set the value for the azimut φ arbitrarily to 0, and compute ψ from R_{12} and R_{13}.

If $\theta = \pi/2$ then

$$R_{12} = \sin\psi\cos\varphi - \cos\psi\sin\varphi = \sin(\psi - \varphi)$$
$$R_{13} = \cos\psi\cos\varphi - \sin\psi\sin\varphi = \cos(\psi - \varphi)$$
$$R_{22} = \sin\psi\sin\varphi - \cos\psi\cos\varphi = \cos(\psi - \varphi) = R_{13}$$
$$R_{23} = \cos\psi\sin\varphi - \sin\psi\cos\varphi = -\sin(\psi - \varphi) = -R_{12}$$

leading to

$$\psi = \varphi + \text{atan2}(R_{12}, R_{13})$$

If $\theta = -\pi/2$, similar development leads to:

$$\psi = -\varphi + \text{atan2}(-R_{12}, -R_{13})$$

B.3. Computation of ellipsoid radius

Let us suppose that the ellipsoid is centered and aligned with the main axes. From centering, we have $m_{pqr} = \mu_{pqr}$. For integration, it is more convenient to use spherical coordinates (ρ, θ, φ), corresponding to the distance to the origin, to the inclination with the vertical, and to the azimuth (these notations are different from the ones used for Euler angles). This corresponds to:

$$x = \rho \cdot r_1 \cdot \cos\varphi \cdot \sin\theta$$
$$y = \rho \cdot r_2 \cdot \sin\varphi \cdot \sin\theta$$
$$z = \rho \cdot r_3 \cdot \cos\theta$$

with $\rho \in [0; +\infty[$, $\varphi \in [0; 2\pi]$ and $\theta \in [0; \pi]$.

The Jacobian matrix of the transformation is as follow (see example 3 in "Jacobian matrix and determinant" on Wikipedia[1]):

$$J_\Phi(x,y,z) = \begin{bmatrix} \frac{\partial x}{\partial \rho} & \frac{\partial x}{\partial \theta} & \frac{\partial x}{\partial \varphi} \\ \frac{\partial y}{\partial \rho} & \frac{\partial y}{\partial \theta} & \frac{\partial y}{\partial \varphi} \\ \frac{\partial z}{\partial \rho} & \frac{\partial z}{\partial \theta} & \frac{\partial z}{\partial \varphi} \end{bmatrix} = \begin{bmatrix} r_1 \cos\varphi \sin\theta & \rho r_1 \cos\varphi \cos\theta & -\rho r_1 \sin\varphi \sin\theta \\ r_2 \sin\varphi \sin\theta & \rho r_2 \sin\varphi \cos\theta & \rho r_2 \cos\varphi \sin\theta \\ r_2 \cos\theta & -\rho r_3 \sin\theta & 0 \end{bmatrix}$$

The determinant equals $\rho^2 \sin\theta$ if $r_1 = r_2 = r_3 = 1$. Introducing a, b and c gives a determinant equal to $r_1 r_2 r_3 \rho^2 \sin\theta$. Then the moment integral is given by:

$$\mu_{pqr} = r_1 r_2 r_3 \int_0^{2\pi} \int_0^{\pi} \int_0^{1} (\rho \cos\varphi \sin\theta)^p (\rho \sin\varphi \sin\theta)^q (\rho \cos\theta)^r \cdot \rho^2 \sin\theta \cdot d\rho \cdot d\theta \cdot d\varphi \tag{B.3}$$

B.3.1. Volume moment

It is easy to check that the moment m_{000} corresponds to the volume of the ellipsoid (equal to $\frac{4\pi}{3} r_1 r_2 r_3$):

$$\begin{aligned} m_{000} &= r_1 r_2 r_3 \int_0^{2\pi} \int_0^{\pi} \int_0^{1} \rho^2 \sin\theta \cdot d\rho \cdot d\theta \cdot d\varphi \\ &= \frac{r_1 r_2 r_3}{3} \int_0^{2\pi} \int_0^{\pi} \sin\theta \cdot d\theta \cdot d\varphi = \frac{abc}{3} \int_0^{2\pi} (-\cos\pi + \cos 0) \cdot d\varphi \\ &= \frac{2}{3} r_1 r_2 r_3 \int_0^{2\pi} d\varphi = \frac{4\pi}{3} r_1 r_2 r_3 \end{aligned}$$

B.3.2. Moment m200

When considering centered ellipsoid aligned with principal axes, the eigen values λ_i simply correspond to the moments μ_{200}, μ_{020} and μ_{002}. Let us focus on the moment μ_{200}:

$$\begin{aligned} \mu_{200} &= r_1 r_2 r_3 \int_0^{2\pi} \int_0^{\pi} \int_0^{1} (r_1 \rho \cos\varphi \sin\theta)^2 \rho^2 \sin\theta \cdot d\rho \cdot d\theta \cdot d\varphi \\ &= r_1^3 r_2 r_3 \int_0^{2\pi} \int_0^{\pi} \int_0^{1} \rho^4 d\rho \cdot \cos^2\varphi \sin^3\theta \cdot d\theta \cdot d\varphi \\ &= \frac{r_1^3 r_2 r_3}{5} \int_0^{2\pi} \int_0^{\pi} \sin^3\theta \cdot d\theta \cdot \cos^2\varphi \cdot d\varphi \end{aligned}$$

[1] https://en.wikipedia.org/wiki/Jacobian_matrix_and_determinant#Example_3:_spherical-Cartesian_transformation

The integral over θ can be developed using the linearisation of $\sin^3\theta = \frac{3}{4}\sin\theta - \frac{1}{4}\sin(3\theta)$:

$$\begin{aligned}\int_0^{\pi} \sin^3\theta\, d\theta &= \int_0^{\pi} \left(\frac{3}{4}\sin\theta - \frac{1}{4}\sin(3\theta)\right) d\theta \\ &= -\frac{3}{4}(\cos\pi - \cos 0) - \frac{-1}{12}(\cos 3\pi - \cos 0) \\ &= \frac{3}{2} - \frac{1}{6} = \frac{4}{3}\end{aligned}$$

Coming back to moment integral:

$$\begin{aligned}\mu_{200} &= \frac{r_1^3 r_2 r_3}{5}\frac{4}{3}\int_0^{2\pi} \cos^2\varphi \cdot d\varphi == \frac{r_1^3 r_2 r_3}{5}\frac{4}{3}\int_0^{2\pi} \frac{1}{2} + \frac{1}{2}\cos(2\varphi)\, d\varphi \\ &= \frac{r_1^3 r_2 r_3}{5}\frac{4}{3}\frac{1}{2}(2\pi - 0) = \frac{4\pi}{3}\frac{r_1^3 r_2 r_3}{5} \\ &= \frac{r_1^2}{5} m_{000}\end{aligned}$$

Then, the central moments are expressed as:

$$\begin{aligned}\lambda_1 &= \mu_{200} = \frac{r_1^2}{5} m_{000} \\ \lambda_2 &= \mu_{020} = \frac{r_2^2}{5} m_{000} \\ \lambda_3 &= \mu_{002} = \frac{r_3^2}{5} m_{000}\end{aligned}$$

We therefore have $\lambda_i = \frac{r_i^2}{5} m_{000}$, leading to:

$$r_i = \sqrt{\frac{5 \cdot \lambda_i}{m_{000}}} \tag{B.4}$$